Measuring whether a code review finds the real defects:
a verified defect benchmark on AACR-Bench, applied to eight reviewers at their best-shot configurations

Mateusz Jakubczak
Quickchat AI
mateusz@quickchat.ai

Draft of 2026-10-04

Abstract

Code-review benchmarks usually score a generated review by how closely it reproduces the comments the original pull request received. We show that on AACR-Bench v1.0, a 200-PR, ten-language benchmark [2], this answer key measures something other than defect finding: on 145 PRs checked in full, only 12.8% of the reference comments belong to a real defect; at least 80% of the references were written by AI reviewers, including at least 80 "human" references that are rewritten review-bot comments; and only 31% of the real defects we verified appear among the references at all. We build a replacement for the 185 runnable PRs: an answer key of 520 defects (90 high, 145 medium, 285 low severity), each verified against the code by LLM agents that could not see who raised it, with 3,093 rejected candidates kept with their reasons; and a judge that reads the whole review and grades every defect as found, partial or missed. On an earlier set of 1,295 reviews, judges from two model families agree on 98.7% of 3,640 verdicts (Cohen's $\kappa = 0.97$); neither the key nor the judge has yet been checked by a human. We also define a best-shot protocol that gives each compared tool the strongest configuration its documentation ships or recommends, picked by a rule fixed in advance. Applied to eight reviewers (seven on gpt-6.1-sol; Claude Code on Claude Opus 5.5), the benchmark spreads them from 37% to 79% of the key's defects, where AACR-Bench's own metric places five of them within 1.8 points of each other.

The authors build one of the reviewers, QuickReview, and design it for recall: to find as many of a change's real problems as possible, the serious ones first. Its frozen build finds 97% of the high-severity defects (87.5 of 90, a partial find counting half) and 89% of all 520, ahead of every other reviewer at every severity; the strongest third-party configuration finds 89% and 67%. On Martian's offline benchmark it finds 87% of the golden comments an audit judged real, and on Qodo's PR-Review-Bench 52% of the functional bugs, the highest recall of the tools each benchmark scores under one judge (45% of all Qodo's issues, second to 46%), and ahead of the same three third-party harnesses on the same model on both (they find 54–78% of the real goldens and 44–46% of the functional bugs). The price is volume, cost and time: it posts 6.9 comments per PR, 92% of them correct but 42% minor, which gives it the lowest Martian F1 and the fourth-highest Qodo F1; it costs $2.06 per PR at list prices (median $1.87; $2.84 and $2.48 on Martian's and Qodo's larger PRs), 51% more than the build of the eight-reviewer comparison and 4 to 29 times the third-party tools, and takes a median 17.5 minutes per PR, 1.8 times that build, against 0.7–2.3 minutes for the others. The key, the exclusions, the judge prompt and every scoring script are released.

Contents

1 Introduction **4**

2 Related work **5**

3 AACR-Bench and its reference key **8**
3.1 The dataset and its metric . . . 8
3.2 What the reference list is . . . 9

4 Design principles **10**

5 The defect key **12**
5.1 What counts as a defect . . . 12
5.2 Why a verified candidate pool, not a blind hunt . . . 12
5.3 Pipeline for R145 . . . 13
5.4 Extension with four more reviewers (v2) . . . 14
5.5 T40 . . . 14
5.6 The final key . . . 15
5.7 Who built the key . . . 15

6 The defect judge **16**
6.1 What the judge reads and returns . . . 16
6.2 Judge reliability . . . 16
6.3 AACR's metric as we ran it . . . 18

7 Reviewers and the best-shot protocol **18**
7.1 Choosing each third-party tool's configuration . . . 19
7.2 Deviations per tool, and their direction . . . 22
7.3 Execution and cost accounting . . . 23

8 Results: the final comparison **24**
8.1 Defects found . . . 24
8.2 What the comments are . . . 25
8.3 By language, kind and size . . . 26
8.4 AACR-Bench's own metric . . . 27
8.5 Cost, tokens and time . . . 27
8.6 Tuned and unseen PRs; the reviewer-independent subset . . . 28
8.7 The frozen build . . . 30
8.8 What the key cannot credit: defects outside the pool . . . 31
8.9 Worked examples . . . 31

9 Robustness checks from the earlier study **36**
9.1 Stability of QuickReview default . . . 37
9.2 Seeding bias: independent-source and leave-one-reviewer-out keys . . . 38
9.3 Copilot code review, 2025, as deployed upstream: an anecdote . . . 39

10 Why the two metrics disagree **41**

11 External check: Martian's offline benchmark **42**
11.1 How we ran it . . . 42
11.2 The score as Martian computes it . . . 43
11.3 What the golden comments are . . . 44
11.4 The real defects the golden lists lack . . . 46
11.5 What posting less does to the Martian score . . . 50
11.6 Where QuickReview is strong: its own severity . . . 50
11.7 Did a tool find every listed blocking problem in a PR? . . . 51
11.8 What the benchmark measures, and what it misses . . . 52
11.9 What we report . . . 53

12 External check: Qodo's PR-Review-Bench **53**

13 Threats to validity **55**
13.1 What we claim and what we do not 58

14 Reproducibility and data availability **58**

15 Conclusion **60**
15.1 Limitations and future work 60

A Prompts (summary) **67**

B Per-set tables **69**

C The earlier study: supporting tables **70**

D Chronology of result-affecting decisions **71**

1 Introduction

Automated code review is now a product category, and several vendors publish benchmarks of their own, each of which its publisher wins [26, 27, 28]. Three problems make most published numbers hard to interpret.

1. **The answer key.** A benchmark needs to know what a good review of a given pull request should say. Public benchmarks usually take this from the comments that human (or AI) reviewers left on the original PR [2, 18, 21, 20]. Those comments were never meant to be a complete or correct list of the PR's defects, and studies of such data find much of it noisy [51, 52].
2. **The matching rule.** A benchmark has to decide whether a generated comment "is" a reference comment. Rules based on file and line position penalise a reviewer that anchors the right finding a few lines away, and precision-style scores punish every true comment that is not on the list.
3. **Who runs the benchmark.** Vendors benchmark their own tools, and tune them on the benchmark. Numbers that are not separated into tuned and unseen data, that compare against competitors in weak configurations [101, 102], or that do not state how the answer key was built, cannot be checked.

We ran into all three. We build QuickReview[1], a production reviewer, and wanted to know whether it finds the problems a careful senior reviewer would raise before merge. AACR-Bench [2] is the most suitable public benchmark we found: 200 real PRs from 50 open-source repositories in ten languages, with a published, runnable harness [3]. When we scored QuickReview with it, the numbers were low for every reviewer and did not track what we saw reading the reviews: early prompt iterations raised AACR F1 by about 20% while the number of real defects found did not move (Section 10).

This paper reports the benchmark we built in response and its first full application (Figure 1). We replace the reference list with a verified defect key for all 185 runnable PRs, grade reviews with a defect-level judge checked across two model families, and compare tools with a best-shot protocol that gives each third-party tool the strongest configuration its own documentation ships or recommends. We then apply it to eight reviewers.

The headline metric. We lead with recall: the share of the key's verified defects a review finds, high-severity defects first, then all of them. That is what QuickReview is designed for — to find as many of a PR's real problems as possible — and what matters most before a merge. Precision, comment accuracy and F1 are reported beside it as the trade-off of that design, and cost and time per PR beside every result.

Conflict of interest. The authors build QuickReview, one of the eight reviewers, and it ranks first. The same AI assistant that designed QuickReview's benchmark-driven changes also orchestrated the construction of the key and wrote the judge prompt. Interested parties are known to bias evaluations in their favour even when the methods look sound: industry-sponsored studies favour their sponsor [107], and providers' selective disclosure of privately tested variants biases public leaderboards [108]. We therefore record the reasoning behind each methodological choice, including the alternatives we rejected; we measure the biases we could measure (Sections 8.6, 9); we list in Section 13 every way we know the method could favour our system; and we release the key, the exclusions, the prompts and the scoring scripts so that others can check them. QuickReview's own design is described in a companion report [1].

[1] QuickReview is the public name of the system; its code, configuration and earlier reports call it `quickreviewd`.

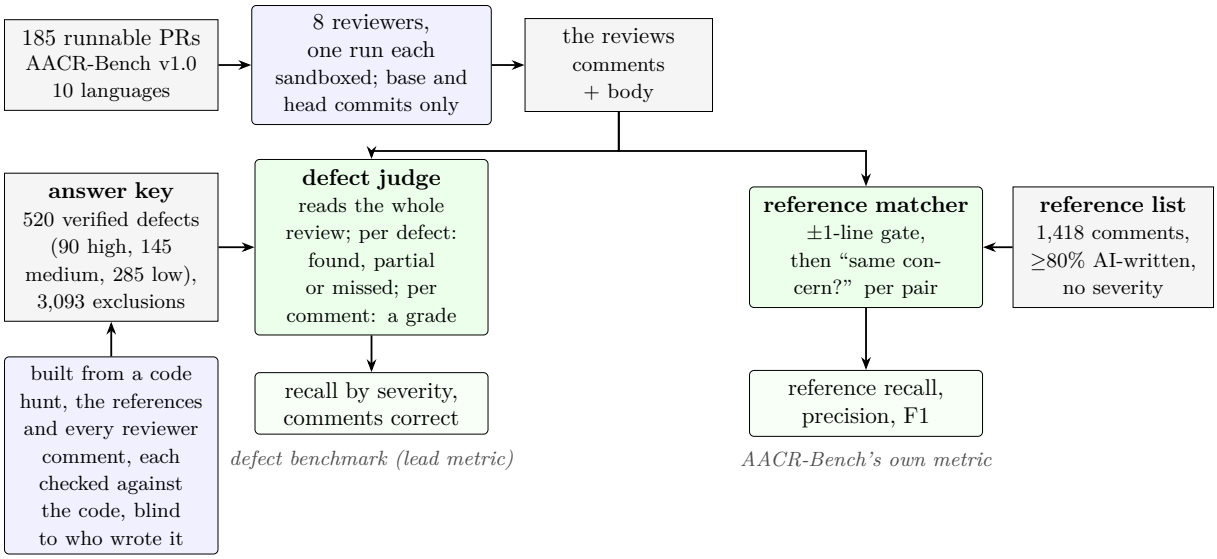


Figure 1: How the benchmark works. Every review is scored twice: by our defect judge against a verified answer key (left; the lead metric) and by AACR-Bench's own reference-comment metric (right). Section 5 builds the key, Section 6 the judge.

Contributions. What is new here, and what is not, is set out against prior work at the end of Section 2.

1. **An audit of a benchmark's answer key.** We measure what AACR-Bench's reference comments are: how few name a real defect, how few real defects they name, and how many of its "human" references are rewritten review-bot comments (Section 3).
2. **A verified defect key** for all 185 runnable AACR-Bench v1.0 PRs: 520 defects with severity, sites, evidence and matching notes, 3,093 recorded exclusions, and per-defect provenance that lets pool bias be measured (Section 5).
3. **A defect-level judge**, with cross-family agreement and self-consistency reported (Section 6).
4. **A best-shot protocol** for configuring competitor tools: the documented configurations measured on PRs no tuning touched, a selection rule fixed in advance, and the selection's optimism measured on held-out PRs (Section 7.1).
5. **An eight-reviewer comparison** under both metrics, with cost, uncertainty, bias checks, and worked examples of what the judge credits and what AACR's metric rewards (Sections 8–10).

Reading guide. Section 2 places the work; Section 3 describes the dataset and why its reference list is not an answer key; Section 4 gives the design principles; Sections 5–6 the key and the judge; Section 7 the reviewers and the best-shot protocol; Section 8 the comparison and the frozen build (Section 8.7); Section 9 the robustness checks run on an earlier study; Section 10 why the two metrics disagree; Sections 11 and 12 two external checks; Section 13 the threats.

2 Related work

What code review finds. Empirical studies of modern code review agree that finding defects is one outcome among several, and not the most frequent. At Microsoft, finding defects was the main motivation developers gave for review, but reviews turned out to be less about defects than

expected and more about knowledge transfer, team awareness and alternative solutions [12]; a later Microsoft report put the point in its title [17]. In industrial and student reviews [14] and in open-source projects [15], about 75% of the issues reviews found concerned evolvability (maintainability) rather than visible functionality. At Google, developers named education, maintaining norms, gatekeeping and accident prevention as what they expect from review [13]. Review coverage and participation are still associated with fewer post-release defects [16]. So the comments a PR received are a record of what its reviewers chose to say, not a list of its defects, and a benchmark that uses them as the answer key measures agreement with that record.

Review-comment generation, its metrics and its data. Learning-based review automation started from mined review data: models trained to produce the change or comment a human reviewer asked for [49], scored by textual overlap (BLEU) with that comment [18]. The CodeReviewer authors themselves note that review comments are "diverse and non-unique" and that BLEU is not suitable for the task [18]. Manual analyses then found the data itself noisy: issues in the datasets used to train and test these models [50], about 36% of sampled CodeReviewer training comments vague or non-actionable [51], and fewer than 10% of benchmark comments judged high quality for automation, with text similarity correlating weakly with defect detection [52]. Reference-free metrics such as CRScore [19] score a comment against model-generated pseudo-references for the claims and code smells of the change. A survey of 99 papers charts the move from these sub-tasks to end-to-end generative, increasingly PR-level review [23].

Pull-request-level review benchmarks. AACR-Bench [2, 3] already recognises that raw PR comments are a noisy, incomplete ground truth: it augments human review threads (rewritten by an LLM) with comments from six LLMs, each kept or dropped by at least two annotators, and matches generated to reference comments by line overlap and an LLM judge. Our finding is that the augmented list is still mostly not defects (Section 3). SWR-Bench [20] uses 1,000 manually verified PRs from 12 Python projects (500 with issues, 500 clean), whose ground-truth issues an LLM extracts from the review discussion and commits and annotators verify; about two-thirds of those issues are documentation, style or structure, and its LLM judge reports about 90% agreement with human judgement. ContextCRBench [21] builds 67,910 context-enriched hunk- and line-level items from mined issue–PR pairs; CodeFuse-CR-Bench [53] (601 Python instances) and SWE-PRBench [54] (350 PRs) use filtered human review comments as the key, the latter explicitly leaving correct but unmatched findings unpenalised. Martian's offline benchmark [55, 56] scores the published comments of commercial reviewers on 50 PRs against 173 golden comments with an LLM judge; we audit its key and run on it in Section 11. Other benchmarks avoid human comments as the key: c-CRAB [22] turns 234 validated human comments into LLM-generated tests that a review must lead a fixing agent to pass, keeping 234 of 1,313 comments after dropping conversational or untestable ones and those whose tests failed validation; CR-Bench [57] derives its defects from SWE-bench issues; MCR-Bench [58] reports that its LLM judge agrees with humans far better than BLEU does. Vendor benchmarks take two further routes: Greptile re-introduced 50 historical bugs into fresh PRs and counted a bug as caught only by a line comment that names it [26]; Qodo injected best-practice violations and 1–3 functional bugs into 100 merged PRs (580 issues) and matched outputs with an LLM judge that also requires the right line [27]. Each vendor's benchmark is run by the vendor, and each vendor wins its own [28].

AI reviewers in deployment. Industrial studies measure a different signal: whether developers act on a bot's comments. Reported figures range from 73.8% of comments resolved for a PR-Agent-based reviewer in an industrial deployment [24], about 40% resolution for Google's AutoCommenter [59] and 75% precision for ByteDance's BitsAI-CR [60], to 8.1% and 7.2% of comments accepted at Mozilla and Ubisoft [61]. Studies of AI review on GitHub find effective-

ness varies widely across tools [62] and that most review agents' comments are mostly noise [63]. Whether a comment is resolved reflects workflow constraints as much as comment quality, and LLM judges agree with developer labels only moderately [25]. These signals measure usefulness in a team; ours measures recall of verified defects, which is narrower and can be computed offline.

Defect benchmarks and their ground truth. Benchmarks of real bugs define the ground truth by a mechanism: Defects4J [64], BugsInPy [65] and ManyBugs [66] isolate real fixes and keep a test that exposes each bug, so only test-expressible bugs enter; BugSwarm [67] mines fail–pass CI pairs, of which a later review found only 3.6% suitable for fault-localisation and repair studies [68]. Every such mechanism has a documented error rate when checked by hand: a third of "bug" reports are not bugs [70], only half of SZZ's bug-fixing commits fix bugs [71], and 20–71% of vulnerability labels in real-world datasets are inaccurate [72]; on a relabelled, de-duplicated and time-split vulnerability dataset a model's F1 fell from 68% to 3% [73]. Label errors in machine-learning test sets likewise change which model looks best [74]. Habib and Pradel measured the recall of static bug detectors against Defects4J and found they miss most known bugs [69]; our setting is the review analogue, with a key built by verification rather than by a test oracle. SWE-bench [39] shows the life cycle of an executable key: a human-screened subset after many tasks proved underspecified [40], then audits finding solution leakage and weak tests [41, 75, 76] and memorisation of the repositories [77].

LLM judges and agreement. Using a strong model to grade open-ended output is standard practice [29, 33], with documented biases: sensitivity to the order of candidates [30, 29], a preference for longer answers [29], and a preference for a model's own generations that grows with its ability to recognise them [31]. Pooling judges from different model families is one mitigation [32]. In software engineering, LLMs approach human agreement on some annotation tasks but not others, and model–model agreement has been proposed as a signal of whether a task suits an LLM annotator [81]; LLM judges correlate with humans on code generation and translation better than on summarisation [82], and reference-free code judges outperform token-matching metrics [83, 84]. Large studies conclude that LLM judges must be validated against human judgements task by task [85, 86], and that high raw agreement can hide large score differences, so chance-corrected agreement should be reported beside it [87]. Our per-defect verdicts follow the decomposition of long-form evaluation into atomic, separately checked claims [88]. We report Cohen's κ [34], read on the usual scale [35], knowing that it can be distorted by skewed class prevalence [89] and that alpha-like coefficients are an alternative [90, 91].

Pooled answer keys. Our key is a pool: the union of what a hunt, the references and the reviewers raised, each verified. Information retrieval met the same problem when it proposed pooling for large test collections [92]: with TREC's pooled relevance judgements, relative rankings proved fairly reliable but recall is overestimated, because many relevant documents are never judged [46]; pools can be biased against systems unlike those that built them [48], which motivated measures robust to incomplete judgements [47]; and large disagreements between relevance assessors change system rankings very little [93]. LLM relevance assessors can match human labellers [95], and even with only fair label agreement can reproduce similar system rankings [94], but when the systems under test are themselves LLMs the evaluation risks circularity [96]. Our independent-source and leave-one-reviewer-out checks follow IR's practice of re-scoring a system on a pool built without its own contribution, and, like that literature, we do not claim absolute recall.

Benchmark methodology, baselines and conflicts of interest. Checklists for benchmark quality ask for documented construction, validated automatic evaluation, uncertainty and

a replication script [98, 99, 97], and warn that a benchmark measures a narrower construct than its name suggests [100]. Comparisons in machine learning have repeatedly been overturned when baselines received the tuning budget of the proposed method [101, 102, 103], and variance from data sampling and configuration choice can exceed the differences reported [104]; small test sets make many comparisons underpowered [105]. Decisions made by looking at a test set can overstate results on it [44], and so can picking the best of several configurations on the data used to report it [45]; a drop on fresh data is not by itself proof of overfitting [43]. Evaluations by an interested party have a documented bias: industry-sponsored clinical studies report favourable results more often [107], and providers' private testing of many variants biases public leaderboards [108]; in fuzzing, an SE field where tool authors evaluate their own tools, every one of 32 surveyed evaluations had problems [109], and later papers still often disregarded the resulting guidelines [110]. Agent evaluations are urged to report cost beside accuracy [111]. Public benchmarks may also have been seen in training [42, 79], which time-segmented benchmarks avoid [80, 78].

What this paper adds. We do not introduce LLM judging of review coverage: SWR-Bench, MCR-Bench and Qodo's benchmark already grade reviews against a list of issues with an LLM judge, and c-CRAB uses an executable key. Nor is configuring competitors fairly a new idea in machine learning. What we add is narrower. First, a measurement of what an existing public benchmark's answer key contains, including the finding that at least 22% of its "human" references on the 185 runnable PRs are rewritten review-bot comments. Second, a defect key for that benchmark, built by verifying every candidate against the code, with each of 3,093 rejected candidates kept with its reason and shown to the judge, and with every defect's provenance recorded so that pool bias can be measured rather than only acknowledged. Third, a defect-level judge run under two model families, with per-verdict agreement, chance-corrected agreement and self-consistency reported (but not yet human agreement). Fourth, the best-shot protocol applied to commercial and open-source review tools, with the selection's optimism measured on held-out PRs. Unlike the vendor benchmarks, we publish the key, the exclusions and the prompts; like them, we benchmark our own system, and Section 13 treats that as the main threat.

3 AACR-Bench and its reference key

3.1 The dataset and its metric

AACR-Bench v1.0 (repository `alibaba/aacr-bench`, commit `ab8af8d5`, dataset file sha256 `d8683cb2...`) holds 200 pull requests from 50 public repositories [2]. Its harness scores 196 of them (four have no positive references), which carry 1,505 reference comments, a median of six per PR. According to the paper and the dataset's metadata, 74% of the references were generated by six LLMs and kept by annotators (GPT-5.2 alone 25%); the rest are human review threads that an LLM rewrote [2]. Each reference has a *category* (code defect 47%, maintainability and readability 42%, performance 8%, security 4%) and a *context* level (diff, file or repository). References carry no severity.

Eleven of the 196 scored PRs could not be run on 2026-09-27 because GitHub no longer serves their reviewed head commit (force-pushed away): ComfyUI ×2, nodejs/node, astral-sh/uv, ClickHouse ×2 and keycloak ×5. They are excluded from every arm, leaving **185 PRs** and **1,418 references** in ten languages: C++ 31, TypeScript 29, Java 25, Go 22, Python 19, C 19, PHP 11, JavaScript 10, C# 10 and Rust 9.[2]

[2]Counted from the `language` field of `bench/aacr/sets/{smoke10,dev10,holdout10,fresh10,rest145}.json`, re-verified 2026-09-28.

Table 1: The PR sets and their roles. Selection rules per set are in Appendix B.

Name	PRs	Defects	What it is, and its role
T40	40	126	smoke10, dev10, holdout10 and fresh10: the PRs whose misses QuickReview's benchmark-driven levers were designed from (used in QuickReview's development)
R145	145	394	`rest145`, every other runnable PR; created mechanically after the last benchmark-driven change; never looked at for QuickReview's design
S40 $\subset$ R145	40	115	seeded sample of R145 on which competitor configurations were *selected* (Section 7.1)
O145	145	405	all PRs except S40 (T40 plus 105 of R145); measures the selection's optimism

The reference-comment metric. The harness compares each generated comment with each reference on the same file. A pair is considered only if the two line ranges overlap or lie within $k = 1$ line of each other (the harness's *line gate*, `line_k = 1` [3]; the paper itself defines a line match as range overlap). A judge model is then asked whether the two comments "express the same concern or suggestion" (yes/no). Recall is references matched over all references; precision is generated comments that matched any reference over all generated comments. The published paper's judge is `Qwen3-235B-A22B-Instruct-2507` at temperature 0.7, top_p 0.95 and top_k 20 [2].

Case sets. We use four named subsets of the 185 PRs (Table 1); two different 40-PR sets play different roles, so the names are used throughout.

3.2 What the reference list is

We checked the reference list against the code in three ways.

Reference labels on 30 PRs. Every reference on smoke10, dev10 and holdout10 was checked against the code at head and labelled: real defect, real maintainability issue, trivial, dubious (unclear or likely wrong), or duplicate. All 253 references carry a label: 29 real defect (11%), 20 real maintainability (8%), 77 trivial (30%), 99 dubious or wrong (39%) and 28 duplicate (11%) (Figure 2).[3] The project has called these "hand labels", but the label files' own `source` fields say they were produced by agents (per-PR analysts with a critic on smoke10, "two labelling agents" on dev10 and holdout10), so we call them *reference labels*. No human checked them: they are agent-produced and unverified, and no human has re-labelled a sample blind to them, although they support one of this paper's arguments; that check is listed in Section 15.1.

Against the verified defect key on R145. During key construction on R145 (Section 5), every reference was checked against the code as a candidate defect. Only 142 of the 1,107 references (12.8%) turned out to be part of a real defect. The rest were rejected as wrong, trivial, style, pre-existing or unreachable, each with a recorded reason.

The converse. Most real defects are not references. Over all 185 PRs, only 162 of the 520 verified defects (31%) have at least one AACR reference among their sources, and 40 of the

[3]Counted from the `reality` field of `bench/aacr/sets/{smoke10,dev10,holdout10}.labels.json`, 2026-09-28. An earlier per-label table (in `FULL-BENCHMARK.md` and `RESULTS.md`) gave denominators summing to 225; that count keyed labels on (PR, path, start line) and collapsed 28 references that share a start line with another. The correct count is 253.

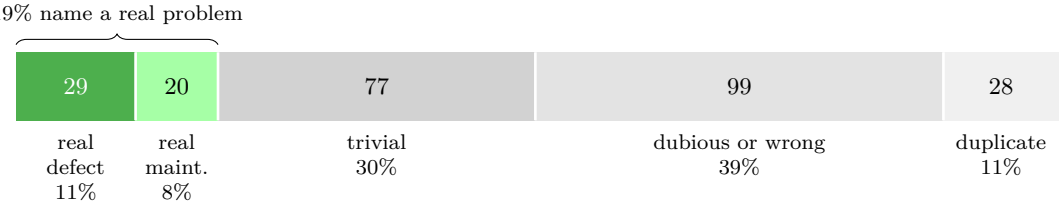


Figure 2: What AACR-Bench's reference comments are: the 253 references on the 30 PRs of smoke10, dev10 and holdout10, each checked against the code at head and labelled (agent-produced labels, Section 3.2).

90 high-severity ones (44%).[4] On three fresh10 PRs used as a pipeline check, the key holds 11 defects (6 high); the references hold 7 comments, of which one is a real defect, one is wrong (an operator-precedence "bug" that is not one) and five are style or readability notes. A timestamp truncation that breaks mobile sync, CI broken by stale mocks, and a mode losing its tools are on no reference list, so a reviewer that reports them loses precision.

"Human" references that are bot comments. On our 185 PRs, 362 references are marked human. At least 80 of them (22%) are rewrites of comments that an AI review bot posted on the original PR, not of a human's: GitHub Copilot 23, CodeRabbit 20, cubic 20, Sourcery 5, LlamaPReview 4, Gemini Code Assist 4, Ellipsis 3 and Cursor Bugbot 1. That raises the AI-written share of the list on these PRs from 74% to at least 80%. To find them, each human reference on the 46 PRs where a bot had reviewed was paired with the bot or human inline comment on the same file, within ±5 lines, with the highest overlap of content words; every pair at an overlap of 0.15 or more was read, and each was the same concern reworded, often with the same identifiers and fix. The count is a lower bound, since review bodies and issue comments were not checked.[5] Under AACR's metric such a bot is credited for repeating its own earlier comment, so any comparison that includes those products is tilted towards them on those PRs; Section 9.3 measures the tilt for Copilot. Our key is largely immune: 19 of the 520 defects cite one of these references, 17 of the 19 were also found by the expert hunt, and every one was checked against the code.

These findings are why we do not treat the reference list as ground truth. They are not a criticism of the benchmark's construction effort: a PR's historical review comments were never produced to be an answer key, and studies of code review find that most review comments are not about defects [12, 14, 15].

4 Design principles

We fixed the following principles before the full run. Each comes with the alternative we rejected; Sections 5–7.1 give the mechanics.

Measure what a review is for. The lead metric is *recall of verified defects*, graded by severity, plus the share of the review's comments that are true. *Rejected: AACR's metric alone.* It measures agreement with a list that is largely not defects, and in our tuning log it rose by about 20% on prompt changes that did not change the number of defects found (Section 10). *Rejected: dropping AACR's metric.* It is the only number computed the benchmark's way, and

[4] Defects whose `provenance.dataset_refs` is non-empty in `bench/aacr/defects/*.json`: 47 of 126 on T40 and 115 of 394 on R145 (the v2 extension added none with a reference). Computed 2026-09-28.

[5] Our research note of 2026-09-30, anonymous public GitHub access only. The 80 references are listed in `bench/aacr/bot_references.json`.

a check that we are not only measuring ourselves with our own yardstick; we report it in full, including where other tools beat ours.

Run the production reviewer, not a benchmark build. Both QuickReview arms run its production round with a BENCHMARK MODE section appended to the order (no GitHub, no CI, no memory; write the would-be review to files; Appendix A). The two arms differ only in a configuration preset (`default` or `low`) that sets the reasoning effort of every model. *Rejected:* a benchmark-tuned variant (a switch that posts every true nit, which AACR rewards, was measured and not adopted), and running every model at maximum effort (the later `max` preset was not used either). Residual differences from production (`bench/aacr/README.md`): memory is off (a foreign repository must not read or write the production reviewer's memory); the session loads a reduced extension set and no same-day feedback addendum; the order is delivered on standard input with at most one `--continue` retry; a multi-line comment is scored at its single anchor line; and, in the build of the earlier study, the frozen `pass-baseline` seed still carried the deployed repository's pytest and Django Ninja conventions (later builds moved them to repository-profile addenda). The rendered round orders of the two arms on R145 are byte for byte identical apart from commit shas.

Tune on some PRs, report on others. Every lever added to QuickReview during this work was designed from misses on T40. R145 was selected mechanically as "every runnable scored PR not in those sets" and created at 2026-09-27 23:34 (commit `1ad710b`), after the last *benchmark-driven* change to QuickReview; no design decision looked at it. Three later changes to QuickReview (2026-09-28/29) came from unrelated work and did not consult benchmark results; a full re-run after them reproduced the earlier result within 1.5 defects (Section 9.1). Results are reported for T40 and R145 separately (Section 8.6). Which sets each build's development used is disclosed at the head of Section 13.

Give every tool its best shot, on one model. Each third-party tool runs the way the AACR harness runs it (where the harness has a runner) or in the closest shape to it, with its own prompts. Where its own documentation ships or recommends a stronger configuration we measured that too, and each tool enters the final comparison in the best configuration found, by a rule fixed before the runs (Section 7.1). All seven OpenAI-based reviewers, QuickReview included, then run on one model, gpt-6.1-sol. Claude Code runs on the model it ships for, Claude Opus 5.5, and is the one tool whose configuration was not varied (Section 7.1). *Why:* a benchmark run by the author of one contestant has every incentive to measure the others in a weak configuration, a known failure of comparisons against under-tuned baselines [101, 102]. Our own earlier study did so in three places without intending to, and correcting those three was worth 12 to 17 points of recall on S40 to the tools concerned (Codex CLI 44% to 56%, pi-review 52% to 68%, OpenCode 43% to 60%; Table 5). *Rejected:* prompt-tuning the other tools (we would tune competitors with less care than our own system; we changed only settings the tools themselves expose and document), and an equal-compute comparison (holding every tool to one budget would compare designs nobody ships; we report cost next to every recall figure instead, as cost-aware agent evaluation recommends [111]).

Sandbox everything. All reviews and judgements ran locally in Docker, never against GitHub's write API. Containers have a read-only root filesystem, all capabilities dropped, `no-new-privileges`, a non-root user and pid/memory/CPU caps. Each PR's repository is a read-only bare store holding only the base and the reviewed head commit, so a reviewer cannot read the PR's future: without this, a reviewer with repository access could find the fix commit and report the defect it fixes, the review-time analogue of the solution leakage found in SWE-bench [41]. The model login is copied per case into the container's tmpfs with its refresh

token replaced by a dummy. Claude Code uses its own API key and exists only inside its own image.

Make every number traceable and re-scorable. Every run records a build fingerprint; every judge verdict is stored under the key version and judge model that produced it, so a changed key never mixes with old verdicts; a failed judgement is an error file counted as "not judged", never as a miss; every rejected candidate defect is kept with its reason.

5 The defect key

5.1 What counts as a defect

A defect is a concrete problem the PR introduces or newly exposes, that a competent reviewer should raise before merge, with a statable consequence: wrong behaviour, crash, hang, data loss or corruption, security hole, broken build/CI/install, broken contract for callers, a real performance regression on a realistic path, a resource leak, a concurrency bug, a compatibility break, or a fix that does not do what the PR says. *Maintainability* is admitted only with a concrete future cost (for example a doc comment the change made wrong, or a duplicated rule that has already drifted). Excluded: pre-existing problems the PR does not touch or worsen, unreachable scenarios, things already handled, requests for tests or docs, style and nits. **Severity** is graded on the consequence, not on who proposed the defect or how it was phrased: *high* would block a merge (a user-visible bug on a normal path, crash, data loss, security hole, broken build/install/CI); *medium* should be fixed before merge (a real bug on a less common path, a real contract, performance or leak issue); *low* is real but fine as a follow-up.

Each defect carries an id, title, statement (mechanism and consequence), one or more *sites* (path, line range, role: cause, symptom or other), kind, severity, evidence (quoted code and the path by which it is reached), *notes* (what a review comment must say to count, and near-misses that must not), and *provenance* (which candidate sources raised it). This is closer to the bug descriptions of curated defect benchmarks such as Defects4J [64] than to a review comment, but unlike those benchmarks it has no executable oracle: a defect is established by reading the code, not by a failing test.

5.2 Why a verified candidate pool, not a blind hunt

The central design decision is where candidate defects come from. *Rejected: the AACR references as truth* (Section 3.2). *Rejected: a blind expert hunt alone.* A single hunt misses defects: on R145 the hunt raised 241 of the 379 defects that survived verification in the first version of the key, so a hunt-only key would have been about a third smaller. The same gap between what one finder sees and what is there is familiar from static bug detectors, which find only a small fraction of known bugs [69]. *Chosen: a pooled, verified, anonymised union* of candidates from an expert hunt, every AACR reference and every comment the benchmarked reviewers posted, each re-checked against the code by a verifier who could not tell which reviewer wrote what, then an adversarial critic. The price is that recall is measured against the union of what somebody in the pool found, which favours pool members, as pooled relevance judgements do in information retrieval [92, 46, 48] (Threat 1, Section 13). We recorded the provenance of every defect so that this bias can be measured rather than only acknowledged.

The first R145 hunt was launched blind (no references). The operator then decided the hunt should also check the references, as leads; the blind run was stopped after about three minutes (32 drafts written), re-launched with references, and the 32 drafts were re-checked as candidates. Reviewer outputs were off limits to the hunt throughout.

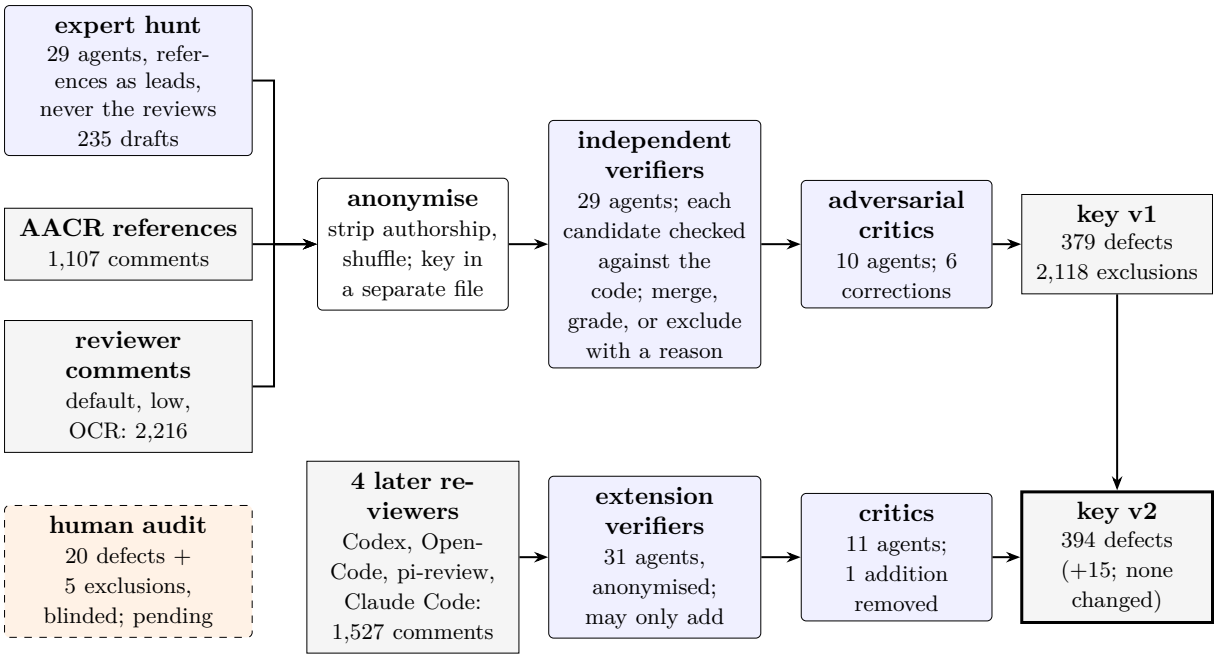


Figure 3: How the answer key for R145 was built. Every candidate — from the hunt, the dataset's references and the reviewers' comments — reaches the verifiers without its author. T40's keys were built the same way a day earlier (Section 5.5). The comments of the final comparison's gpt-6.1-sol runs were never candidates (Section 8.8).

5.3 Pipeline for R145

1. **Code extraction** in a network-less container: per PR the diff, base and head versions of every changed file, a bare git store for reading the whole repository at head, the PR metadata, and the AACR references with model names stripped.
2. **Expert hunt**: 29 Claude agents, about five PRs each. Output: 235 draft defects (46 high); 134 references judged real, 973 rejected.
3. **First reviews**: QuickReview default, QuickReview low and open-code-review ran on R145 (gpt-6-sol).
4. **Independent verification**: 29 Claude agents, about five PRs each. Every candidate (the hunt's drafts, all 1,107 references, and all 2,216 reviewer comments: default 978, low 663, OCR 580) was checked against the code. Reviewer comments were anonymised and shuffled; the key mapping them to reviewers sat in a separate file the agents were told never to open. Duplicates were merged, severities graded, and every rejection recorded with a one-sentence restatement of the claim first, so the judge can later recognise it in a review. 2,118 exclusions.
5. **Adversarial critic**: 10 agents, one per about 15 PRs, spot-checked every high-severity defect, a sample of other defects, exclusions, merges and grades against the code, and fixed the key in place: 6 corrections.
6. **Repair and assembly**: one PR's candidate list was renumbered after verification and re-verified by one agent so every provenance id is correct; assembly into `defects/rest145.json` v1, **379 defects** (55 high / 113 medium / 211 low), validated with the judge's own parser.

Provenance audit. Of the 2,118 exclusions, 1,150 were reviewer comments (default 550, low 345, OCR 255), 965 were AACR references and 3 were hunt drafts. Of the 379 v1 defects, 138

were not in the hunt: 8 high, 30 medium and 100 low (129 raised only by reviewer comments, 9 by a reference and reviewer comments). Against the key's 55/113/211, the hunt alone missed 15% of the high-severity defects and 47% of the low ones.[6] Where both raised a defect, the verifier mostly kept the hunt's grade: on the 234 confident matches of a defect to its hunt draft the grade was unchanged for 220 (94.0%, $\kappa = 0.91$), raised for 8 (5 low→medium, 3 medium→high) and lowered for 6 (3 medium→low, 2 high→low, 1 high→medium); eight hunt drafts were split into more than one defect.[7]

5.4 Extension with four more reviewers (v2)

After Codex CLI, OpenCode, pi-review and Claude Code had run (on gpt-6-sol and Opus 5.5 respectively), every one of their comments on all 185 PRs (1,527 candidates: OpenCode 557, pi-review 395, Codex 390, Claude Code 185 whole review texts) was anonymised, shuffled by a hash of the comment text, and checked against the code and the current key by 31 verifier agents. A Claude Code review is prose, so each whole review is one candidate whose claims are checked one by one (which is why the outcomes below sum to more than 1,527). Results: 931 matched an existing defect, 691 excluded with reasons (572 of them on R145), 16 added. Eleven critics checked every addition and removed one (4 critic corrections in total). Net **+15 defects**: R145 v2 has **394 defects** (56 high / 114 medium / 224 low). The assembly script refused to write if any existing defect was dropped or changed, and the version number was bumped so verdicts from the two versions never mix. The same pass over T40 added no defect, so those files were not changed; their new exclusions are kept only in the analysis folder.

Why extend rather than rebuild. Rebuilding the key with all seven reviewers in the pool from the start would have given every tool an equal-sized seat, at the cost of redoing a critic-checked verification. We extended, and measured the resulting seeding bias directly (Section 8.6). The small yield (15 new defects from 1,527 comments) suggests the pool was close to saturated for these reviewers. The saturation curve says the same: on R145, start from the 250 defects that the hunt or an AACR reference raised and add the seven reviewers one at a time; averaged over all 5,040 orders, they add 69.1, 30.1, 17.3, 11.0, 7.4, 5.2 and 3.7 defects, so the seventh contributes under 1% of the 394. The average marginal contribution per reviewer is: default 42.6, Claude Code 21.9, low 21.7, OCR 19.6, pi-review 14.2, Codex CLI 12.1, OpenCode 11.8.[8] The curve flattens for this pool only; it says nothing about defects that none of these reviewers would raise, and the final comparison's runs were not in it (Section 8.8).

5.5 T40

T40's keys were built earlier (2026-09-26/27) with the same shape: a blind hunt per PR, then a consolidator checking the hunt, the AACR references (with their labels) and every anonymised comment from our runs and OCR's, then a critic per set (12 corrections). Version 2 of smoke10, dev10 and holdout10 added the judge's "new defect" candidates after checking them against the code: 44 candidates from 12 judged runs, 1 added, 2 restated existing defects, 41 rejected. fresh10 (v1) was built from a blind hunt and the comments of both QuickReview arms plus the references; **OCR's comments were not candidates for fresh10**, so OCR may be under-credited on those 10 PRs.

[6] Computed from `rest145-code/<id>/truth.json`, `candidates-key.json` and `hunt.json` in the analysis tree, 2026-09-29.

[7] Computed from `hunt.json` and `truth.json` per PR (matching script `wp-data/prov.py`), 2026-09-29. Matching is by site overlap and text similarity; the seven weakest of the 241 matches were left out because some paired a defect with the wrong draft; with them the figures are 92.9% and $\kappa = 0.89$.

[8] Computed from `provenance` in `truth-ext.json` and the two candidate keys per PR (`wp-data/prov.py`), 2026-09-29. Without the hunt and the references, the reviewers' union grows 237, 303, 336, 356, 368, 377, 382.

Table 2: The answer key. Counts from FULL-BENCHMARK.md.

	T40	R145
Defects	126 (34 high / 31 medium / 61 low)	394 (56 high / 114 medium / 224 low)
Version	smoke10, dev10, holdout10 v2; fresh10 v1	v2 (v1: 379 defects)
Built	2026-09-26/27	v1 2026-09-28; v2 extension 2026-09-28

Table 3: Sources of the 379 rest145 v1 defects.

Source	Defects it raised	Only this source
QuickReview default's comments	334	29
QuickReview low's comments	269	3
Expert hunt	241	12
open-code-review's comments	230	4
AACR references	115	—

5.6 The final key

In total: **520 defects (90 high / 145 medium / 285 low)** over 185 PRs (Table 2). 26 PRs have no defect; the median PR has 2 and the maximum 17; T40 averages 3.15 defects per PR and R145 2.72.[9] By kind: bug 229, maintainability 72, incomplete-fix 57, build 42, contract 29, data 23, compat 21, security 13, resource 12, concurrency 11, performance 9, other 2. 3,093 rejected candidates are recorded with their reasons: 403 on T40 and 2,690 on R145 (2,118 from v1 plus 572 from the extension).[10] Three CI-gate defects (a lint rule, stale generated SQL, a clippy failure) carry a `benchmark_limitation` note: a read-only review can find them only by knowing the repository's CI. They are counted in every table.

Provenance on R145 (v1). Table 3 shows where the 379 v1 defects came from; a defect may have several sources.

QuickReview default (on gpt-6-sol) is the largest single contributor: 334 of 379 defects (88%), and 29 that no other source raised. This is the main reason for the independent-source checks of Section 8.6. Of each first-pass reviewer's own comments, 44% (default, 978 comments), 48% (low, 663) and 56% (OCR, 580) map onto a real defect; the rest are maintainability notes, questions and nits, graded separately ("comments correct"). Default's contribution is partly a matter of volume: it posts the most comments, and the lowest share of them map onto a defect.

5.7 Who built the key

Claude agents (Claude Code subagents on Opus 5.5) built and critiqued the key throughout. That family is independent of the OpenAI-based reviewers but is the same family as the Claude Code reviewer and the second judge. The same assistant that designed QuickReview's benchmark levers also orchestrated the key, wrote the admission rule above and wrote the judge prompt. **No human has checked any part of the key. [NEED EXPERIMENT: Human audit of the key, scope decided: one engineer audits 20 defects on Python PRs and 5 exclusions, blinded; the audit sheet, with full context per item, is ready and awaits the auditor. Publish the agreement rate here and correct the key where the audit overturns it. This is the single most important missing experiment. A larger sample stratified by severity and provenance (over-sampling defects raised only by**

[9] Computed from `bench/aacr/defects/*.json`, re-verified 2026-09-28.

[10] Sum of the `excluded` lists of the five committed defect files, computed from `bench/aacr/defects/*.json`, 2026-09-28.

QuickReview) would bound the error rate where it matters most; see REVIEW-benchmark.md, M1.]

6 The defect judge

6.1 What the judge reads and returns

One judge session per PR, with read-only access to the repository at the PR head, reads the PR's defect list (statement, sites and "counts if" notes); the claims already checked and rejected for this PR, each with its reason (capped at 600 characters each, 14,000 in total); and the review as the author would receive it: verdict, body (capped at 12,000 characters) and every comment with path and line. It returns, in strict JSON checked by a parser, a verdict per defect, *found* (some comment or the body identifies the defect with its essential point right), *partial* (points at the problem or its site with a wrong or missing mechanism or consequence) or *missed*, and a grade per comment: which defect it finds (or none), and correct-valuable, correct-minor, partly-correct, incorrect, or unverifiable. Grading each defect separately against a short statement of its essential point is the same decomposition that fact-level evaluation of long-form text uses [88]. The full prompt is in `docs/PROMPTS.md` §2 and reproduced verbatim in Appendix A.

Design decisions.

- *Location-agnostic.* A defect counts as found wherever in the review it is said: cause site, symptom site, another line, or the body. *Rejected:* a line gate like AACR's, which drops the right finding two lines away before any judge sees it.
- *Substance over form.* Wording, tone, tags and exact line numbers do not matter. The "counts if" notes, written by the verifier, state the essential point, so a vague comment near the right line does not count.
- *Partial credit of one half. Rejected:* binary scoring, which either credits a comment whose mechanism would make the author fix the wrong thing, or gives nothing to one that pointed at the site. Counting partials as zero moves no reviewer by a point and changes no order (Section 8).
- *The judge sees the rejected claims.* A comment repeating a rejected claim is graded against its recorded reason instead of being re-adjudicated. *Rejected:* hiding exclusions, which lets the judge re-accept a plausible claim the verifier already refuted with code. Since the exclusions were written from the tools' own comments, this makes the judge stricter for every tool and cuts both ways.
- *Errors are not misses.* A failed judgement is an error file; `compare.py` counts it as "not judged" and prints the row as incomplete. Every reported row is 185/185.

Metrics. **Recall**: found $+\frac{1}{2}$ partial, summed over defects, over the number of defects ("problems found" is the numerator). **Recall by severity**: the same per class. **Weighted recall**: severity weights high 3, medium 2, low 1. **Comments correct**: the share of the review's comments graded correct-valuable or correct-minor; because this merges a reviewer that posts many true minor notes with one that posts few valuable ones, every comparison also reports the full grade breakdown.

6.2 Judge reliability

These checks were run on the earlier study's 1,295 reviews (seven reviewers × 185 PRs, Section 9); the final comparison's reviews were graded by the primary judge only.

Two judges from different model families. The primary judge is `gpt-6-sol` at high reasoning. The same prompt, byte for byte, with the same inputs and parser, was run a second time on `claude-opus-5-5` over all 1,295 reviews. *Why:* the primary judge shares a model family with six of those seven reviewers, including ours, and LLM judges can prefer their own family's outputs [31, 29]. Using judges from different families is a proposed mitigation [32]. *Rejected:* a single judge with that caveat stated; a Claude-only judge, which would share a family with the key builder and with Claude Code.

The two judges gave the same verdict on **3,593 of 3,640** reviewer–defect pairs (98.7%; 7 reviewers × 520 defects) and on 628 of 630 high-severity pairs. Raw agreement flatters a three-way verdict whose classes are unbalanced [89], so we also report Cohen's κ [34]: $\kappa = 0.974$ on found/partial/missed, and $\kappa = 0.986$ when partial and found are merged (99.3% raw), both in the top band of the usual scale [35]. Of the 47 disagreements, 45 involve a *partial* on one side; only 2 are found-vs-missed. Problems found moves by at most ±3 defects per reviewer, and recall is identical after rounding for all seven.[11]

Per comment the judges agree less: the same five-way grade on 3,972 of 4,455 comments (89.2%), and the same correct/not-correct split on 91–95% of each reviewer's comments. The largest movements are partly-correct graded incorrect (99) or correct-valuable (91) by the second judge, and correct-minor graded incorrect (82). Comments correct therefore differs by up to 5 points between judges; the largest gaps are Claude Code (81% under GPT, 86% under Opus) and OCR (73% vs 77%).

Self-consistency. We re-ran the primary judge once on the same reviews for 30 random PRs × all 7 reviewers (210 reviews, 637 reviewer–defect pairs, 742 comments). It gave the same defect verdict on 99.7% of pairs ($\kappa = 0.994$); both disagreements involve a *partial*. On the same sample the two judges agree on 99.2% ($\kappa = 0.984$), so the difference between the judges is barely larger than the primary judge's run-to-run noise. Comment grades are noisier within one judge too: the same five-way grade on 92.2% ($\kappa = 0.87$) and the same correct/not-correct split on 93.5%, against 88.4% and 90.6% between the judges on this sample, so about two-thirds of the per-comment disagreement between judges (7.8 of 11.6 points) is at the level of the primary judge's disagreement with itself.[12]

What this shows and what it does not. The checks show that the *grading* (whether a given review finds a given defect, and whether a comment is true) is not an artefact of one model family or of one model's reading of the prompt: a preference of either family for its own reviewers would appear as disagreement, and there is almost none. The one visible same-family effect is on comments correct: the Claude judge is 5 points kinder to Claude Code's comments, while Claude Code's recall does not move (62% under both). This mirrors the finding from information retrieval that assessor disagreement changes absolute scores more than system rankings [93].

They do *not* show that the grading agrees with a human. Both judges read the same prompt and the same "counts if" notes, written by the key builder, and a bias they share would not appear as disagreement. LLM judges have been found to agree with human annotators unevenly across tasks [85, 81], so a human-graded sample of verdicts is needed and has not been done (Section 15.1). Nor do they validate the key: both judges score against the same 520 defects. Whether the key is complete and correct rests on the verification pipeline, the independent-source checks (Section 8.6) and the human audit not yet done. The final comparison's reviews, which only the primary judge graded, come from the same model family as that judge (gpt-6.1-sol and gpt-6-sol); the second-judge check was not repeated on them.

[11] Computed from the two judges' verdict files (`judge/v<ver>-gpt-6-sol-high/` and `judge/v<ver>-claude-opus-5-5/`) for the runs in `FULL-BENCHMARK.md`, 2026-09-28.

[12] Computed from `judge/wpself-gpt-6-sol-high/` (the re-run, seed 20260929) against the published primary and second-judge verdicts (`wp-data/selfcons.py`), 2026-09-29. One round, 596 model calls, $7.18 at list price.

Table 4: The eight reviewers of the final comparison, in the configuration Section 7.1 chose: pi-review [8] on pi [7], Claude Code [10], open-code-review [4], Codex CLI [5], OpenCode [6] and PR-Agent [9]; the three harness runners are AACR-Bench's own [3]. Versions are pinned by sha256/sha512. The earlier gpt-6-sol study's configurations are listed with Table 5.

Reviewer	Model / effort	How it runs
QuickReview default	gpt-6.1-sol; lenses medium, validator and risk passes high	Production round with BENCHMARK MODE; preset `default`: every benchmark-driven lever, code-quality lenses, demoting validator, strict nits. Reads the PR title and body, as in production.
QuickReview low	gpt-6.1-sol, every role low	Same harness, preset `low` (lean plan).
pi-review 0.18.0 (pi 0.99.1)	gpt-6.1-sol, thinking high	Its `code-experts` panel: three isolated read-only reviewers (correctness, security, testing), clustered, quorum two, singletons kept as advisories; diff attached as a file.
Claude Code 2.1.283	Claude Opus 5.5, session default	The harness's `reviewers/claude.py` (`/code-review base...head`); findings parsed from its prose answer.
open-code-review 1.3.1	gpt-6.1-sol, effort high	Exactly as the harness's `reviewers/ocr.py`; model calls through a loopback bridge.
Codex CLI 0.159.2	gpt-6.1-sol, effort high	The harness's `reviewers/codex.py`: `codex exec review`, its prompt and MCP findings server; ChatGPT login.
OpenCode 1.18.33	gpt-6.1-sol, variant medium	Native ChatGPT provider, built-in `review` command (`subtask: true`) with the Codex runner's findings fragment, plus the PR title and body.
PR-Agent 0.46.0	gpt-6.1-sol, medium	Shipped defaults: `/describe`, `/review`, `/improve` on a local git provider, with the PR title and body.

6.3 AACR's metric as we ran it

The harness's evaluation stage ran unmodified at a pinned commit (`68a5697`), `all` variant, 3 rounds averaged, line gate $k = 1$. Its judge prompt was answered by `gpt-6-sol` at low reasoning through a local OpenAI-compatible shim, not by the paper's `Qwen3-235B-A22B-Instruct-2507` [2]. The shim makes one `pi -p` call per request and does not pass on the harness's sampling parameters, so those are the model's defaults. Evaluation was sharded by PR and merged with the harness's own summary functions; on a copy of a finished run, sharded and unsharded results were identical per round, average and PR. The AACR judge is noisy (the same pair was judged yes/yes/no across three calls in testing), which is why three rounds are averaged. *Consequence:* our AACR table is comparable across our own rows, not with the published leaderboard. We chose this for consistency: every benchmark in this paper is graded by our own OpenAI judge rather than by the judge each benchmark publishes, so that a score means the same thing on AACR, Martian and Qodo. (The README describes how to point the shim at Qwen3 through an OpenAI-compatible endpoint.)

7 Reviewers and the best-shot protocol

QuickReview in brief. QuickReview is the authors' own reviewer. It runs one main session per PR, which launches about ten independent lens passes chosen from the diff (a baseline, tests, a per-file sweep, code-quality lenses and content-triggered sub-lenses) and then a validator that re-derives each candidate finding from the code before one review is posted. Its `default` and `low` presets differ only in reasoning effort. Its design, the levers it gained while we built this benchmark, and its base-model study are in the companion report [1]. QuickReview was not re-configured by the protocol below: its `default` preset is its row, with `low` as a second, cheaper

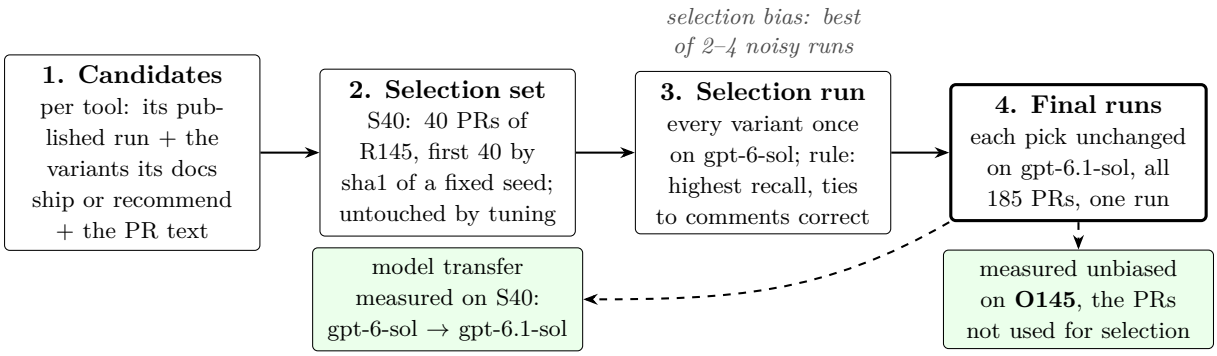


Figure 4: The best-shot protocol. QuickReview was not tuned in steps 1–3: its `default` preset is its row, with `low` as a second, cheaper point. Claude Code stayed at its published configuration, whose medium level matches QuickReview's medium lenses.

point.

7.1 Choosing each third-party tool's configuration

Our earlier study (Section 9) ran each third-party tool as shipped, or as the AACR harness runs it, on gpt-6-sol. A fairness audit of those runs (2026-09-30) read each tool's documentation, shipped configuration files and binaries, and every run's transcripts, and asked whether each tool had been given its best shot. It found three that had not: pi-review ran below its own default reasoning level and without its panel, OpenCode ran through our compatibility bridge instead of its own provider, and no tool but QuickReview (and PR-Agent) was given the PR title and body. The protocol (Figure 4) measures each tool in the configurations its own documentation ships or recommends, picks one per tool by a rule fixed in advance, and measures what the pick itself is worth. It follows the advice from machine learning that baselines get a tuning budget comparable to the proposed method's, and that the choice itself is reported with its optimism [101, 102, 45].

The selection set S40. A seeded sample of R145: sort R145's PRs by the sha1 of the string `fairness-2026-09-30|` followed by the instance id, and take the first 40. They hold 115 defects (17 high, 33 medium, 65 low) and 360 AACR references; none was used to tune QuickReview. Every configuration ran once on S40 on gpt-6-sol, and every arm is 40/40 judged by the primary judge. `final/select40.py --check` re-derives the set.

The variants. All on gpt-6-sol unless stated. "PR text" is the PR title and body, passed through each tool's own channel exactly as QuickReview receives it.

- **pi-review 0.18.0** [8]: `--thinking high`, its shipped `code` preset's default; high plus the PR text; and the `code-experts` panel (three reviewers for correctness, security and testing, quorum two, singletons kept as advisories) at high, which is what pi-review's `/rv` command runs by default in the pi host [8].
- **Codex CLI 0.158.0** [5]: reasoning effort high (its default for gpt-6-sol is medium); medium plus the PR text.
- **open-code-review 1.3.1** [4]: effort high at the model endpoint (1.3.1 has no effort flag); medium plus the PR text through `--background`, which its own agent skill marks as recommended [4].
- **OpenCode 1.18.33** [6]: its native ChatGPT provider (Responses API, reasoning carried across turns, prompt cache) with the built-in `review` command at its default `subtask:`

Table 5: Best-shot configurations on S40 (40 PRs of R145), 115 defects (17 high), primary judge. One run per configuration. Bold: the best configuration found per tool. AACR columns: 360 references on these PRs, 3 judge rounds averaged; the last column removes the 30 bot-derived references (a comment matching only a removed reference counts as unmatched).

Tool	Configuration	Found	Recall	High	Correct	$/PR	Min	AACR R / P	AACR F1	F1, bot refs out
QuickReview	default, as published	98.5	86%	16	81%	2.11	9.5	11.0% / 13.1%	12.0%	9.1%
	default, current build	**103**	**90%**	**17**	87%	2.12	9.7	11.0% / 13.1%	12.0%	9.6%
	gpt-6.1-sol	98	85%	**17**	**94%**	1.64	10.9	11.4% / 15.4%	13.1%	9.7%
	low preset	77.5	67%	14	76%	0.93	4.7	8.3% / 14.3%	10.5%	8.1%
Claude Code (Opus 5.5)	**as published (medium)**	68.5	60%	14	77%	0.26	0.9	3.1% / 12.2%	4.9%	4.8%
pi-review	as published (medium, 1 reviewer)	59.5	52%	14	72%	0.11	1.0	3.9% / 15.7%	6.2%	5.3%
	high	71	62%	15	71%	0.17	3.0	5.5% / 17.9%	8.4%	7.6%
	high + PR text	72.5	63%	13	74%	0.18	3.0	5.2% / 17.0%	7.9%	7.6%
	3-reviewer panel, high	78	68%	15	78%	0.46	3.6	7.2% / 16.0%	10.0%	9.3%
open-code-review	as published (medium)	76	66%	13	73%	0.86	1.3	9.3% / 17.3%	12.1%	8.5%
	high	78	68%	14	76%	1.12	4.1	11.4% / 17.9%	13.9%	10.0%
	medium + PR text	78	68%	14	73%	0.87	2.7	9.1% / 16.7%	11.8%	8.5%
OpenCode	as published (bridge)	49	43%	12	57%	0.96	1.7	4.4% / 14.4%	6.8%	5.4%
	native provider	57	50%	13	67%	0.40	5.2	5.1% / 11.3%	7.0%	5.8%
	native + PR text	69.5	60%	13	66%	0.40	5.3	5.1% / 10.9%	6.9%	6.6%
Codex CLI	as published (medium)	51	44%	14	62%	0.19	1.7	2.8% / 11.5%	4.5%	3.8%
	high	64	56%	15	78%	0.24	4.8	4.1% / 15.4%	6.4%	6.0%
	medium + PR text	62.5	54%	13	70%	0.18	3.5	4.6% / 17.7%	7.3%	6.9%
PR-Agent	gpt-6-sol	34.5	30%	10	61%	0.10	3.2	6.8% / 24.6%	10.6%	9.0%
	gpt-6.1-sol	39.5	34%	11	63%	0.09	2.3	5.7% / 23.8%	9.2%	7.5%

`true` and `--variant medium`; the same plus the PR text.

- **PR-Agent 0.46.0** [9], new to the benchmark: its shipped defaults and full GitHub-App flow (`/describe`, `/review`, `/improve`; at most 3 key issues and 3 suggestions per chunk; a 32k-token prompt budget that prunes 4 of the 40 diffs, as it would for any user), reading the PR title and body as it does on GitHub; on gpt-6-sol and on gpt-6.1-sol.
- **Claude Code** was *not* varied. It stays at its published `/code-review` on Opus 5.5 at the session default (medium), by the owner's decision, to match QuickReview's medium lenses. Anthropic documents that at low and medium effort the review "reports only the findings it's most confident in" and that higher levels broaden coverage [10]. This is an exception to the protocol, and it plausibly disfavours Claude Code (Threat 3).

What configuration moved (Table 5, Figure 5). **pi-review** gains 16 points (52% to 68%): its default reasoning level alone gives 10, and the three-reviewer panel the rest, at 2.7 times the cost of a single high reviewer ($0.46 against $0.17). **OpenCode** gains 17 points (43% to 60%): the native provider gives 7 and cuts the cost from $0.96 to $0.40 per PR, because our bridge had lost prompt caching; the PR text gives the other 10. **Codex CLI** gains 12 points at high effort (44% to 56%) and its comments-correct share rises from 62% to 78%, for $0.05 more per PR. **open-code-review**, already run as its harness runs it, moves least: 2 points either way, with the best AACR F1 of any row at high effort (13.9%). **PR-Agent** finds 30–34%, the fewest, with the highest AACR precision (24–25%), which is what a shipped cap of three key issues produces. Against the published configurations the gap between QuickReview default and the best third-party tool on S40 was 20 points (86% against 66%); against the best configurations found it is 18–22 points (86–90% against 68%). Giving every tool its best shot moved the field up; it did not close the gap. These are 40 PRs and one run per configuration, and two runs of one QuickReview build differed by 4–8 points on single 10-PR sets (Section 9.1), so single-digit differences between rows are not established.

Selection rule and the picks. The rule, fixed before any selection run: per tool, the configuration with the highest defect recall on S40; a tie goes to the higher share of correct comments. The picks (bold in Table 5): pi-review's three-reviewer `code-experts` panel at thinking high

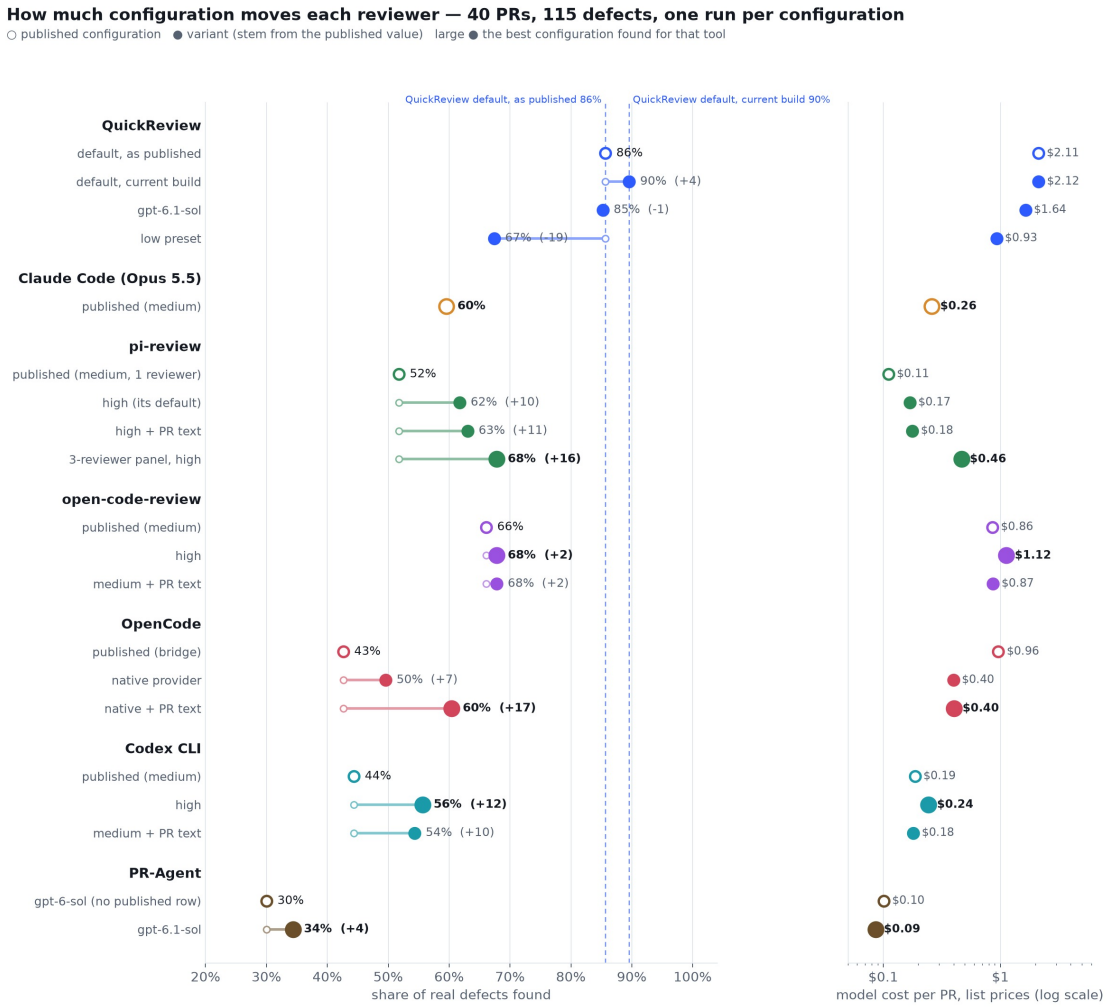


Figure 5: Configuration sensitivity on S40. QuickReview's rows are reference points, not selection candidates. Left: share of real defects found per configuration; hollow markers are the published configurations, stems run from the published value to each variant, and the large marker is the best configuration found for that tool. Dashed lines: QuickReview default as published (86%) and on the current build (90%). Right: model cost per PR at list prices, log scale.

(68%); open-code-review at effort high (68%, tied on defects found with medium plus PR text and ahead on comments correct, 76% against 73%); OpenCode's native provider with the PR text (60%); Codex CLI at effort high (56%); PR-Agent's shipped defaults on gpt-6.1-sol, the better of its two models (34%); and Claude Code unchanged (60%). These are the best configurations *we found* among those the tools' documentation ships or recommends, not an exhaustive search. Not run: Claude Code's `/code-review` at a higher level and pi-review at `xhigh`.

Carrying the picks to gpt-6.1-sol. QuickReview's gpt-6.1-sol run already existed (it was made for the companion report's base-model study [1], on which QuickReview finds 5 points *fewer* defects than on gpt-6-sol). The owner then decided that every OpenAI-based contender should run on gpt-6.1-sol, so that no comparison rests on a model difference between them. The picks were re-run unchanged apart from the model, on S40 and O145 (2026-09-30 19:59 to 2026-10-01 00:16 UTC, after a two-PR smoke run per tool). Each tool needed one change to reach the new model, none of which alters its review behaviour: pi-review moved to pi 0.99.1, the first release whose model catalogue lists gpt-6.1-sol, with pi's new built-in extensions switched

Table 6: The model transfer on S40 (115 defects): each pick on gpt-6-sol (selection run) and on gpt-6.1-sol (final run), one run each.

Tool	Configuration	gpt-6-sol: found	gpt-6.1-sol: found	Change	Comments correct	$/PR
pi-review	panel, high	78 (68%)	80 (70%)	+2	78% → 91%	0.46 → 0.30
open-code-review	high	78 (68%)	75.5 (66%)	−2.5	76% → 88%	1.12 → 0.92
OpenCode	native + PR text	69.5 (60%)	57.5 (50%)	**−12**	66% → 89%	0.40 → 0.19
Codex CLI	high	64 (56%)	64.5 (56%)	+0.5	78% → 94%	0.24 → 0.18
PR-Agent	defaults	34.5 (30%)	39.5 (34%)	+5	61% → 63%	0.10 → 0.09
QuickReview	default	98.5 / 103 (86 / 90%)	98 (85%)	−0.5 / −5	81–87% → 94%	2.11 → 1.64
QuickReview	low	77.5 (67%)	86.5 (75%)	+9	76% → 93%	0.93 → 1.04

Table 7: Each final configuration on S40 and on O145 (405 defects, 73 high), gpt-6.1-sol except Claude Code.

Reviewer	S40: recall	O145: found	O145: recall	O145: high	Change
QuickReview default	85%	311	77%	66.5	−8
QuickReview low	75%	279	69%	62.5	−6
pi-review panel (high)	70%	267.5	66%	63	−4
Claude Code (Opus 5.5)	60%	256	63%	58.5	+3
open-code-review high	66%	223	55%	48.5	−11
Codex CLI high	56%	222.5	55%	65	−1
OpenCode native + PR text	50%	206	51%	60	+1
PR-Agent	34%	154.5	38%	44.5	+4

off so it has the same tools as before; OpenCode's native provider got a custom model entry for gpt-6.1-sol; Codex CLI moved from 0.158.0 to 0.159.2, because the ChatGPT backend refused 0.158.0's requests for the new model; open-code-review and PR-Agent reach the model through our loopback bridge, which now serves gpt-6.1-sol.

Every reviewer's comments became more often correct on gpt-6.1-sol (Table 6); recall moved by less than the run-to-run spread we measured for QuickReview for four of the five picks. OpenCode is the exception: its pick lost 12 defects (10 points), and its other variants were not re-run on gpt-6.1-sol, so a different OpenCode configuration may now be its best. That the picks remain each tool's best on gpt-6.1-sol is an assumption we did not test further.

What the selection is worth. A configuration picked as the best of two to four noisy runs on 40 PRs is biased upward on those 40 [45]. O145 played no part in the choice and measures each pick without that bias (Table 7).

QuickReview's default, which was not selected on S40, drops 8 points from S40 to O145; the picks move between −11 (open-code-review) and +4 (PR-Agent), so S40 is not uniformly easier and the selection advantage is small next to the gaps between tools. On O145 the order is the same as on all 185, and the default leads the panel by 11 points.

7.2 Deviations per tool, and their direction

- **Codex CLI**: web search disabled could disfavour it, but no arm could search. Version 0.159.2 instead of the harness-era 0.158.0, because the ChatGPT backend refuses 0.158.0's requests for gpt-6.1-sol. Otherwise neutral.
- **OpenCode**: no harness runner exists; built in the Codex runner's shape on OpenCode's own provider and review command, with the Codex runner's findings fragment, so its reporting format is ours. gpt-6.1-sol is declared as a custom model on that provider, because 1.18.33's catalogue predates it. It reads the PR title and body, which the harness never passes.

- **pi-review**: its panel at its shipped reasoning level, on pi 0.99.1 with pi's new built-in extensions switched off; the diff is attached as a file because its reviewer has no shell. It does not read the PR text; on gpt-6-sol the PR text was worth 1.5 defects to it on S40 (Table 5), so this mildly *disfavours* it.
- **Claude Code**: one configuration, a different model, and its read-only `git` commands were denied on 46 of 185 PRs by the harness's permission set, which may disfavour it. Its findings are parsed from prose (below); parsing errors mainly lower its AACR numbers and may move its comments-correct share either way.
- **open-code-review**: runs exactly as the harness's runner, at effort high set at our model bridge (1.3.1 has no effort flag).
- **PR-Agent**: run on a local git provider rather than GitHub, with its shipped defaults, including the 32k-token prompt budget.

Claude Code's prose parser. In Claude Code 2.1.283, `/code-review` is a bundled skill [10]. It ignores the harness's appended system prompt and its findings tool, and answers in markdown prose, so the harness records zero structured findings (the logs say `0 finding(s)` for 182 of the 185 logged cases). We wrote a deterministic parser (`src/bench/claudeprose.ts`, 12 golden fixtures, rules read off 150 real outputs) that reads `path:line` findings out of the prose, applied after the fact without re-reviewing. *Rejected:* re-prompting Claude Code for structured output (that would change the tool as shipped) and scoring it as zero (that would measure the harness's integration, not the review). The defect judge reads the full prose as the review body as well as the parsed comments, so defect recall depends little on the parser; comments correct grades the parsed comments; the AACR metric scores only parsed comments, so a finding without a line number is lost there. The parser read 174 of the 185 reviews (the other 11 came back through the findings tool: 3 with findings, 8 empty) and extracted 402 located findings (mean 2.3 per review, median 2, maximum 9; 29 reviews with none, mostly "I found no bugs"). A hand check of 20 random reviews found 45 findings, of which the parser extracted 44 with the right path and line, and nothing that was not a finding; the miss names a file but no line, which the rules leave out by design.[13]

7.3 Execution and cost accounting

Arms ran through a machine-wide scheduler with at most 10 concurrent reviews per model login; the final third-party and `low` runs were launched fresh in two halves (S40 and O145) and every case finished. Costs are list-price equivalents, not bills. gpt-6.1-sol arms are priced at $2 per million uncached input tokens, $0.10 per million cached input tokens and $10 per million output tokens; gpt-6-sol arms the same with $0.20 per million cached tokens. For QuickReview and pi-review these are pi's list prices, summed over the main session and every pass; arms reporting OpenAI-style usage (OCR, OpenCode and PR-Agent through the loopback bridge; Codex CLI natively) have the cached count subtracted from input and priced at the cached rate; Claude Code uses its own reported `total_cost_usd`, an Anthropic list-price equivalent whose uncached column includes cache writes. All OpenAI-model runs went through a ChatGPT subscription login, so what they would have cost on OpenAI's API is unknown, and the relative cost of OCR, OpenCode and QuickReview low depends on the cached-token rate. Judging cost is in Section 14.

[13] Computed from `bench.claude_prose_parsed` in each result file and a reading of 20 sampled reviews (seed 20260929), 2026-09-29.

Table 8: The final comparison: defect benchmark, 185 PRs, 520 defects, primary judge. A partly found defect counts $\frac{1}{2}$. Weighted recall: high 3, medium 2, low 1. $ are list prices, not bills (gpt-6.1-sol at $2 / $0.10 / $10 per million uncached input / cached input / output tokens; Claude Code its own reported cost).[15]

Reviewer	Found	Recall	High (90)	Med. (145)	Low (285)	Weighted	Correct	$/PR	Median min/PR
QuickReview default	**409**	**79%**	**83.5**	**121**	**204.5**	**82%**	**95%**	1.36	9.5
QuickReview low	365.5	70%	79.5	104.5	181.5	74%	94%	0.88	6.7
pi-review panel (high)	347.5	67%	80	119	148.5	74%	89%	0.27	1.8
Claude Code (Opus 5.5)	324.5	62%	72.5	105	147	68%	81%	0.20	0.7
open-code-review high	298.5	57%	63.5	102	133	62%	86%	0.53	1.8
Codex CLI high	287	55%	81	101.5	104.5	65%	93%	0.14	2.1
OpenCode native + PR text	263.5	51%	76	99	88.5	61%	93%	0.17	2.3
PR-Agent	194	37%	55.5	70	68.5	44%	76%	0.07	1.9

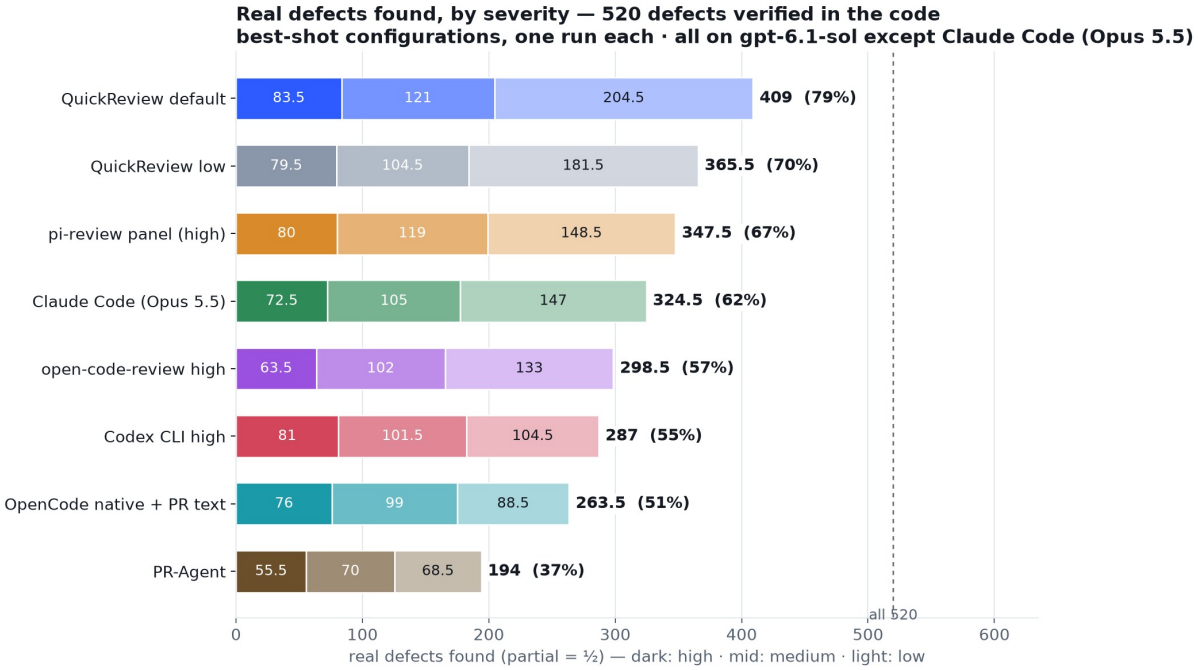


Figure 6: The final comparison: real defects found, split by severity (dark: high, mid: medium, light: low), 185 PRs, primary judge.

8 Results: the final comparison

Eight reviewers, each in the configuration of Section 7.1, on all 185 PRs; every OpenAI-based reviewer on gpt-6.1-sol, Claude Code on Claude Opus 5.5. Every reviewer ran once and every review was judged once by the primary judge (gpt-6-sol at high reasoning). Every PR has a usable judgement for every reviewer (185/185). Recall leads every table, high severity first; comment accuracy, cost and time sit beside it.

8.1 Defects found

QuickReview's default finds 409 of the 520 defects (79%), with 95% of its comments correct (Table 8, Figure 6). Next are its own `low` preset (70%), pi-review's three-reviewer panel (67%), Claude Code on Opus 5.5 (62%), open-code-review (57%), Codex CLI (55%), OpenCode (51%) and PR-Agent (37%).

[15] `compare.py --cases all30+fresh10+rest145` on the runs listed in `bench/aacr/final/arms.txt`; reproduced independently from the judge files, 2026-10-01.

Table 9: Recall, high-severity recall and weighted recall with 95% bootstrap intervals over PRs (seed 20261001), and cost per defect found (total list cost over the 185 PRs divided by problems found).

Reviewer	Recall [95% CI]	High recall [95% CI]	Weighted [95% CI]	$/defect	$/high defect
QuickReview default	78.7% [74.8, 82.2]	92.8% [87.2, 97.4]	82.5% [79.1, 85.6]	0.61	3.01
QuickReview low	70.3% [66.2, 74.1]	88.3% [81.0, 94.7]	74.4% [70.5, 78.2]	0.44	2.04
pi-review panel (high)	66.8% [63.3, 70.4]	88.9% [82.8, 94.9]	74.1% [70.8, 77.4]	0.14	0.62
Claude Code (Opus 5.5)	62.4% [58.6, 66.4]	80.6% [71.1, 89.6]	68.0% [64.0, 72.1]	0.11	0.50
open-code-review high	57.4% [51.9, 62.5]	70.6% [58.8, 82.0]	62.4% [56.2, 68.1]	0.33	1.55
Codex CLI high	55.2% [51.0, 59.0]	90.0% [83.8, 95.5]	65.1% [61.4, 68.6]	0.09	0.33
OpenCode native + PR text	50.7% [47.1, 54.4]	84.4% [77.6, 90.8]	60.9% [57.3, 64.4]	0.12	0.42
PR-Agent	37.3% [33.7, 41.1]	61.7% [51.2, 72.5]	44.4% [40.1, 48.7]	0.06	0.22

Where the lead is. It is in low severity. On the 90 high-severity defects the default finds 83.5, Codex CLI 81, the panel 80 and `low` 79.5. On medium the default and the panel are level (121 and 119 of 145). On low severity the default finds 204.5 of 285, against 148.5 for the panel and 104.5 for Codex CLI. Restricted to the 235 high- and medium-severity defects, recall is: default 87.0%, panel 84.7%, `low` 78.3%, Codex CLI 77.7%, Claude Code 75.5%, OpenCode 74.5%, open-code-review 70.4% and PR-Agent 53.4%. On those defects the default's lead over the panel is 2.3 points, 95% interval [−2.7, +7.0]: not established. Its lead over every other reviewer is (over `low` +8.7 [+5.1, +12.4], over Codex CLI +9.4 [+3.5, +15.1], over Claude Code +11.5 [+4.8, +18.0]).[16] A reader who cares only about merge-blocking defects should read this as the default and the panel tied at the top, at very different costs ($1.36 against $0.27 per PR). The later frozen build does establish a lead there (Section 8.7).

Uncertainty. Table 9 gives 95% bootstrap intervals over PRs (10,000 resamples, recall recomputed as a ratio of sums [36, 37]); differences are paired, as in the paired bootstrap for comparing systems [38]. Every difference between the default and another reviewer on overall recall excludes zero: over `low` +8.4 points [+5.6, +11.1], over the panel +11.8 [+7.8, +15.7], over Claude Code +16.2 [+10.8, +21.3]. Not established: `low` against the panel (+3.5 [−0.8, +7.7]), the panel against Claude Code (+4.4 [0.0, +8.7]), Claude Code against open-code-review (+5.0 [−0.7, +11.3]) and open-code-review against Codex CLI (+2.2 [−3.7, +7.5]). On high-severity recall the default's lead over the panel (+3.9 [−1.9, +9.6]) and over Codex CLI (+2.8 [−3.0, +8.7]) is not established, and no pair among `low`, the panel, Codex CLI and OpenCode is separated; its lead over `low`, Claude Code, OpenCode, open-code-review and PR-Agent is. With 90 high-severity defects, gaps of a few points are below what this benchmark can resolve [105]. These intervals cover PR sampling only: each reviewer ran and was judged once, so they contain no run-to-run or judge variation (Section 9.1 measures the first for QuickReview, and Section 6.2 the second).

Partial credit does not drive the order. Counting a partial find as zero instead of $\frac{1}{2}$ moves every reviewer by under one point (the default from 78.7% to 78.1%; at most 9 partial verdicts per reviewer), and the order is unchanged. The three CI-gate defects are found by the default (2 of 3), `low` (1) and no other reviewer.

8.2 What the comments are

Table 10 splits "comments correct". QuickReview's default posts the most comments (5.4 per PR) and has the fewest graded incorrect (2.5%), but a third of its comments are true and minor, and only 56% map to a verified defect. Codex CLI and OpenCode post under two comments

[16] Recall: (high + medium found) /235 from the High and Medium columns of Table 8. Intervals: paired bootstrap over PRs as in Table 9 (10,000 resamples, seed 20261001, recall recomputed as a ratio of sums per resample, percentile interval), computed for this check by `paper-data/factcheck_analyses.py` from the archived judge files; no model calls.

Table 10: Comment grades in the final comparison, all 185 PRs, primary judge; shares of each reviewer's comments. "Mapped" is the share the judge tied to a verified defect.

Reviewer	Comments	Per PR	Valuable	Minor	Partly	Incorrect	Mapped
QuickReview default	999	5.4	60.9%	33.8%	2.6%	2.5%	55.6%
QuickReview low	801	4.3	59.4%	34.1%	2.4%	4.0%	58.2%
pi-review panel (high)	585	3.2	72.6%	16.8%	4.6%	6.0%	67.0%
Claude Code (Opus 5.5)	407	2.2	75.9%	4.7%	6.9%	11.8%	80.8%
open-code-review high	570	3.1	69.5%	16.7%	4.7%	9.1%	63.3%
Codex CLI high	343	1.9	91.0%	1.7%	2.9%	4.4%	85.4%
OpenCode native + PR text	354	1.9	88.7%	4.0%	3.4%	4.0%	87.3%
PR-Agent	404	2.2	71.8%	4.0%	6.7%	17.6%	76.5%

Table 11: Recall by language (top; the 159 PRs that hold at least one defect) and by defect kind (bottom; kinds with fewer than 9 defects omitted), final comparison, primary judge.

	PRs	Defects	default	low	panel	Claude Code	OCR	Codex	OpenCode	PR-Agent
TypeScript	29	114	81%	80%	71%	67%	61%	58%	52%	37%
Go	20	80	78%	65%	65%	54%	49%	56%	52%	41%
C++	26	77	81%	73%	60%	61%	61%	51%	48%	38%
Python	17	65	77%	72%	70%	67%	71%	55%	53%	36%
Java	22	62	73%	64%	60%	69%	40%	45%	44%	34%
C	15	57	79%	65%	70%	52%	60%	60%	48%	28%
JavaScript	10	33	79%	64%	67%	65%	62%	55%	45%	42%
Rust	8	16	88%	81%	66%	56%	56%	56%	69%	50%
PHP	6	10	80%	50%	85%	85%	45%	70%	55%	50%
C#	6	6	67%	67%	83%	67%	67%	83%	83%	33%
bug		229	77%	71%	73%	68%	69%	64%	57%	41%
maintainability		72	81%	72%	38%	51%	32%	16%	24%	24%
incomplete-fix		57	89%	77%	73%	58%	47%	46%	55%	23%
build		42	81%	69%	71%	48%	31%	76%	62%	38%
contract		29	72%	55%	62%	48%	48%	41%	31%	28%
data		23	74%	65%	67%	67%	76%	65%	50%	35%
compat		21	67%	64%	55%	64%	57%	52%	52%	48%
security		13	92%	65%	85%	69%	85%	62%	62%	69%
resource		12	83%	75%	83%	67%	67%	83%	50%	42%
concurrency		11	91%	91%	91%	91%	91%	91%	91%	100%
performance		9	56%	56%	56%	67%	44%	56%	33%	33%

per PR, nine in ten of them correct and valuable, and 85–87% tied to a defect. Every OpenAI-based reviewer's comments are more often correct on gpt-6.1-sol than its counterpart's were on gpt-6-sol (55–86% in the earlier study, Table 33). A reader choosing a reviewer should weigh both columns: QuickReview finds more, and says more.

8.3 By language, kind and size

By language (Table 11) the default is first or tied first in eight of ten languages; pi-review's panel and Claude Code lead on PHP, and the panel, Codex CLI and OpenCode on C#, both with 10 or fewer defects. By kind, the largest spread is *maintainability* (72 defects): the default 81% and `low` 72%, against 16–51% for the third-party tools. That fits QuickReview's code-quality lenses, but those lenses were designed on T40 and the key's admission rule for maintainability was written by the same pipeline, so this is where an alignment between QuickReview's design and the key's definition of a defect would show first (Threat 5). Removing the 72 maintainability defects, the default still finds 78% of the remaining 448, against 71% for the panel and 70% for `low`. On *build* defects Codex CLI (76%) is close to the default (81%); on *data* open-code-review leads (76% against 74%); on *performance* Claude Code does (67% against 56%).

Table 12: AACR-Bench's metric on the final comparison: 185 PRs, 1,418 references; harness `all` variant, unmodified; its judge prompt on gpt-6-sol:low through a local shim; 3 rounds averaged; ±1 line gate. Right: the 80 bot-derived references removed (1,338 left; a comment whose only match was removed counts as unmatched).

	All 1,418 references				Bot-derived removed		
Reviewer	Found	Recall	Precision (comments)	F1	Recall	Precision	F1
QuickReview default	**151.7**	**10.7%**	15.2% (999)	**12.5%**	**9.9%**	13.3%	**11.4%**
QuickReview low	127.3	9.0%	15.9% (801)	11.5%	8.1%	13.5%	10.1%
pi-review panel (high)	84.7	6.0%	14.5% (583)	8.5%	5.7%	13.2%	8.0%
Claude Code (Opus 5.5)	81.0	5.7%	19.9% (407)	8.9%	5.8%	**18.9%**	8.8%
open-code-review high	115.3	8.1%	20.2% (570)	11.6%	7.7%	18.1%	10.8%
Codex CLI high	68.7	4.8%	20.0% (343)	7.8%	4.7%	18.3%	7.5%
OpenCode native + PR text	61.3	4.3%	17.3% (354)	6.9%	4.1%	15.6%	6.5%
PR-Agent	86.0	6.1%	**21.3%** (404)	9.4%	5.6%	18.6%	8.6%

Table 13: Mean tokens and cost per PR in the final comparison, 185 PRs. List prices, not bills. Uncached input includes cache writes.

Reviewer	Uncached input	Cached input	Cache share	Output	$/PR	Whole benchmark	Median min/PR
QuickReview default	473,842	1,401,469	75%	26,983	1.36	$251	9.5
QuickReview low	318,488	808,383	72%	15,794	0.88	$162	6.7
pi-review panel (high)	88,829	265,717	75%	4,721	0.27	$50	1.8
Claude Code (Opus 5.5)	19,341	148,749	88%	3,494	0.20	$36	0.7
open-code-review high	236,710	57,731	20%	5,298	0.53	$98	1.8
Codex CLI high	46,072	273,171	86%	2,538	0.14	$27	2.1
OpenCode native + PR text	63,952	231,632	78%	1,986	0.17	$32	2.3
PR-Agent	27,074	0	0%	1,131	0.07	$12	1.9

By size (insertions plus deletions; median 118 changed lines, largest 996): on PRs of at most 50 lines (48 PRs, 63 defects), 51–200 lines (70 PRs, 177 defects) and 201–1,000 lines (67 PRs, 280 defects), the default finds 70%, 76% and 82%. Claude Code goes the other way (67%, 64%, 60%), as does Codex CLI from the middle bucket (51%, 60%, 53%). On the smallest PRs the default's lead over Claude Code is 3 points; on the largest it is 22.

8.4 AACR-Bench's own metric

On AACR-Bench's own metric (Table 12) every reviewer recovers between 4.3% and 10.7% of the references. The default has the highest recall and F1, open-code-review the next F1, and PR-Agent and open-code-review the best precision, which is what posting few comments earns under a metric that counts every comment not on the list against precision. With the bot-derived references removed the default and open-code-review stay first and second on F1 and the default first on recall; lower down, Claude Code passes PR-Agent on F1 (8.8% against 8.6%) and both PR-Agent and the panel on recall, and moves to the top on precision. The two metrics order the middle of the field differently: pi-review's panel is third on defects and fifth on AACR recall, PR-Agent last on defects and fourth on AACR recall. Section 10 explains why.

8.5 Cost, tokens and time

QuickReview's default is the most expensive reviewer: $1.36 and a median 9.5 minutes per PR, 2.6 times open-code-review, 5 times the panel, 7–9 times Claude Code, OpenCode and Codex CLI and 21 times PR-Agent; it writes 5.1 to 24 times their output tokens (Table 13, Figure 8). Every other reviewer is cheaper per defect found (Table 9): Codex CLI finds a high-severity defect for $0.33 and pi-review's panel for $0.62, against $3.01 for the default. Going from the panel to the default adds 61.5 defects found for about $201 more over the 185 PRs, about

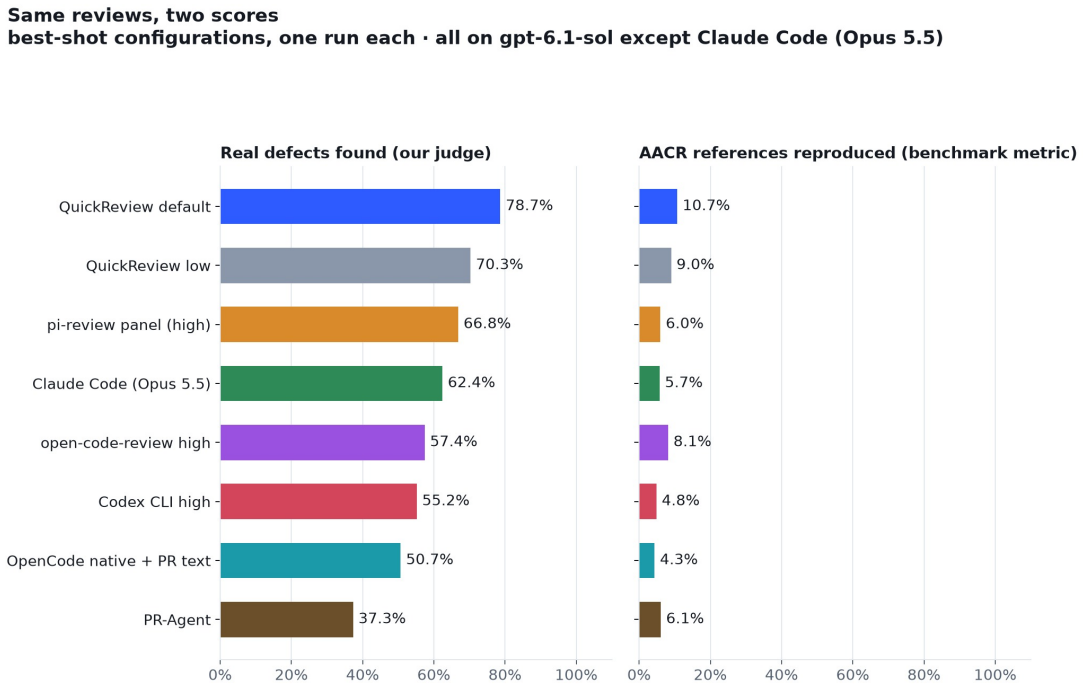


Figure 7: The final comparison's reviews on the defect judge and on AACR-Bench's reference metric.

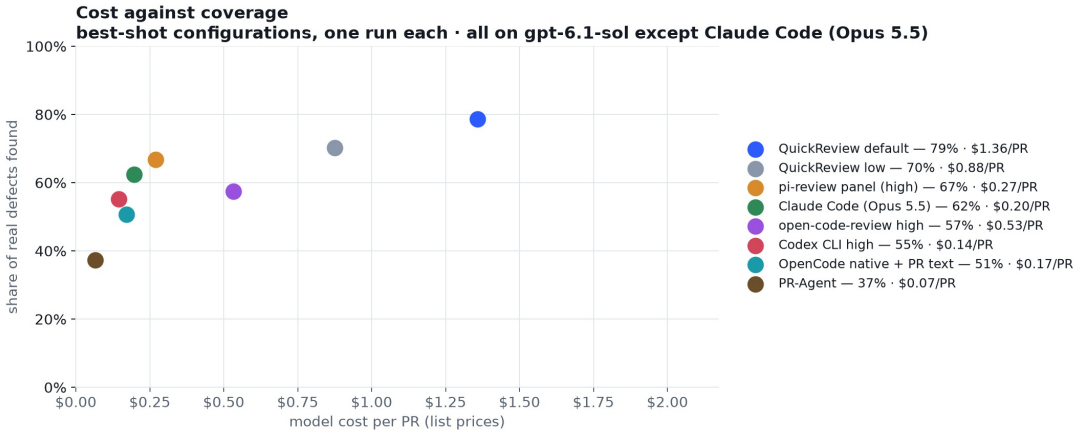


Figure 8: Model cost per PR (list prices) against the share of real defects found, final comparison. Linear axes.

$3.27 per additional defect, almost all of them low severity. On a cost–recall plot (Figure 8) PR-Agent, Codex CLI, Claude Code, the panel, `low` and the default each lie on the frontier of what is cheapest for its recall, so which reviewer is "best" depends on the price a team puts on a low-severity finding [111]. The companion report [1] explains where QuickReview's cost goes. Minutes are wall-clock under a shared scheduler with contention, not a speed comparison.

8.6 Tuned and unseen PRs; the reviewer-independent subset

QuickReview's levers were designed on T40; R145 was never looked at for its design. Every reviewer scores lower on T40 (the default 70% against 81% on R145; the panel 62% against 69%; Claude Code 60% against 63%), so T40 is harder for everyone (its PRs were selected by changed-line windows and, for 30 of them, a minimum reference count, Appendix B), and the

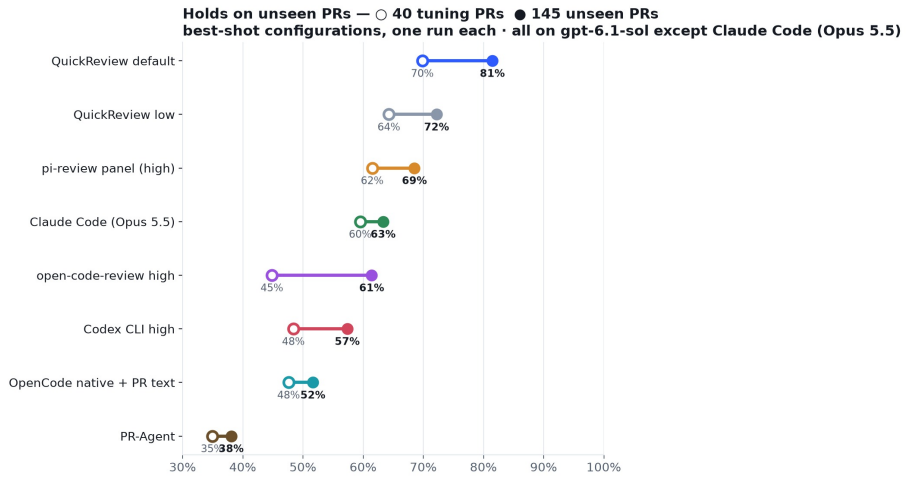


Figure 9: Recall on T40, the tuning PRs (open circles), and on R145, the unseen PRs (filled), final comparison.

default shows no visible in-sample advantage. The default's recall minus the mean of the other seven reviewers is 18.2 points on T40 and 22.5 on R145; in the earlier study the same margin was 29.0 and 27.6 points (difference +1.4, 95% CI −6.3 to +8.9, Section 9), so no tuning advantage is measurable, but the intervals are wide enough to hide one of several points. Decisions taken by looking at test data can overstate results on it [44], while a drop on fresh data alone does not prove overfitting [43]; here the fresh PRs are the easier ones, so neither effect is visible (Figure 9; per-set figures in Table 29).

The key was built partly from reviewers' comments (Section 5). On the 369 of the 520 defects (82 of the 90 high) that the expert hunt or an AACR reference raised, which would be in the key had no reviewer's comment ever been a candidate, the default finds 81% and 76.5 of the 82 high-severity ones; then `low` 74%, Claude Code 70%, the panel 69% (75 of 82 high), Codex CLI 60% (76 of 82 high), open-code-review 59%, OpenCode 58% and PR-Agent 43%. The default's lead over the next third-party tool is 10 points. Every reviewer scores higher on this subset, which leans towards defects easier to see from the diff. Two limits: membership in this subset does not depend on any reviewer, but the *definition* of a defect, its severity and its "counts if" notes were still written by a verifier with every reviewer's comments in view; and the stricter leave-one-reviewer-out check was computed only for the earlier study (Section 9).

A less strict check keeps every defect that any source other than QuickReview's own earlier runs raised: the hunt, an AACR reference, or the comment of another reviewer (open-code-review's earlier runs on R145 and the four extension reviewers). It drops the 28 defects (1 high, 3 medium, 24 low) whose only sources are QuickReview's earlier default and `low` runs. On the remaining 492 defects the order is unchanged and every reviewer's recall rises, because the dropped defects are ones the others mostly missed: the default 79.4% (82.5 of 89 high), `low` 71.4%, the panel 68.7%, Claude Code 65.7%, open-code-review 59.0%, Codex CLI 57.5%, OpenCode 52.7% and PR-Agent 39.1%. The default's lead over the panel is +10.7 points [+6.5, +14.7] on this key, against +11.8 on the full key; the default had 18.5 of its 409 credits on the dropped defects, the panel 9.5.[17] This is a leave-QuickReview-out key, not a symmetric leave-each-reviewer-out key: no other reviewer's own contribution is removed.

[17]Computed for this check by `paper-data/factcheck_analyses.py` from the `provenance` fields of `defects/*.json`, the per-PR candidate keys of R145 (`candidates-key.json`, `new-candidates-key.json`) and the primary judge's verdict files; paired bootstrap as in Table 9; no model calls. On T40 no per-comment key survives, so every reviewer-comment source there (QuickReview's and, on smoke10/dev10/holdout10, open-code-review's earlier runs) is treated as QuickReview's; that drops 7 of the 28 and errs against QuickReview.

Table 14: The frozen build against the compared default and the strongest third-party tools, 185 PRs, primary judge, same key; a partly found defect counts $\frac{1}{2}$. Sets: R145, whose misses the frozen build's changes were mined from; smoke10 and dev10, used to validate them; holdout10 and fresh10, kept out of that work. List prices, not bills.[19]

	QuickReview frozen	QuickReview default	pi-review panel	Claude Code	Codex CLI
Found, all 185 PRs (520)	**461 (89%)**	409 (79%)	347.5 (67%)	324.5 (62%)	287 (55%)
high (90)	**87.5**	83.5	80	72.5	81
medium (145)	**134.5**	121	119	105	101.5
low (285)	**239**	204.5	148.5	147	104.5
Weighted recall	**91%**	82%	74%	68%	65%
R145, mined (394)	**356 (90%)**	321 (81%)	270 (69%)	249.5 (63%)	226 (57%)
smoke10 + dev10, validation (71)	**59.5 (84%)**	49.5 (70%)	38 (54%)	39 (55%)	32 (45%)
holdout10 + fresh10, kept out (55)	**45.5 (83%)**	38.5 (70%)	39.5 (72%)	36 (65%)	29 (53%)
Comments per PR	6.9	5.4	3.2	2.2	1.9
Comments correct	92%	95%	89%	81%	93%
of which graded incorrect	4.5%	2.5%	6.0%	11.8%	4.4%
$/PR (mean)	2.06	1.36	0.27	0.20	0.14
Median min/PR	17.5	9.5	1.8	0.7	2.1

8.7 The frozen build

The comparison above is QuickReview as of 2026-10-01 (build `b2ce03a`). It was then changed for three more days and frozen (build `e32f0cf`, 2026-10-04), on the same model, gpt-6.1-sol. The changes, described in the companion report [1]: two lenses (one compares base and head behaviour on every modified or deleted line of non-test code where base behaviour is at risk; one is planned only when CI runs a linter or formatter over a changed file); shared instructions to check a confirmed mechanism's sibling sites and consequences, to anchor a finding at its cause, and that no token, time or finding budget applies; a per-file coverage check that sends a targeted sweep to changed source files no lens commented on; a deterministic merge of duplicate findings into one comment per root cause; the baseline, sweep and behaviour-comparison lenses at high reasoning effort; and a prompt cleanup that cut the composed lens instructions by 32% in tokens.

Which sets its development used is listed at the head of Section 13. The build ran once on all 185 PRs and was judged once by the primary judge; AACR-Bench's own metric was not computed for it.

The frozen build finds 87.5 of the 90 high-severity defects (97.2%, 95% CI 93.9 to 99.5; Table 14), ahead of the panel by +8.3 points [+1.9, +14.6] and of Codex CLI by +7.2 [+0.6, +14.1]; 94.5% [91.2, 97.2] of the 235 high- and medium-severity defects, ahead of the panel by +9.8 [+5.1, +14.3] and of Codex CLI by +16.8 [+11.3, +22.4]; and 461 of all 520 (88.7%, [85.7, 91.4]), 10.0 points more than the compared default [+6.8, +13.3], 21.8 more than the panel [+17.8, +25.7], 26.2 more than Claude Code and 33.5 more than Codex CLI. Unlike the compared default, its lead is not confined to low severity.

What it costs, and what it posts. $2.06 per PR on average (median $1.87, $380 for the 185 PRs), 51% more than the compared default and 4 to 29 times the third-party tools (Table 13); a median 17.5 minutes per PR, 1.8 times the default's 9.5, against 0.7–2.3 minutes for the third-party tools. It posts 6.9 comments per PR against 5.4, 42% of them true but minor (34% for the default) and 4.5% graded incorrect (2.5%); 40% of its comments map to a verified defect, against 56%. That is the trade the design makes: more of the real problems found, for more comments, money and time.

By set. The gain over the default is similar on the three groups (Table 14): 9 points on R145, 14 on smoke10 and dev10, 13 on holdout10 and fresh10. On those last 20 PRs the frozen

[19] Runs `final1-{smoke10,dev10,holdout10,fresh10,rest145}`, image `qcr-bench:final1`, 2026-10-04, with `compare.py` as in Table 8; intervals below by the paired bootstrap of Table 9 (10,000 resamples, seed 20261001) over the same judge files; no model calls.

Table 15: Comments the judge flagged as a possible real defect missing from the key (`new_defect`), 185 PRs. In the pool: the reviewer's comments were candidates when the key was built. These flags are the judge's reading; none has been verified against the code.

Reviewer	Pool	Flags	Per PR	Per 100 comments	On R145
QuickReview default	out	61	0.33	6.1	55
QuickReview low	out	22	0.12	2.7	21
pi-review panel (high)	out	47	0.25	8.0	42
Claude Code (Opus 5.5)	in	0	0.00	0.0	0
open-code-review high	out	49	0.26	8.6	46
Codex CLI high	out	26	0.14	7.6	25
OpenCode native + PR text	out	10	0.05	2.8	10
PR-Agent	out	4	0.02	1.0	4
Earlier gpt-6-sol runs (6; Claude Code above)	in	2–34	0.01–0.18	0.4–6.1	2 each

build leads the default by +12.7 points [+1.0, +24.6] and the panel by +10.9 [+0.8, +20.3]. The two earlier-study builds on gpt-6-sol found 50.5 and 48.5 of those 55 defects (Section 9.1), 3 to 5 more than the frozen build, a gap at the edge of one build's run-to-run spread (1.5–2 found defects per 10-PR set) and on a different model.

8.8 What the key cannot credit: defects outside the pool

The candidate pool from which the key was built contains the comments of the earlier gpt-6-sol runs (Figure 3), not those of the final gpt-6.1-sol runs. Claude Code's final run is its earlier run, so it alone is fully in the pool. A real defect that only a final run raised is in no reviewer's recall, and its comment is graded correct without being credited. The judge records exactly this case: its `new_defect` field marks a correct, valuable comment that describes a real defect which is neither in the key nor among its recorded exclusions (both are in the judge's prompt).

On R145, where the earlier runs were fully in the pool, they leave at most 2 such flags each; the final runs leave 4 to 55 (Table 15).[20] So the key probably misses some real defects that only the final runs found, and every final reviewer's recall is understated by its share of them. In absolute terms the default has the most flags (61); per comment open-code-review (8.6 per 100), the panel (8.0) and Codex CLI (7.6) have more than the default (6.1). Taking every flag as a distinct real defect, which overstates it (several reviewers often flag the same problem), would add at most 0.33 defects per PR to the default's credit (fewer to any other reviewer's), against a key of 2.8 per PR. We have not verified the flags against the code; extending the key with the final runs' comments, as was done once before (Section 5.3), would settle it.

8.9 Worked examples

The examples below are taken from the final comparison (eight reviewers, gpt-6.1-sol except Claude Code on Opus 5.5, judge gpt-6-sol at high reasoning). Comment text is quoted verbatim from the scored review files; only Markdown markup is rendered (backticks as typewriter, double asterisks as bold), and cuts are marked [. . .]. "Found", "partial" and "missed" are the defect judge's verdicts; a partial counts as half in every recall figure. Code excerpts are from the PR head, with line numbers; indentation common to the shown lines is removed. Each quoted excerpt remains under its repository's own licence at the reviewed commit; all six repositories

[20] The earlier runs' residue is on T40, whose keys were built from older runs: OpenCode 32 and Codex CLI 8 flags there. Computed from the `new_defect` field of the primary judge's verdict files, 2026-10-01; the 219 flagged comments of the final runs, and 45 groups where several reviewers flag the same file of the same PR, are listed in the analysis notes.

quoted here are open source.[21] The examples were picked to cover each severity and each kind of verdict, not to flatter QuickReview: in the medium example it is outperformed by five other reviewers.

A high-severity defect: a shared AST node re-parented (Go). `microsoft/typescript-go@b970689` (PR #831), `internal/parser/reparser.go:120--126`. The PR turns a JSDoc `@import` tag into a synthetic import declaration, but clones only the import-clause shell: the bindings, module specifier and attributes are the same nodes as in the JSDoc tag. For any host that is not a function declaration, binding the host afterwards re-parents those shared nodes to the JSDoc tag, and the checker's walk up the parent chain then reaches a node kind it no longer handles and panics, so a file containing `/** @import { Foo } from './types' */ export const x = ...` crashes type-checking. The PR's conformance tests use only function hosts, which is why they pass.

```
118  case ast.KindJSDocImportTag:
119      importTag := tag.AsJSDocImportTag()
120      importClause := importTag.ImportClause.Clone(&p.factory)
121      importClause.Flags |= ast.NodeFlagsReparsed
122      importClause.AsImportClause().IsTypeOnly = true
123      importDeclaration := p.factory.NewJSImportDeclaration(
             importTag.Modifiers(), importClause,
             importTag.ModuleSpecifier, importTag.Attributes)
124      importDeclaration.Loc = core.NewTextRange(tag.Pos(), tag.End())
125      importDeclaration.Flags = p.contextFlags | ast.NodeFlagsReparsed
126      p.reparseList = append(p.reparseList, importDeclaration)
```

QuickReview default: **found**	QuickReview low: missed	pi-review panel: **found**	Claude Code: missed
open-code-review: *partial*	Codex CLI: missed	OpenCode: missed	PR-Agent: missed

QuickReview default, found (QuickReview default, comment C2 at `internal/parser/reparser.go:123`; graded *correct-valuable*):

> Shared children make ordinary import tags crash checking
>
> Checking an import comment attached to a variable can panic because its bindings lose their synthetic declaration ancestry. [...] The synthetic declaration binds before its host (parser.go:478–482; binder.go:1642–1653), but binding the variable then calls setJSDocParents, which reparents the shared children under the original tag (binder.go:757–761,1616). Namespace alias checking follows Parent.Parent into KindJSDocImportTag, which getModuleSpecifierFromNode does not handle (checker.go:13770–13773,14149–14179); named imports have the analogous GetExternalModuleName failure. [...]

Strongest competitor that found it (pi-review panel, comment C2 at `internal/parser/reparser.go:120`; graded *correct-valuable*):

> Shared import children are reparented back into the original JSDoc tree [...] After binding the synthetic import, binding its non-function host calls `setJSDocParents`, which recursively overwrites those shared children's parents (`internal/binder/binder.go:757-760`). [...] `getTargetOfNamespaceImport` then reaches `getModuleSpecifierFromNode`, whose switch does not handle that kind and panics. [...] Function-only baselines conceal this because the binder processes functions before other statements. [...]

Partial (open-code-review, comment C2 at `internal/ls/utilities.go:23`; graded *partly-correct*):

> [P2] Keep `KindJSDocImportTag` in this case as well. The reparser shares the original module-specifier node with the synthetic `JSImportDeclaration`, and binding the host subsequently calls `setJSDocParents`, which assigns that node's parent back to the original JSDoc import tag. [...] Consequently, a module specifier in `/** @import { Foo } from "./..." */` can still have a `KindJSDocImportTag` parent; this change makes the helper return nil and causes `isValidTrigger` to reject `/` completion triggers in those imports.

[21] Licences read at the base commit (licence review of 2026-09-28): browser-use, bitcoin and ollama MIT; typescript-go, gofr and dbeaver Apache-2.0.

Judge, on the partial: *"C2 identifies the shared module specifier's stale JSDoc parent and lost completion trigger, but misses the resulting type-checker panic."* open-code-review saw the same mechanism but followed it only to the lost completion trigger, the minor symptom, not to the type-checker crash.

A medium-severity defect that QuickReview only partly found (Python). `browser-use/browser-use@72f1585` (PR #3172), `browser_use/llm/google/chat.py:78` and `:201--203`. The PR changes the default of `thinking_budget` from `None` to `0`, so the `is not None` guard is always true and every default request now carries a `thinking_config`, including requests to the Gemini 2.0 and Gemma models the same file lists, which do not support thinking. Callers that never set the parameter (the repository's own examples and tests use `gemini-2.0-flash`) can now get an invalid-argument error.

```
 24  VerifiedGeminiModels = Literal[
 25      'gemini-2.0-flash',
 26      'gemini-2.0-flash-exp',
    ...
 78      thinking_budget: int | None = 0
    ...
201          if self.thinking_budget is not None:
202              thinking_config_dict: types.ThinkingConfigDict = {'thinking_budget': -1}
203              config['thinking_config'] = thinking_config_dict
```

QuickReview default: *partial*	QuickReview low: missed	pi-review panel: **found**	Claude Code: **found**
open-code-review: **found**	Codex CLI: **found**	OpenCode: missed	PR-Agent: **found**

QuickReview default, partial (QuickReview default, comment C2 at `browser_use/llm/google/chat.py:78`; graded *partly-correct*):

> Preserve thinking configuration when the budget argument is omitted
>
> Callers supplying thinking_config through config now silently lose their settings even without requesting an override. For example, config={'thinking_config': {'thinking_budget': 1024}} previously survived because thinking_budget defaulted to None; the new default 0 enters lines 201–203 and replaces the copied configuration with -1. [...]

Judge, on the partial: *"C2 identifies a regression caused by the new default at chat.py:78, but misses that it sends thinking configuration to unsupported models."* The default anchored a blocker on the right line and the right cause but described the other consequence (a caller's own `thinking_config` being overwritten); it missed this PR's retry-removal defect (D2) altogether. **Strongest competitor that found it** (pi-review panel, comment C2 at `browser_use/llm/google/chat.py:78`; graded *correct-valuable*):

> Default thinking configuration breaks non-thinking models
>
> Evidence: The default changes from `None` to `0`, so every default invocation now enters lines 201–203 and sends `thinking_config`. This is unconditional even for listed models such as `gemini-2.0-flash` and Gemma, which do not support configurable thinking. [...]

The cheapest reviewer, also found (PR-Agent, comment C2 at `browser_use/llm/google/chat.py:78`; graded *correct-valuable*):

> Unsupported configuration
>
> Changing the default `thinking_budget` from `None` to `0` causes requests to include `thinking_config` even for non-thinking models such as `gemini-2.0-flash`. Those models reject this configuration, so previously working calls with default settings can now fail with an API validation error. [...]

A low-severity defect, and a partial for a correct observation dismissed (C++). `bitcoin/bitcoin@e98c51f` (PR #32425), `src/init.cpp:753--766`. The PR makes proxy setup read every `-proxy` value (`GetArgs`, last one wins), but the privacy parameter interaction still reads the single `GetArg("-proxy")`, which returns the *first* config-file value. With `proxy=0` followed by `proxy=127.0.0.1:9050` in `bitcoin.conf`, the proxy is installed while listening, NAT-PMP and address discovery stay on.

```
 753  std::string proxy_arg = args.GetArg("-proxy", "");
 754  if (proxy_arg != "" && proxy_arg != "0") {
 755      // to protect privacy, do not listen by default if a default proxy server is specified
 756      if (args.SoftSetBoolArg("-listen", false))
 757          LogInfo("parameter interaction: -proxy set -> setting -listen=0\n");
     ...
 766  }
     ...
1598  for (const std::string& param_value : args.GetArgs("-proxy")) {
```

QuickReview default: **found**	QuickReview low: **found**	pi-review panel: **found**	Claude Code: *partial*
open-code-review: missed	Codex CLI: **found**	OpenCode: **found**	PR-Agent: missed

QuickReview default, found (QuickReview default, comment C5 at `src/init.cpp:1598`; graded *correct-valuable*):

> Derive privacy defaults from the effective proxy configuration
>
> A node with an active proxy can still listen and discover and advertise its clearnet address by default. With consecutive bitcoin.conf entries proxy=0 and proxy=127.0.0.1:9050=ipv4, this loop installs the later IPv4 proxy, but InitParameterInteraction at lines 753–768 reads scalar GetArg, which selects the first configuration value (src/common/settings.cpp:162–169,195), and skips disabling listening and discovery. [...]

Strongest competitor that found it (pi-review panel, comment C2 at `src/init.cpp:1598`; graded *correct-valuable*):

> Effective proxies can leave listening and address discovery enabled
>
> Evidence: Proxy initialization now processes every configuration value, but `InitParameterInteraction()` still uses `GetArg("-proxy", "")` at line 753. For configuration entries `proxy=0` followed by `proxy=127.0.0.1:9050`, `GetArg()` returns the first value while this loop installs the second proxy. [...]

Partial (Claude Code, from its review body, under "Two behaviours I looked at and chose not to report"):

> **Listening:** the check that turns off `-listen`, `-natpmp` and `-discover` (`src/init.cpp:753`) still only looks at the last `-proxy` value. As far as I can tell that only ever makes it more cautious, and never turns listening back on when a proxy is still in use.

Judge: *"The body points to the single-value parameter-interaction check but incorrectly says it reads the last value and cannot leave listening enabled while a proxy is active."* Claude Code looked at the right line and dismissed it on a wrong premise (`GetArg` returns the first config-file value, not the last).

A comment graded correct-minor. A comment can be true and still not a defect. On `gofr-dev/gofr@dc0b3e6` (PR #1355, Go) the PR builds a MongoDB URI with `url.URL`:

```
 149  u := &url.URL{
 150      Scheme: "mongodb",
 151      Host:   fmt.Sprintf("%s:%d", config.Host, config.Port),
 152      Path:   "/" + url.PathEscape(config.Database),
 153  }
 154
 155  if config.User != "" && config.Password != "" {
 156      u.User = url.UserPassword(url.QueryEscape(config.User), url.QueryEscape(config.Password))
 157  }
```

Correct-minor (Claude Code, comment C3 at `pkg/gofr/datasource/mongo/mongo.go:154`; graded *correct-minor*):

> **mongo.go:154** — The database name is escaped with `url.PathEscape` and then put into `u.Path`, so `u.String()` escapes it again. For example, `my db` becomes `/my%2520db` in the URI.

Judge: *"mongo.go:153 double-escapes the URI path, but authSource and the separately selected database make this inconsequential."* The key had already rejected the same claim when it was built (it was in the anonymised candidate pool), with the reason: *"Claims url.PathEscape on the database name plus URL.String double-encodes the path ('my db' -> 'my%2520db'). The encoding is doubled, but it has no consequence: authSource=admin is always set (mongo.go:160) so the path database is not used for auth, and the actual database is selected with m.Database(c.config.Database) (line 125)."* Three lines further down, line 156 double-encodes the user name and password the same way, and there it matters: credentials with special characters fail authentication. That is a high-severity defect in the key (D1), and all eight reviewers found it.

A comment graded incorrect. On `dbeaver/dbeaver@ed40132` (PR #38239, Java) the PR makes object filters match a function by either its bare name or its signature (`DBSObjectFilter.matches(String... names)`, which applies include and exclude patterns if any name matches). The QuickReview default posted this as a blocker: **Incorrect** (QuickReview default, comment C1 at `plugins/org.jkiss.dbeaver.model/src/org/jkiss/dbeaver/model/navigator/DBNDatabaseNode.java:730`; graded *incorrect*):

> Preserve existing name-only wildcard filtering
>
> Existing filters can now unexpectedly hide functions or show functions they previously excluded. Every mask is matched against both the bare name and the full signature: an include mask `%text%` now admits `foo(text)`, and an exclude mask `%text%` now hides it, although its name remains `foo`. [...]

Judge: *"DBSObjectFilter.java:163-188 does match either name, but applying include and exclude masks to signatures is the intended behavior, not a defect."* The key had rejected the same claim, from the anonymised candidate pool it was built from: *"Claims exclude patterns are now also checked against the overloaded signature, so an infix exclude such as '%char%' hides functions like 'get_user(character varying)' merely because of their argument types. This is the PR's stated contract (an include or exclude applies if any of the names matches, DBSObjectFilter.java:163-188 via atLeastOneNameMatchesPattern), required so the navigator's 'hide' action, which stores the signature display name, works; prefix patterns behave as before because the overloaded name starts with the plain name, and the claimed hit needs an infix wildcard exclude that coincidentally matches a type name - speculative, no concrete regression shown."* Claude Code made the same claim and was graded incorrect for the same reason. The same QuickReview review also found the PR's one defect in the key (signature matching applied in the navigator but not in SQL completion or Go-to-object).

What AACR's metric rewards, on one PR. `ollama/ollama@b6002f6` (PR #12185, Go; a smoke10 PR, and one of the 16 PRs the QuickReview benchmark-driven levers were designed from) carries 12 reference comments. One of them, at `model/model.go:285`, reads:

> The change from `len(batch.Positions)` to `batch.Positions.Dim(0)` is correct for the new tensor-based API. However, there's a potential nil pointer dereference risk if `batch.Positions` is nil. Consider adding a nil check before calling `Dim(0)`, or ensure that batch validation happens earlier in the call chain to guarantee `Positions` is always initialized.

The reference labels mark it *dubious*, and the key rejects its claim against the code: *"A nil Positions tensor in model.Forward is unreachable: its only callers (forwardBatch runner.go:582 and reserveWorstCaseGraph runner.go:1069) always set Positions, and forwardBatch returns early when the batch is empty."* Three reviewers matched it in all three AACR judge rounds: pi-review panel, open-code-review, PR-Agent. PR-Agent's matching comment was: **Matched on AACR** (PR-Agent, comment C1 at `model/model.go:285`; graded *incorrect*):

> Nil dereference
>
> Calling `Forward` with an empty `input.Batch` now panics because `batch.Positions` is a nil interface. Previously, the length checks returned a validation error for this input. Check that `batch.Positions` is non-nil before calling `Dim` so malformed batches continue to return errors rather than panic.

Our judge graded it *incorrect*: *"Although a zero-value batch would panic at model/model.go:285, the in-tree caller always supplies Positions and returns before calling Forward for an empty batch."*

On the same PR the key's one high-severity defect is on no reference list (its `provenance.dataset_refs` is empty). In fit mode the runner builds the positions tensor without allocating memory, so ggml gives it a placeholder data pointer; the new `Ints()` wraps that pointer as a Go slice, and the llama4 and mllama reservation paths then read it, so the model load crashes and both architectures cannot be loaded at all:

```
1029  func (t *Tensor) Ints() (data []int32) {
1030      if t.sync != nil {
1031          data = make([]int32, C.ggml_nelements(t.t))
      ...
1035      } else if bts := C.ggml_get_data(t.t); bts != nil {
1036          // tensor is in cpu memory
1037          data = unsafe.Slice((*int32)(bts), C.ggml_nelements(t.t))
1038      }
1039
1040      return
1041  }
```

Seven of the eight reviewers found it; PR-Agent did not. Here is the QuickReview default's comment on it: **Found, worth nothing on AACR** (QuickReview default, comment C1 at `ml/backend/ggml/ggml.go:1035`; graded *correct-valuable*):

> Allocation-free model fitting can crash when positions are read
>
> Model fitting for Llama4 and multimodal mllama can crash instead of returning a memory estimate. [...] The no-allocation GGML buffer supplies an alignment-valued placeholder pointer (ggml/src/ggml-backend.cpp:133–136), and FromIntSlice skips writing values in this mode. This branch treats that non-null pointer as readable storage and wraps it with unsafe.Slice. [...]

AACR's per-PR reference recall on this PR (references matched, averaged over its three judge rounds, out of 12): QuickReview default 0, QuickReview low 0, pi-review panel 1, Claude Code 0, open-code-review 1, Codex CLI 0, OpenCode 0, PR-Agent 1. The three reviewers that repeated the unreachable nil check score 1 and the other five, which matched no reference, score 0. All five found the load crash, which no reference names; PR-Agent, which missed it, scores as high on this PR as anyone. The defect benchmark credits the seven that found the crash and grades the three nil-check comments incorrect.

9 Robustness checks from the earlier study

Before the best-shot protocol, the benchmark ran every third-party tool as shipped or as the AACR harness runs it, on gpt-6-sol (the four third-party OpenAI-based tools at medium reasoning), with Claude Code on Opus 5.5 (2026-09-28/30). That study's third-party rows are *not*

Table 16: The earlier study (every third-party tool as shipped; gpt-6-sol except Claude Code on Opus 5.5): 185 PRs, 520 defects, under both judges. "Current build" is the 2026-09-29 re-run of QuickReview default (Section 9.1); every other row is the original seven-reviewer comparison. AACR F1: harness metric as in Section 6.3.

	Primary judge (gpt-6-sol, high)				Second judge (Opus 5.5)			
Reviewer	Found	Recall	High (90)	Correct	Recall	Correct	$/PR	AACR F1
QuickReview default, current build	436	84%	85	88%	—	—	1.71	12.6%
QuickReview default	437.5	84%	86	86%	84%	86%	1.71	12.6%
QuickReview low	345	66%	75	80%	66%	82%	0.80	11.9%
Claude Code (Opus 5.5)	324.5	62%	72.5	81%	62%	86%	0.20	8.9%
pi-review (medium, 1 reviewer)	292.5	56%	71.5	79%	56%	79%	0.11	9.1%
open-code-review (medium)	291.5	56%	64.5	73%	56%	77%	0.60	14.0%
OpenCode (bridge)	250	48%	64.5	55%	48%	55%	0.83	8.4%
Codex CLI (medium)	248.5	48%	70	69%	48%	69%	0.15	6.9%

the tools' best shots: three of them ran in configurations that disfavoured them (Section 7.1). We keep it because every check that the final comparison lacks was run on it: a second judge over all 1,295 reviews (Section 6.2), a full reproduction and repeat runs of QuickReview, and the leave-one-reviewer-out key. Table 16 summarises it; per-set, grade, label and bot-reference tables are in Appendix C. The two studies agree on the order at the top (QuickReview default, then QuickReview low, then the strongest third-party tool); they do not agree on the size of the default's lead over the strongest third-party tool, which shrinks from 22 points (over Claude Code) to 12 points (over pi-review's panel, which this study ran at medium with one reviewer).

Under the primary judge every difference between the default and another reviewer in this study excludes zero over PR resampling (the smallest, over `low`, is +17.8 points, 95% CI +13.4 to +22.3); the order within three pairs is not established: `low` vs Claude Code (+3.9, −1.9 to +9.5), pi-review vs open-code-review (+0.2, −5.3 to +5.9) and OpenCode vs Codex CLI (+0.3, −5.3 to +5.6).[22] Under the second judge the order by problems found is unchanged except that pi-review and open-code-review, 1 and 1.5 defects apart under the two judges, swap places; the rank correlation of the seven reviewers between the judges is Kendall's $\tau = 0.90$ (one discordant pair of 21).[23]

9.1 Stability of QuickReview default

A full reproduction. Between the original run and the 2026-09-29 re-run of QuickReview default, three reviewer changes landed: one (the validator became optional and its veto a mark rather than a deletion), one (three frozen passes made stack-neutral, their stack-specific rules moved to repository-profile addenda) and one (the style guide also read from agent-directive files). On all 185 PRs the re-run found 436 defects against 437.5, 85 high-severity against 86, and scored AACR F1 12.6% against 12.6%; comments correct moved from 86% to 88%. On R145 the two runs find the same number of defects (335.5 each); the whole 1.5-defect difference is on T40 (100.5 against 102).[24]

Run-to-run spread. The default ran four more times (`stab-r1` … `stab-r4`, build `9106a64`) on 8 PRs: `ppy__osu@748ab3d`, `microsoft__typescript-go@1a84d8d` and six from R145. Two

[22] Computed from the primary judge's verdict files, seed 20260928, 10,000 resamples, 2026-09-28; per-arm intervals in Table 32.

[23] From the problems-found columns of the two judges (`FULL-BENCHMARK.md`, Tables S1 and S2): 437.5/439, 345/345.5, 324.5/321.5, 292.5/289.5, 291.5/291, 250/249.5, 248.5/247.

[24] Computed from runs `v2-default-*` (build `9106a64`) against the original default runs with `compare.py`, primary judge, 2026-09-29/30; the per-half figures are the T40 row of the companion report's base-model study [1] and its difference from the 185-PR total.

Table 17: Six runs of QuickReview default on the same 8 PRs (17 defects on the 6 PRs that have any), primary judge.

Run	Found / 17	Recall	High (1)	Med. (5)	Low (11)	Correct	$/PR
stab-r1	11	65%	1	3	7	76%	1.97
stab-r2	14	82%	1	5	8	70%	1.92
stab-r3	12.5	74%	1	4	7.5	78%	2.06
stab-r4	11.5	68%	1	5	5.5	77%	2.03
current build, full re-run	13.5	79%	1	5	7.5	79%	2.01
original run	13.5	79%	1	5	7.5	64%	1.94

Table 18: Recall on the 369 defects the hunt or an AACR reference raised (high-severity found, of 82), both judges, with recall on all 520 for comparison.

Reviewer	GPT recall	GPT high	Opus recall	Opus high	All 520
QuickReview default	**86%**	**80**	**86%**	**80**	84%
QuickReview low	71%	71	71%	71.5	66%
open-code-review	59%	62	59%	62	56%
Codex CLI	53%	67.5	53%	67.5	48%
OpenCode	53%	62.5	53%	62.5	48%
pi-review	61%	69.5	60%	70	56%
Claude Code (Opus 5.5)	70%	69.5	70%	69.5	62%

of the eight have no defect in the key, so recall rests on the other six PRs and their 17 defects (Table 17).[25]

The six runs (five on the current build) range from 11 to 14 found of 17, recall 65–82%; most of the spread is one PR (`wavetermdev__waveterm@61b0fb4`, 2 to 4 of its 5 defects), and the one high-severity defect was found in every run. Earlier repeat runs of older QuickReview builds on T40 differed by about 1–2 found defects per 10 PRs (54.5 vs 52.0 of 97 over 30 PRs for two runs of one build), and pairs of runs on fresh10 agreed on 78–89% of the defects either found. Per 10-PR set the two runs of one build differed by 1.5–2 found defects, 4–8 points of that set's 26–37 defects.[26] The 185-PR reproduction is the stronger evidence that the headline is not one lucky run.

9.2 Seeding bias: independent-source and leave-one-reviewer-out keys

The obvious objection to a pooled key is that seeding it with QuickReview's comments inflated QuickReview's lead, since an in-pool reviewer gets credit for everything it said that turned out to be true. This study, whose reviewers were all in the pool, tests it in two ways.

The independent-source subset is the 369 defects (82 high) that the expert hunt or an AACR reference raised. 330 of them were also raised by a reviewer, but their presence in the key does not depend on it.

On this subset the default does as well as or better than on the whole key (86% against 84%) and leads every other reviewer by 15 points or more under both judges (Table 18, Figure 10). Every reviewer scores higher on it, so it leans towards defects that are easier to see from the diff, and it is not a measure of absolute recall. The 39 defects with no reviewer among their sources are not a usable test: on R145 (12 of them) every reviewer's comments were candidates, so "no

[25] Computed from runs `stab-r{1..4}-{dev10,fresh10,rest145}`, the `v2-default-*` runs and the original default runs, cut to the same 8 PRs, with `compare.py`, primary judge, 2026-09-29/30.

[26] `RESULTS.md`, "Noise": 15.5/17.0, 21.0/19.0 and 18.0/16.0 found on smoke10 (34 defects), dev10 (37) and holdout10 (26).

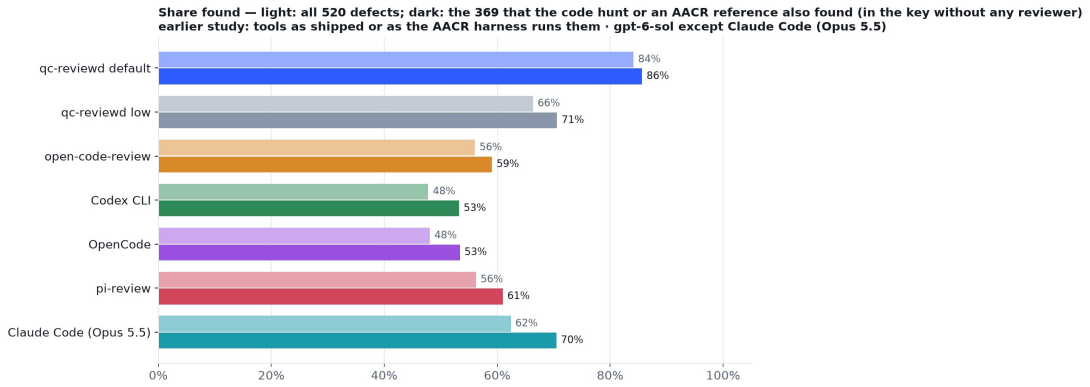


Figure 10: Share found on all 520 defects vs on the 369 raised by the code hunt or an AACR reference.

reviewer source" means every reviewer missed them; on T40 (27) the provenance records only the older runs from which those keys were built.

The leave-one-reviewer-out key scores each reviewer only on the defects that at least one source other than itself raised (the hunt, a reference or another reviewer's comment), which removes each tool's own seeding advantage symmetrically, as IR evaluation re-scores a system on a pool built without its own contribution [46, 48]. On R145, where every comment is traced to its reviewer, the default's key shrinks from 394 to 382 defects and its recall from 85.2% to 84.7%; Claude Code's key drops to 386 and its recall from 63.3% to 62.6%; OCR (391 defects) moves from 59.5% to 59.2%; Codex CLI, OpenCode and pi-review (393 each) lose 0.1 points; `low` (394) does not move. On all 185 PRs the same check moves Codex CLI, OpenCode, pi-review and Claude Code by 0.0–0.6 points. Own-seeding therefore explains at most 0.8 points of any reviewer's recall in this study.[27] The check removes a reviewer's *own* contribution only; it leaves a sibling configuration's (for QuickReview default, `low`'s) and it was not repeated for the final runs; for those, Section 8.6 removes both QuickReview configurations' contribution together.

Difficulty-adjusted halves. In this study the default's recall minus the mean recall of the six other reviewers is 29.0 points on T40 and 27.6 on R145. The difference, +1.4 points in favour of the tuning PRs, has a 95% bootstrap interval of −6.3 to +8.9 (PRs resampled within each half).[28] The two halves' keys also differ. By source type (hunt, AACR references, any reviewer comment), 23 of T40's 126 defects (18%) have one source against 151 of R145's 394 (38%), mostly defects raised only by reviewers (7 on T40, 144 on R145, where the verifier read the comments of three benchmarked reviewers). Per scored reviewer (found or partial by exactly one of the seven), T40 has more single-source defects: 16 of 126 (13%) against 34 of 394 (9%).[29] The tuning log gives a direct view of in-sample shrinkage for the levers themselves: on 16 in-sample PRs they added about 0.94 defects per PR, on fresh10 (before it was used for design) about 0.45, roughly half. Those were single runs on 16 and 10 PRs.

9.3 Copilot code review, 2025, as deployed upstream: an anecdote

This is not a controlled arm. GitHub Copilot code review [11] never ran in our sandbox. On 19 of the 185 PRs, the upstream repository had Copilot review enabled, and its reviews are still

[27] Computed from `provenance` in `truth-ext.json`, the candidate keys per PR and the primary judge's verdict files (`wp-data/prov.py`), 2026-09-29.

[28] Computed from the primary judge's verdict files, 5,000 resamples, 2026-09-28.

[29] Computed from `provenance` in `truth-ext.json`, the candidate keys per PR and the primary judge's verdict files (`wp-data/prov.py`), 2026-09-29.

Table 19: The 19 PRs where upstream ran Copilot code review (59 defects, 10 high). AACR columns: the 112 references no review bot wrote.

	Defect benchmark			AACR metric, bot-written references removed			
Reviewer	Found	High	Correct	Refs found	Recall	Precision	F1
QuickReview default	**50 (85%)**	**10**	88%	**15.0**	**13.4%**	12.5%	12.9%
QuickReview default, current build	46.5 (79%)	9.5	92%	9.7	8.6%	8.1%	8.3%
QuickReview default, gpt-6.1-sol	46.5 (79%)	**10**	**97%**	11.7	10.4%	11.6%	11.0%
QuickReview low	38.5 (65%)	9	84%	9.3	8.3%	10.6%	9.3%
Claude Code (Opus 5.5)	37 (63%)	8	83%	8.0	7.1%	19.0%	10.4%
open-code-review	34.5 (58%)	7.5	80%	11.7	10.4%	18.2%	**13.3%**
pi-review	32.5 (55%)	9	75%	9.0	8.0%	**22.5%**	11.8%
OpenCode	28 (47%)	9	56%	6.0	5.4%	10.2%	7.0%
Codex CLI	23.5 (40%)	9.5	66%	4.3	3.9%	11.4%	5.8%
Copilot 2025 (upstream)	6.5 (11%)	1.5	65%	2.0	1.8%	3.8%	2.4%

public on the original PR. We harvested them read-only with anonymous GitHub access and scored them like any other reviewer.[30] Four limits apply:

- **The repos chose the sample.** Ten of the 19 PRs come from typescript-go and cline alone.
- **The product is a 2025 version.** The reviews date from 2024-12-19 to 2025-09-02.
- **The inputs differ.** On 15 PRs Copilot reviewed AACR's head commit. On 4 it reviewed an earlier push. A comment from an earlier push kept its line only if the file, or the lines and three lines either side, were identical at both commits. That kept 4 of 7 lines; the other 3 are scored as path-only comments.
- **The AACR metric is contaminated in Copilot's favour.** 23 of AACR's "human" references on these PRs are LLM rewrites of Copilot's own inline comments.

Copilot's findings are its 41 inline comments plus the 11 items in its collapsed low-confidence block, 52 in all. Table 19 compares it with every other arm on the same 19 PRs.

The AACR columns of Table 19 use only the 112 references that no review bot wrote. Of the 154 AACR references on these PRs, 42 are LLM rewrites of review-bot comments on the original PRs: 23 of Copilot's own, 17 of cubic's and 2 of Ellipsis's. Scoring Copilot against rewrites of its own comments would measure whether it repeats itself, not whether it reviews well, so those 42 are removed for every reviewer, and a comment whose only match was a removed reference counts as unmatched.[31]

On both metrics Copilot is last on these 19 PRs. On the defect benchmark it found 6.5 of 59 defects and 1.5 of the 10 high-severity ones, and 34 of its 52 comments are correct. On the AACR metric with bot-written references removed it matches 2 of 112 references. On the unfiltered list of 154 it would instead look best of all (16.2% recall, 48.1% precision, F1 24.3%), because 23 of its 25 matches are against rewrites of its own comments; that is why no raw AACR figure appears in the table. The defect key is not free of this either. Five of the seven defects Copilot found list a Copilot-derived reference among their sources. Each was checked against the code, and the blind hunt raised it independently. Every reviewer is credited for those defects on the same terms.

This anecdote shows that the AACR metric can rank a reviewer from last to first when the reviewer wrote part of the reference list. It does not measure today's Copilot.

[30] Converter `bench/aacr/harvest_upstream.py`; runs `upstream-copilot2025-<set>`; harvested and scored 2026-09-30.

[31] `compare.py --exclude-refs bench/aacr/bot_references.json` on these 19 PRs. Dropping such comments from the count instead of counting them as unmatched puts Copilot's precision at 6.9%.

10 Why the two metrics disagree

Figure 7 shows the final comparison's reviews under both metrics. On the defect benchmark QuickReview default leads every other reviewer by 8 points or more, and the field spreads from 37% to 79%. On AACR's metric the default leads recall by under two points, open-code-review is within one point of it on F1, and five reviewers that are 30 points apart on defects (PR-Agent 37% to the panel 67%) are within 1.8 points of each other on AACR recall (4.3–6.1%); PR-Agent, last on defects, is fourth on AACR recall. We see four mechanisms behind the difference, all measured.

1. **Most references are not defects.** About 19% of labelled references name a real defect or a real maintainability issue; on R145 only 12.8% of references are part of a verified defect. A review that says only what is true and consequential cannot match the rest, and the rest are the majority of the recall denominator.
2. **Most real defects are not references.** A true finding that is not on the list is a false positive under AACR precision; only 31% of the verified defects have any reference among their sources. QuickReview default posts the most comments (999) and the most true ones, so it pays the largest precision penalty. Not all of that penalty is the list's fault: by our own judge 56% of the default's comments map onto a verified defect, the lowest share of the eight (`low` 58%, open-code-review 63%, the panel 67%, PR-Agent 77%, Claude Code 81%, Codex CLI 85%, OpenCode 87%); the rest are mostly true minor notes (Table 10).
3. **The line gate.** A comment is judged only if it lands within ±1 line of a reference. The right problem anchored two lines away is never judged. Tools that comment on line ranges (OCR, Codex, pi-review) get more candidate pairs than a tool that anchors single lines; QuickReview's posting shape scores a multi-line span at its single line.
4. **Nits are rewarded.** AACR's references are largely minor notes. In the tuning log, iterations 1–4 raised AACR F1 by about 20% while real defects found did not move: the extra matches were minor comments that happened to match the dataset.

The list is also less human than it is labelled: at least 80 of its 362 "human" references on our PRs are rewritten review-bot comments (Section 3.2). Removing them lowers AACR F1 by 0.1 to 1.4 points per reviewer and leaves the top three on recall and F1 as they were (Claude Code passes PR-Agent lower down; Table 12), so this is not a fifth cause of the disagreement; it bears on comparisons with the bots themselves.

In the pipeline check on three fresh10 PRs, each of the three reviewers then run (default, low, OCR) scored 1 of 7 references (14%) while the defect table separated them by 3–4 real defects.

What AACR's metric is still good for. It is the only number computed the benchmark's way, and it shows category coverage: the maintainability gain of the code-quality lenses is visible there (43.0 maintainability references matched by the default in the final comparison, against at most 12.7 for a third-party tool), with the defect table holding. A gain in AACR that costs defect recall would indicate a reviewer drifting towards the list and away from defects; we did not observe that. *The line-gate effect, measured* (on the earlier gpt-6-sol study's reviews). We re-scored the reviews of all seven reviewers with the harness's matching at a line gate of ±5 and at ±999, which in practice means any line of the same file: no PR in the set changes more than 1,000 lines. This was one judge round each, so it is compared with round 1 of the published ±1 evaluation. References found, published ±1 → ±5 → ±999: default 167 → 171 → 190; low 134 → 146 → 165; OCR 148 → 151 → 171; Codex CLI 63 → 68 → 76; OpenCode 81 → 84 → 91; pi-review 83 → 86 → 97; Claude Code 80 → 85 → 113. Recall at ±999 is 5.4–13.4% (from 4.4–11.8%) and F1 8.4–16.2% (from 7.0–14.1%). Loosening the gate to any line adds 12–41%

more matched references. The largest gain is Claude Code's (+41%, whose prose findings carry one line each), then low (+23%) and Codex CLI (+21%); default gains +14%. OCR leads F1 at every gate; at ±999 low overtakes default on F1 (14.7% against 14.6%) and Claude Code rises from sixth to fourth on recall and from fifth to fourth on F1. Even with the line gate gone, no reviewer finds more than 13.4% of the references, so the gate explains only a small part of the gap between AACR's metric and the defect table. The rest is the reference list itself (Section 3.2).[32]

11 External check: Martian's offline benchmark

AACR-Bench's references are one answer key; we ask whether what we found there holds on a second benchmark built by different people with a different key. Martian's offline code-review benchmark [55] reviews 50 PRs from five projects (Sentry, Grafana, Keycloak, Discourse, cal.com) against 173 golden comments, about 3.5 per PR, each with a category and a severity. Its leaderboard [56] scores the published comments of the commercial and open tools it lists, ranks the offline tab by F_2, and judges with Claude Opus 4.5 by default. We did not change its key, its pipeline or its scorer. The section ends with what we report from it and why (Section 11.9).

11.1 How we ran it

Pipeline. Martian's pipeline fetches each tool's comments from GitHub (step 1), has an LLM extract the distinct issues they raise, discarding the location (step 2), merges duplicates (step 2.5), asks an LLM judge whether each (golden, candidate) pair is the same issue (step 3), and scores precision, recall and F_β under three category profiles; the published default is *Core* (158 of the 173 goldens). We ran steps 2–3 and the scorer from its repository at commit `e616e84`, unmodified, in a container built from its own lock file. Step 1 is replaced by a converter that writes QuickReview's posted review in exactly the record step 1 builds from GitHub, including step 1's 30-item page limit; no review was posted anywhere. Each case is the PR as the benchmark's forks carry it, the code every published tool reviewed.

Reviewer and judge. QuickReview reviewed on gpt-6.1-sol, the model of every other run in this paper, in its `default` configuration. The judge is also gpt-6.1-sol: Martian publishes three judges (Opus 4.5, Sonnet 4.5, GPT-5.2) and none of them is the model our runs use, so the same judge re-scores the *published* comments of three published tools (cubic-v2 and qodo-extended-v2, first and second on the Core leaderboard, and Claude Code) on the same PRs, byte for byte from Martian's committed data; no third-party tool was run. On ten PRs the re-judge of identical comments moves by up to 4.3 F1 points from one pass to the next, sits 1.5–6.3 F1 points below the published Opus 4.5 scores of the same comments, and keeps their order. The other tools appear with Martian's published Opus 4.5 judgements, which we did not re-run.

Split. Before any tuning we split the 50 PRs into `tune20` (ten PRs picked by a size rule plus ten by a seeded hash) and `test30`, the other thirty, held out. Every analysis below that reads individual golden comments or candidates uses `tune20` only, except the golden audit, which reads all 173 and is kept out of every reviewer change.

Table 20: Martian's offline benchmark as scored: Core profile, judge gpt-6.1-sol, Martian's pipeline unmodified. QuickReview rows are our reviews; the three comparators are their published comments re-judged by the same judge in the same pass (a second re-judge of cubic-v2 on `tune20` gave 62.7 / 72.4 / 67.2 / 70.2; the pass that scored round 4 on `test30` gave cubic-v2 50.5 / 54.0 / 52.2 / 53.3, qodo-extended-v2 59.4 / 60.0 / 59.7 / 59.9 and Claude Code 37.0 / 37.0 / 37.0 / 37.0, so about three points of judge noise). Published Opus 4.5 scores on `tune20`: cubic-v2 65.6 / 71.2 / 68.3, qodo-extended-v2 62.1 / 61.0 / 61.5, Claude Code 42.0 / 49.2 / 45.3 (P / R / F1). Issues/PR: inline comments QuickReview posted (round 4: on `test30`); for the comparators, the issues Martian's step 2 extracted. Frozen policy and snappy: Section 11.5.

		`tune20` (58 Core goldens)				`test30` (100 Core goldens)			
Reviewer	Issues/PR	P	R	F1	F_2	P	R	F1	F_2
QuickReview default, as posted	9.0	19.7	58.6	29.4	42.0	18.2	54.0	27.2	38.8
QuickReview default, round 3 (five lens changes)	9.2	20.2	60.3	30.3	43.2	not run			
QuickReview default, round 4 (six more changes)	10.6	not run (targeted check only)				18.6	55.0	27.8	39.6
QuickReview frozen posting policy	2.0	49.0	43.1	45.9	44.2	36.5	19.0	25.0	21.0
QuickReview `snappy` profile, iteration 1	3.3	27.0	34.5	30.3	32.7	not run			
cubic-v2 (re-judged)	3.3	62.1	70.7	66.1	68.8	53.3	57.0	55.1	56.2
qodo-extended-v2 (re-judged)	3.3	50.0	58.6	54.0	56.7	60.6	60.0	60.3	60.1
Claude Code (re-judged)	3.8	35.1	46.6	40.0	43.7	39.2	38.0	38.6	38.2

11.2 The score as Martian computes it

Table 20 is the result as Martian computes it, in full. On the held-out PRs QuickReview has recall close to the leaders (54% against 57% and 60%) and by far the lowest precision (18%), so the lowest F1 and F_2 of the four. It posts about nine comments per PR where the golden lists hold about three and a half, and Martian counts every extracted issue that is not on the list against precision. This is the pattern of AACR-Bench's own metric (Section 10): a reference list that is short, partly not about defects, and silent about most of the real ones rewards posting little.

All 50 PRs. Combining the latest default on each half (round 3 on `tune20`, round 4 on `test30`) with the comparators judged in the same passes gives, on Core with our judge, QuickReview 19.2 / 57.0 / 28.8 / 40.9 (precision / recall / F1 / F_2; 90 matched, 378 unmatched candidates), cubic-v2 55.2 / 60.8 / 57.8 / 59.6, qodo-extended-v2 55.6 / 59.5 / 57.5 / 58.7 and Claude Code 36.2 / 40.5 / 38.2 / 39.6.

The frozen build. The frozen build of Section 8.7 reviewed all 50 PRs once, and one pass of the same judge scored it with the three comparators' published comments (Table 21). Its recall rises to 65.8 and its precision falls to 16.2: it posts more, and Martian's step 2 extracts 12.8 issues per PR from its reviews. Its F_2, 40.9, is level with Claude Code's and below cubic-v2's and qodo-extended-v2's; its F1 is the lowest of the four. On `test30` it scores 15.3 / 62.0 / 24.6 / 38.6, against cubic-v2 50.0 / 54.0 / 51.9 / 53.1 and qodo-extended-v2 59.8 / 61.0 / 60.4 / 60.8 in the same pass, and against 54.0 and 55.0 recall for the default as posted and round 4 (Table 20): recall up by seven to eight points, F_2 unchanged within judge noise. QuickReview has never been judged by Opus 4.5. On the same 50 PRs Opus credits 7–12 more matches per tool than our judge across our passes (in this one cubic-v2 95 → 104, qodo-extended-v2 95 → 102, Claude Code 65 → 76); the same shift puts the frozen build at roughly 17–18 precision

[32]Computed with `bench.sh eval --line-k 5|999 --rounds 1` (judge gpt-6-sol:low through the shim, 3 in flight) on copies of each run's `results/all`, so the official ±1 files were never touched. 3,227 and 10,983 judge calls by the count taken at run time (2 failed and were scored as non-matches; about $18 at list price); the shim logs that survive record 3,368 of them, the rest having been overwritten. Scores from `wp-data/linek_scores.json`, 2026-09-30.

Table 21: Martian, all 50 PRs, Core profile, the frozen build and the three comparators' published comments in one pass of the gpt-6.1-sol judge (run `final1-martian50`, 2026-10-04); the middle block is the three third-party harnesses of the eight-reviewer comparison (Table 4), run here in that configuration on gpt-6.1-sol (runs `final1-{ocr,pragent,opencode}-martian50`), each judged alone by the same judge and steps (Martian dedups and matches per PR and tool, so the rows compare); the last three rows are Martian's published Opus 4.5 judgements of the published comments. Issues/PR: what Martian's step 2 extracted. Cost at list prices and mean wall-clock minutes per PR, for the runs we made.

Tool	Issues/PR	Matched / 158	P	R	F1	F_2	$/PR	min/PR
QuickReview frozen build	12.8	104	16.2	**65.8**	26.1	40.9	2.84	26.1
qodo-extended-v2 (re-judged)	3.4	95	56.2	60.1	**58.1**	**59.3**	—	—
cubic-v2 (re-judged)	3.5	95	54.3	60.1	57.1	58.9	—	—
Claude Code (re-judged)	3.6	65	36.5	41.1	38.7	40.1	—	—
open-code-review 1.3.1, effort high	7.0	93	26.7	58.9	36.8	47.4	0.86	3.3
PR-Agent 0.46.0, medium	2.7	57	41.9	36.1	38.8	37.1	0.09	1.4
OpenCode 1.18.33, medium + PR text	3.2	53	32.7	33.5	33.1	33.4	0.19	2.4
qodo-extended-v2 (Opus 4.5)		102	67.1	64.6	65.8	65.1		
cubic-v2 (Opus 4.5)		104	61.5	65.8	63.6	64.9		
Claude Code (Opus 4.5)		76	46.3	48.1	47.2	47.7		

and 70–73 recall, $F_2 \approx 43.5$–45.6, fourteenth to sixteenth of 23 on the published leaderboard (Figure 11): recall at the top, precision at the bottom.

The same third-party harnesses. The published comparators ran on models of their vendors' choosing. To compare harnesses on one model we also ran the three third-party harnesses of the eight-reviewer comparison, open-code-review, PR-Agent and OpenCode, in exactly their configurations there (Table 4: gpt-6.1-sol, one review per PR), on all 50 PRs, and scored each with the same judge and steps; the same three ran on Qodo's benchmark (Section 12). On recall QuickReview is first and open-code-review second, as on AACR: 65.8, 58.9, then PR-Agent 36.1 and OpenCode 33.5 (Table 21); on the 86 golden comments the audit judged real (Section 11.3), 87%, 78%, 54% and 56%. Precision and F1 order them the other way: PR-Agent has the highest F1 of the four (38.8) and QuickReview the lowest (26.1), and open-code-review, which posts about half as many issues as QuickReview at nearly its recall, has the highest F_2 of the four (47.4 against 40.9). The cost of the recall is as on AACR, only larger: $2.84 and 26.1 minutes per PR for QuickReview, against $0.09–0.86 and 1.4–3.3 minutes. The PRs are larger than AACR's, so QuickReview's cost per PR is above its $2.06 there (Section 8.7).

11.3 What the golden comments are

We audited all 173 golden comments the way the defect key was built (Section 5): one investigator per comment (Claude Opus 5.5 agents, reading the code at the reviewed head, its callers, the framework semantics at the pinned versions and the tests, never any reviewer's output), told to make the strongest case that the comment names a real defect the PR introduces; every verdict other than "real" went to a second, independent investigator told to try again. A comment is *real* if it names a real defect, possibly partly; *not real* if it is trivial, latent, pre-existing, intended or not a defect; and *disputed* when the two investigators disagree. The result is **86 real, 77 not real and 10 disputed**: 45% of the golden comments do not name a real defect introduced by the PR. As with the defect key, no human has checked these verdicts.

Restricted to the real goldens (Table 22), the order changes. Under one judge the frozen build finds the most real goldens, 75 of 86 (87.2%), ahead of qodo-extended-v2 (68), cubic-v2 (58) and Claude Code (45); a quarter of its credits come from goldens that are not real, as for qodo-extended-v2, against over a third for cubic-v2 and Claude Code. By half: 32 of 34

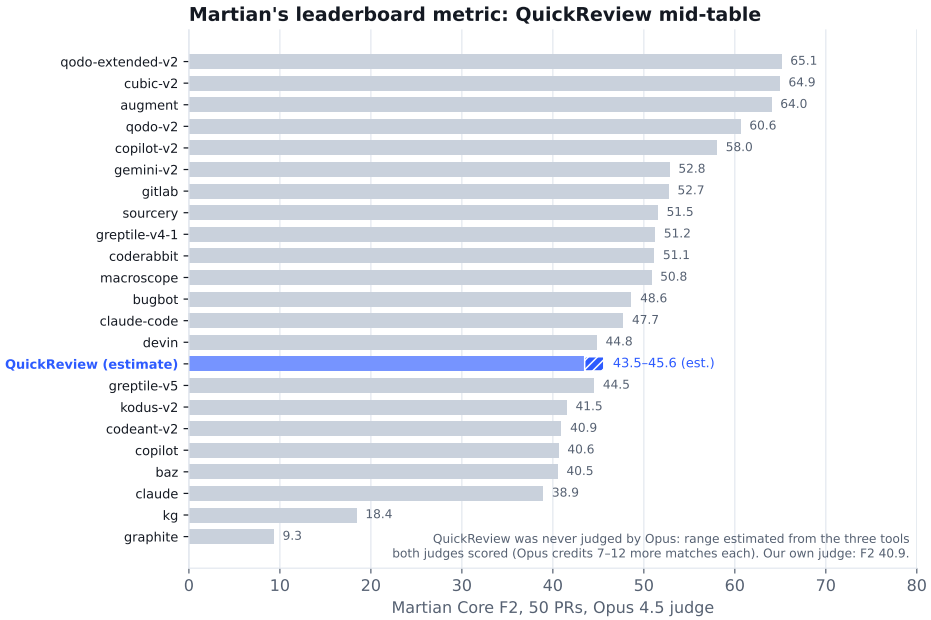


Figure 11: Martian's leaderboard metric (F_2, Core, 50 PRs, published Opus 4.5 judgements). The QuickReview bar is the frozen build's estimate, a range scaled from the three tools both judges scored; under our own judge it is 40.9.

Table 22: Recall of the 86 golden comments the audit judged real, and the share of each tool's Martian credits that went to the 77 judged not real ("Not real"). Left: one judge (gpt-6.1-sol), all 50 PRs: the frozen build and the comparators' published comments in one pass, and below the rule the three harnesses run on gpt-6.1-sol, each judged alone (Table 21). Right: Martian's published Opus 4.5 judgements, all 50 PRs (QuickReview was not judged by Opus 4.5).

Same judge, gpt-6.1-sol			Published judge, Opus 4.5		
Tool	Real recall	Not real	Tool	Real recall	Not real
QuickReview frozen	**87.2%**	25%	qodo-extended-v2	83.7%	28%
qodo-extended-v2	79.1%	26%	copilot-v2	82.6%	33%
cubic-v2	67.4%	37%	augment	79.1%	30%
Claude Code	52.3%	34%	qodo-v2	75.6%	28%
open-code-review	77.9%	26%	cubic-v2	74.4%	34%
OpenCode	55.8%	**11%**	coderabbit	68.6%	35%
PR-Agent	53.5%	23%	bugbot	64.0%	22%
			sourcery	64.0%	36%
			gitlab	62.8%	35%
			greptile-v4-1	61.6%	34%
			gemini-v2	61.6%	36%
			Claude Code	59.3%	35%
			macroscope	59.3%	30%
			devin	54.7%	24%

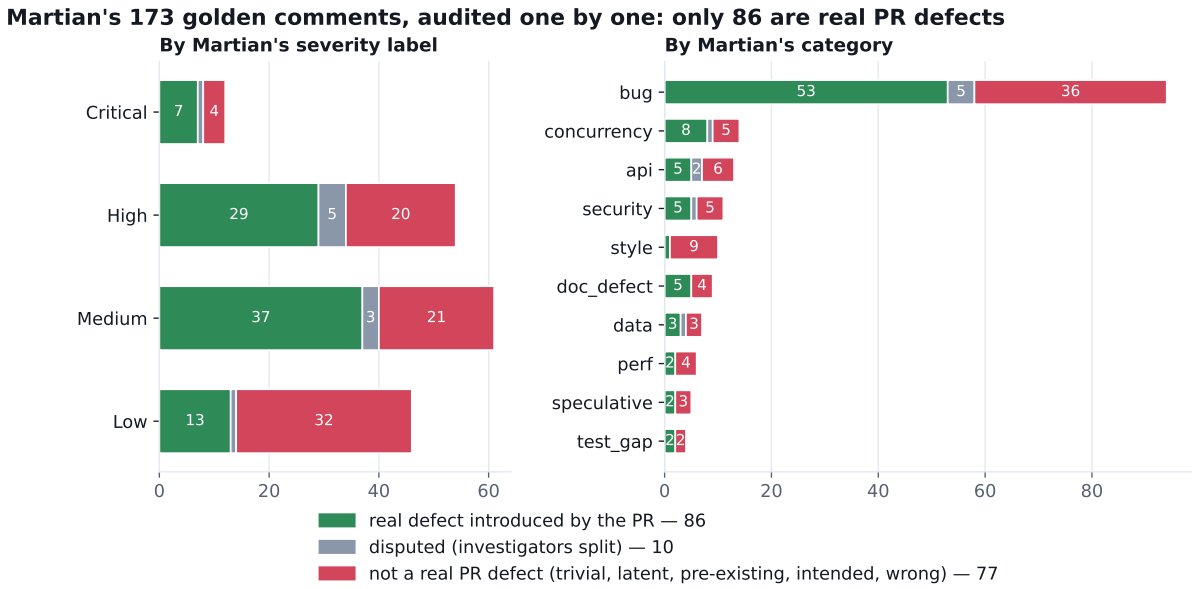


Figure 12: The 173 golden comments by the audit's verdict, against Martian's own severity and category labels. 24 of the 66 Critical or High goldens are not real PR defects and 6 are disputed.

real goldens on `tune20` (cubic-v2 26, qodo-extended-v2 25) and 43 of 52 on `test30`, level with qodo-extended-v2 and ahead of cubic-v2 (32) and Claude Code (26). The default as posted, before those changes, found 69 of 86 (40 of 52 on `test30`). Of the three harnesses run on gpt-6.1-sol, open-code-review finds 67 (77.9%), OpenCode 48 and PR-Agent 46; OpenCode earns only 11% of its credits on goldens that are not real, the lowest share of any tool. The published judgements tell the same story for the rest of the field: every tool earns 22–36% of its credits on goldens the audit does not count as real.

11.4 The real defects the golden lists lack

A short list also misses defects. On `tune20` we measured every tool against one key the way the defect key was built: all 580 candidates that Martian's step 2 extracted from eight tools (QuickReview, three re-judged tools and four with published Opus 4.5 judgements: augment, qodo-v2, macroscope, devin) were shuffled within each PR, stripped of the tool's name and classified against the code by ten annotator sessions (Claude Opus 5.5) as *golden* (states a golden comment), *real* (a defect the PR introduces that no golden states, with a site and a code-grounded reason), *minor* (true but not a PR-introduced defect: style, docs, missing tests, questions, pre-existing gaps) or *wrong*. The real bar is the one of Section 5 and is the same for every candidate. The extended key is the 38 goldens an earlier review of `tune20` rated valid plus the 44 real defects that at least one tool raised and no golden states: 82 entries. *Strict* precision counts minor candidates as noise; *lenient* precision leaves them out of the denominator, so only duplicates and wrong claims count against a tool.

On the extended key (Table 23) QuickReview finds 71 of 82 entries (87%); the next tool finds 43 (52%). Its strict F1 ties the best (54.7 against qodo-v2's 55.4) and its lenient F1 leads by 14 points. Its noise is of one kind: 47% of its candidates are minor and 2% wrong, where cubic-v2's are 29% minor and 14% wrong. Cubic-v2, first on Martian's leaderboard, raises 4 of the 44 unlisted defects and falls to the middle of the field.

The tie at the top of strict F1 depends on the 18 unlisted defects only QuickReview raised (12 low, 6 medium). They pass the same bar as everyone's, but they are also the place where a key built partly from our own reviewer could flatter it, as in Section 8.6. Table 24 removes them three ways: QuickReview's recall stays the highest in every variant (83–88%), and its strict F1

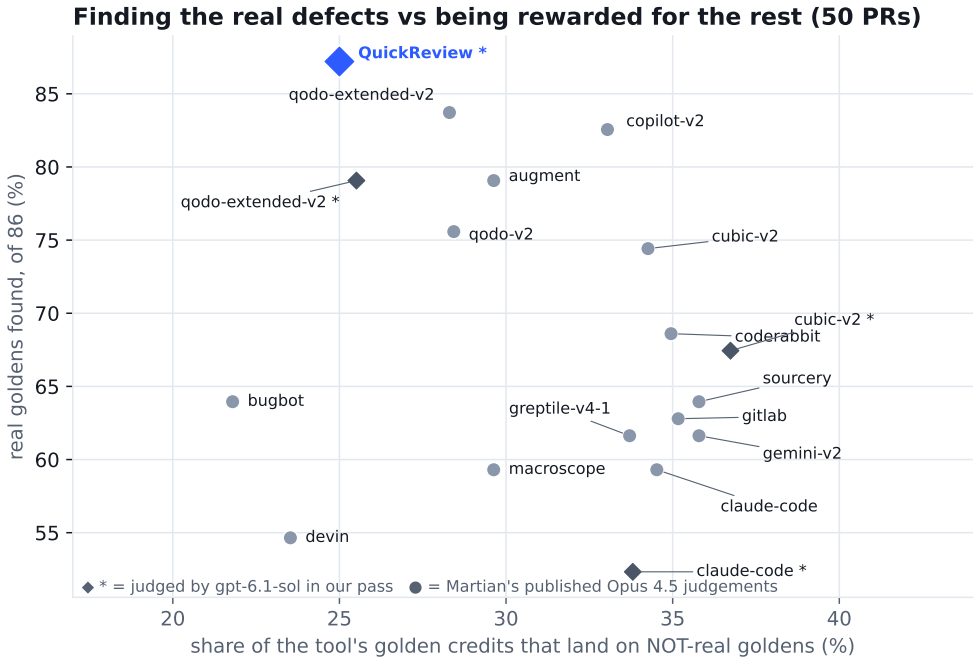


Figure 13: Real-golden recall against the share of each tool's golden credits that fall on goldens the audit judged not real (50 PRs). Diamonds: judged by gpt-6.1-sol in our pass (QuickReview = the frozen build); circles: Martian's published Opus 4.5 judgements.

Table 23: `tune20` against the extended key (82 entries: 38 valid goldens + 44 real unlisted defects), candidates classified blind to the tool. "Found" counts key entries stated; the last four columns split each tool's candidates, duplicates within a tool counted after labelling.

Tool	Cand./PR	Found	Golden / real	Recall	P strict	P lenient	F1 strict	F1 lenient	Minor	Dup	Wrong
QuickReview default	8.5	**71**	32 / **39**	**86.6**	40.0	75.6	54.7	**80.7**	47%	11%	2%
qodo-v2	3.1	43	32 / 11	52.4	58.7	**92.5**	**55.4**	66.9	37%	0%	5%
augment	3.4	40	32 / 8	48.8	54.4	86.0	51.4	62.3	37%	4%	4%
qodo-extended-v2	3.3	38	28 / 10	46.3	56.1	82.2	50.7	59.3	32%	5%	8%
cubic-v2	3.3	35	31 / 4	42.7	50.0	70.2	46.1	53.1	29%	8%	14%
Claude Code	3.8	32	21 / 11	39.0	42.7	71.1	40.8	50.4	40%	11%	7%
macroscope	1.9	26	22 / 4	31.7	63.2	82.8	42.2	45.8	24%	3%	11%
devin	1.7	22	17 / 5	26.8	**64.7**	78.6	37.9	40.0	18%	15%	3%

Table 24: Sensitivity of Table 23 to which unlisted defects count: recall / strict F1 per variant. Variants keep the unlisted defects rated medium or high, raised by two or more tools, or raised by at least one tool other than QuickReview; the last keeps none. Precision counts a candidate as good only if its entry is in that variant's key. CC: Claude Code.

Key variant	n	QuickReview	cubic	qodo-ext.	CC	augment	qodo-v2	macro.	devin
Full key	82	86.6 / 54.7	42.7 / 46.1	46.3 / 50.7	39.0 / 40.8	48.8 / 51.4	52.4 / 55.4	31.7 / 42.2	26.8 / 37.9
Unlisted: medium/high	59	88.1 / 43.4	54.2 / 49.5	55.9 / 51.9	49.2 / 43.3	61.0 / 54.1	62.7 / 56.1	40.7 / 47.8	33.9 / 43.0
Unlisted: ≥ 2 tools	61	86.9 / 43.9	55.7 / 51.9	60.7 / 57.4	50.8 / 45.6	65.6 / 59.5	70.5 / 64.1	42.6 / 50.9	36.1 / 46.3
Unlisted: not QuickReview alone	64	82.8 / 43.4	54.7 / 52.2	59.4 / 57.7	50.0 / 46.0	62.5 / 58.2	67.2 / 62.7	40.6 / 49.4	34.4 / 44.9
Valid goldens only	38	84.2 / 29.2	81.6 / 57.1	73.7 / 52.6	55.3 / 37.2	84.2 / 56.6	84.2 / 59.5	57.9 / 55.1	44.7 / 47.2

Table 25: QuickReview default after the round-4 lens and validator changes, reviewed once on the 30 held-out PRs (image built from commit `21d592b`), scored as in Table 20 and against the audited real goldens as in Table 22 (52 real on `test30`). The comparator rows are their published comments re-judged in the same pass. "Not real" is the share of a tool's golden credits that went to goldens the audit judged not real. One run each.

Reviewer (`test30`)	P	R	F1	F_2	Real found	Not real
QuickReview default, as posted (round 1)	18.2	54.0	27.2	38.8	40 / 52	22%
QuickReview default, round 4	18.6	55.0	27.8	39.6	42 / 52	22%
qodo-extended-v2 (re-judged)	59.4	60.0	59.7	59.9	43 / 52	25%
cubic-v2 (re-judged)	50.5	54.0	52.2	53.3	33 / 52	35%
Claude Code (re-judged)	37.0	37.0	37.0	37.0	25 / 52	37%

falls to about 43–44, behind qodo-v2, augment, qodo-extended-v2 and cubic-v2. On the valid goldens alone, QuickReview, augment and qodo-v2 tie on recall (84%) and QuickReview has the lowest precision. The classification is one model's, blind to the tool but not checked by a human, and it covers 20 PRs.

What QuickReview missed. Of the 38 valid `tune20` goldens, QuickReview posted 33. Four were never raised by any lens: a missing re-check after taking a write lock (Grafana), a metric tag spelled `shard` on one path and `shards` on another (Sentry), a generic error that surfaces as an HTTP 500 (cal.com), and a removed float that breaks the server-rendered header (Discourse). One was raised and then dropped by the validator as unverifiable: a resize argument that the external image tool does not accept (Discourse). The round-3 configuration in Table 20 adds five changes aimed at these classes: the concurrency lens planned when the diff touches synchronisation or the PR text names a race; a consistency check on new identifiers (metric names, tag keys, log keys); unverifiable claims about an external tool kept as questions rather than dropped; a violated declared contract held at medium severity; and a test that cannot observe what it claims treated as a defect. On `tune20` the net effect was one more golden, within run-to-run noise.

Round 4 adds six more changes, each aimed at a class still missed after round 3 rather than at a particular golden: a lazily filled cache that is not re-checked after the exclusive lock is taken is judged as a duplicated initialisation; every branch of a `DELETE` or `UPDATE` filter must carry the operation's scoping conditions; a direct write that skips the model lifecycle is checked against what the lifecycle maintains; a removed style rule, selector or key is checked against every consumer, server-rendered templates included; a renumbered or reused exit code, status code or wire enum value is a contract break; and a source the validator cannot open lowers its confidence, never its reachability. They were checked only by re-reviewing the nine `tune20` PRs they target, with no judge (six of nine improved); the full `tune20` set was not re-run. Round 4 was then reviewed once on the 30 held-out PRs (Table 25).

On Martian's own score round 4 is flat: precision 18.6 against 18.2, recall 55.0 against 54.0, inside the roughly three points by which two judgings of the same comparator differ. Its volume is unchanged (10.6 inline comments per PR against 10.8; 299 extracted issues against 301), so precision remains the whole of the gap. On the audited real goldens it moved from 40 to 42 of 52, one behind qodo-extended-v2 (43) and ahead of cubic-v2 (33) and Claude Code (25); the previous judging of those two gave 34 and 26. This is one run on each side, and the two-golden difference is within what we have seen between repeated runs, so we claim no improvement from it.

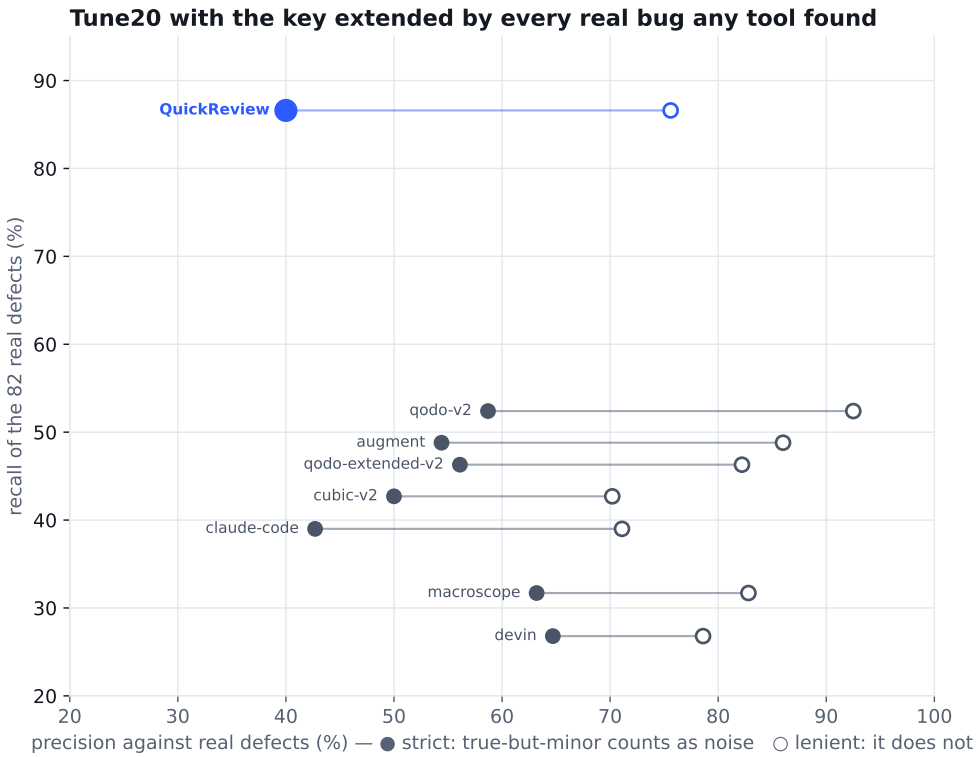


Figure 14: `tune20` against the extended key of 82 real defects: recall against strict precision (filled; a true-but-minor comment counts as noise) and lenient precision (open; it does not).

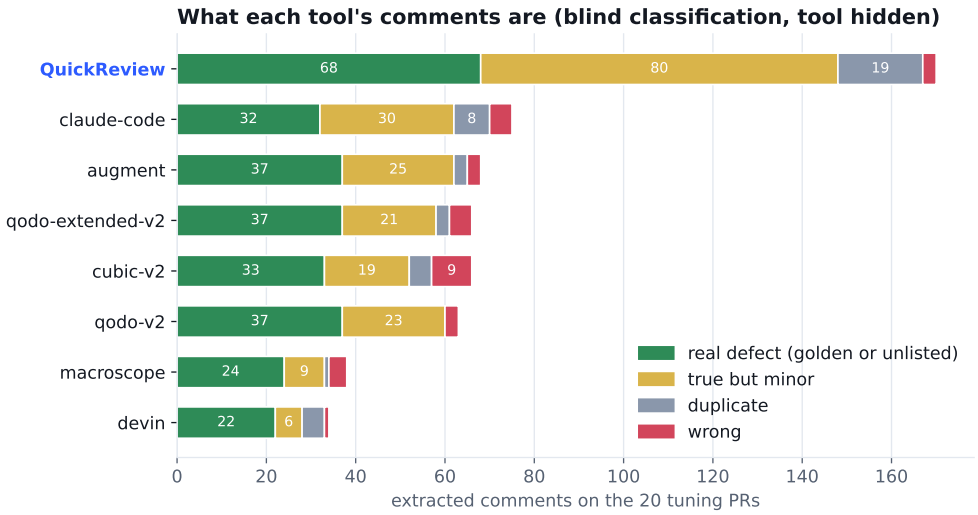


Figure 15: What each tool's `tune20` comments are, classified blind against the code with the tool hidden: a real defect (a golden or an unlisted bug), true but minor, a duplicate of the tool's own comment, or wrong.

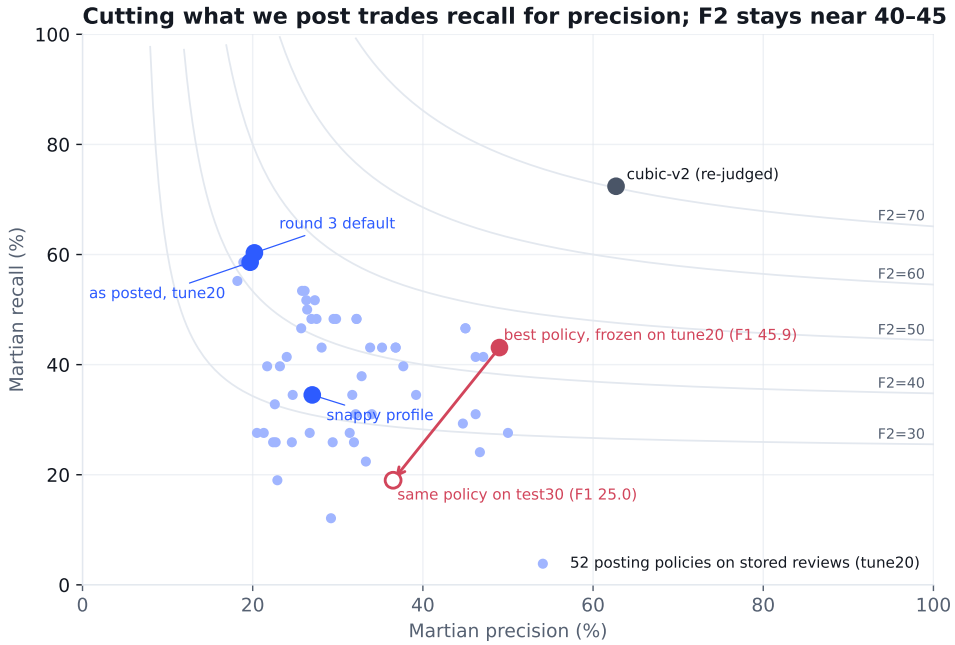


Figure 16: Martian precision and recall on `tune20` for 52 posting policies applied to stored QuickReview reviews, with F_2 iso-lines. The policies slide along the frontier; the best one by F1, frozen, fell from F1 45.9 on `tune20` to 25.0 on `test30` (arrow).

11.5 What posting less does to the Martian score

Cutting volume does not close the gap. Before the audit we tried to raise QuickReview's Martian score by changing only what it posts. Over about 50 posting policies applied to the stored `tune20` reviews (no nits, each finding once, a verdict-only review body, per-PR caps, severity floors, agreement between lenses, validator confidence), F_2 stayed between 42 and 44 at best and fell with every cap, because the cap removed golden matches along with the noise. The best policy by F1, frozen before the held-out PRs were reviewed, went from F1 45.9 on `tune20` to 25.0 on `test30` (Table 20): its confidence threshold did not transfer. A `snappy` profile that has the validator choose at most four comments per PR reached 3.3 comments per PR and the same F1 as posting everything, with F_2 down to 32.7. Even a perfect cut, raising precision to 60% with no loss of recall, would reach $F_2 \approx 59$ on `tune20`, below cubic-v2's 69–70.

The rest of the gap is the list. On `tune20` cubic-v2 is credited with 41 goldens and QuickReview with 34. On the 38 valid goldens QuickReview is ahead (33 against 31); cubic-v2's lead comes from 10 of the 21 trivial, doubtful or wrong goldens, where QuickReview has 1. These are one-line remarks (a vendor-prefixed CSS property that does not exist, an atomic increment in a sequential job, a claim that a call exits the process). Our reviewer's severity bar drops such remarks by design.

11.6 Where QuickReview is strong: its own severity

Splitting QuickReview's comments by its own severity shows where the benchmark cannot see it (Figure 17, Table 26). In the blind `tune20` classification, 84% of its P0–P1 comments name a real defect, a golden or an unlisted bug; that falls to 41–44% at P2 and P3 and to 15% for nits, which are mostly true but minor. Recounting the existing Martian judgements on all 50 PRs for subsets of what QuickReview posted, P0–P1 alone (2.4 comments per PR) scores 26.4 precision: the comments that are 84% real defects are three-quarters "false positives" to the list, because

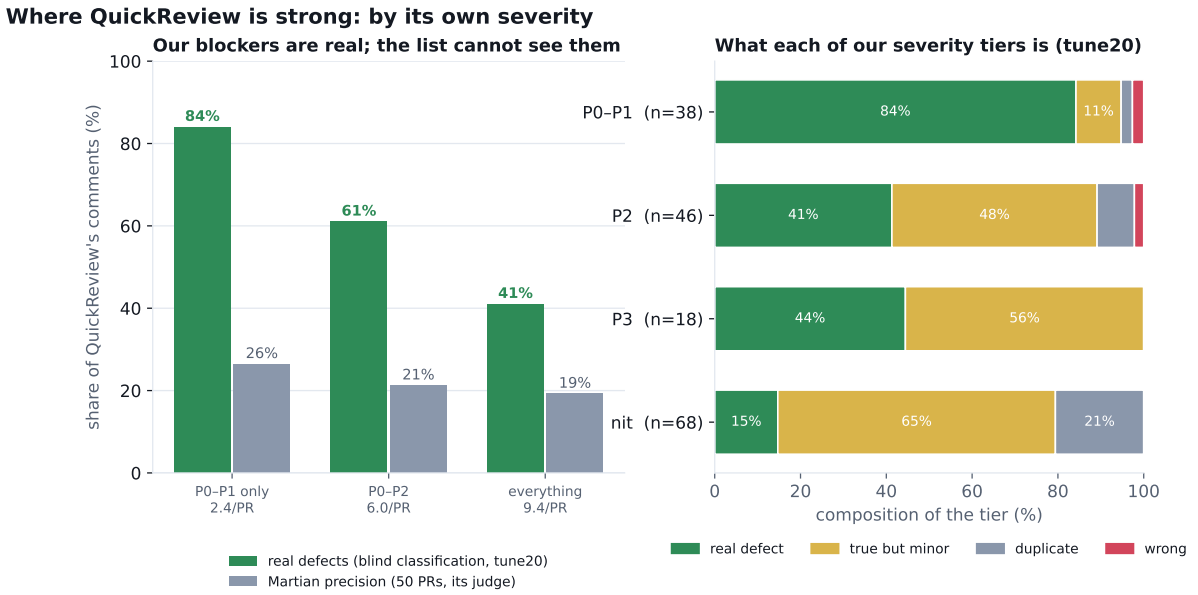


Figure 17: QuickReview by its own severity. Left: share of comments that are real defects (blind `tune20` classification) against Martian's precision for the same subset (50 PRs, existing judgements recounted). Right: what each tier is.

Table 26: QuickReview by its own severity. Left block: blind `tune20` classification of 170 candidates (GOLDEN matches include the few trivial or wrong goldens, so counts are ±1 against Table 23). Right block: Martian Core on all 50 PRs when only that subset is posted, recounted from the existing judgements.

Tier	n	real	minor	dup	wrong	Posted	per PR	P	R	F_2
P0–P1	38	32 (84%)	4	1	1	P0–P1 only	2.4	26.4	20.3	21.2
P2	46	19 (41%)	22	4	1	blockers only	4.1	22.0	28.5	26.9
P3	18	8 (44%)	10	0	0	P0–P2	6.0	21.2	40.5	34.3
nit	68	10 (15%)	44	14	0	everything	9.4	19.2	57.0	40.9

the list lacks most of the bugs they name. Cutting to them also drops Martian recall from 57.0 to 20.3 and F_2 from 40.9 to 21.2, since many listed goldens are the minor remarks QuickReview posts at lower severity.

11.7 Did a tool find every listed blocking problem in a PR?

Martian's metric does not ask whether a tool caught the problems that would block a merge. We count the PRs in which a tool was credited with *all* of the PR's Critical and High goldens (Figure 18). On Martian's own labels (34 PRs have one) the frozen build does so in 19 (56%), level with qodo-extended-v2 (19), behind cubic-v2 (21) in the same pass, and ahead of Claude Code (9). Many of those labels sit on goldens the audit found not real; on the 19 PRs whose Critical or High goldens include a real one, counting only the real ones, the frozen build found all of them in 15 (79%), against qodo-extended-v2's 14, cubic-v2's 12 and Claude Code's 6. Martian's published Opus judgements, which credit more matches, give 13–15 of 19 for the best published tools. On real goldens of any severity the frozen build found all of a PR's in 26 of 35 PRs, qodo-extended-v2 in 21, cubic-v2 in 20 and Claude Code in 10. (The default as posted, before the frozen build's changes: 17 of 34, 14 of 19 and 21 of 35.) Of the three harnesses run on gpt-6.1-sol (not in the figure), open-code-review does better than the frozen build on the real Critical and High goldens, 16 of 19 against 15, and is level on all labelled ones (19 of 34), with

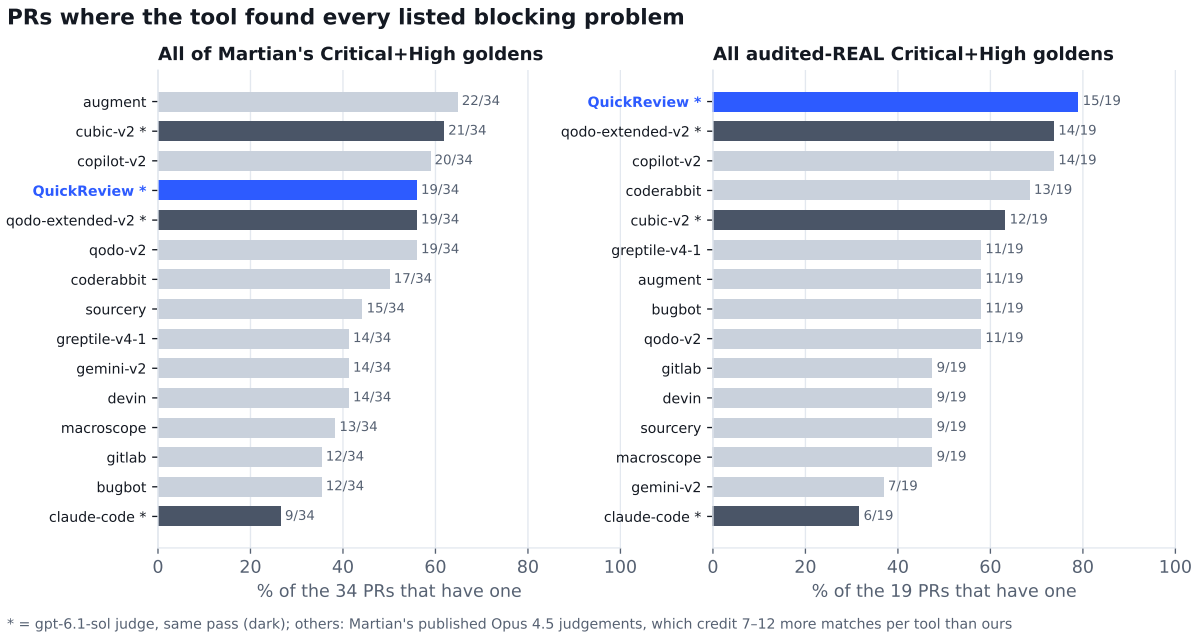


Figure 18: PRs in which a tool was credited with every one of the PR's Critical and High goldens: all of them as Martian labels them (left, 34 PRs) and only those the audit judged real (right, 19 PRs). Starred bars: gpt-6.1-sol judge in our pass; the rest: Martian's published Opus 4.5 judgements.

20 of 35 on real goldens of any severity; OpenCode has 12, 11 and 13 and PR-Agent 9, 7 and 11.

11.8 What the benchmark measures, and what it misses

Taken together, the results above point to eight properties of the benchmark that a reader of its leaderboard should weigh.

1. *The key is half noise.* 77 of 173 golden comments are not real PR defects and 10 are disputed (Section 11.3); tools earn 22–37% of their credit on them (OpenCode on gpt-6.1-sol 11%), QuickReview 25%.
2. *A real bug off the list is a false positive.* On `tune20` eight tools found 44 real PR-introduced bugs that the list lacks; every one counts against the tool that found it.
3. *Minor and wrong are scored alike.* Precision cannot tell a noisy reviewer from a wrong one: 47% of QuickReview's `tune20` comments are true but minor and 2% wrong; cubic-v2's are 29% and 14%.
4. *Volume and phrasing move the score.* Extraction splits long comments, counts each repeated site and re-extracts a review body that restates its findings.
5. *Severity is labelled, inflated and unused.* 24 of the 66 Critical or High goldens are not real, and missing a Critical golden costs the same as missing a Low one.
6. *The score depends on the judge.* Opus 4.5 credits 7–12 more matches per tool than gpt-6.1-sol on the same comments, and one judge moves by about three points between passes.
7. *The sample is small.* 50 PRs from five projects; a posting policy tuned on 20 of them fell from F1 45.9 to 25.0 on the other 30.

8. *The leaderboard rewards matching the list's style*: a handful of short, line-local remarks per PR, the trivial ones included. That is a reasonable product shape, and a different objective from finding the defects a PR introduces.

11.9 What we report

We care whether a review finds the real defects a PR introduces, above all the high- and medium-severity ones, and how many of its claims are wrong. A true but minor remark costs a reader's attention; a wrong one costs more. Martian's score measures agreement with a list of about three and a half comments per PR, 45% of which do not name a real defect, counts real defects missing from the list and minor remarks the same as wrong ones, and ignores severity. We therefore report three numbers side by side, none of them alone: Martian's score as computed, unmodified and in full (Table 20); recall of the audited real goldens (Table 22); and the extended-key results with strict and lenient precision (Tables 23 and 24). On all three the reading is the one AACR-Bench gave: QuickReview finds the most real defects, and its cost is volume, nearly all of it true but minor rather than wrong. QuickReview's default is to find as many real problems as possible and to post each with a severity; a team that wants fewer comments sets a posting budget in its profile. That budget is a product choice about reader attention, not a measure of review quality. Alongside the three numbers we report the two severity views of Sections 11.6 and 11.7.

Data. The case sets, the converter and the scoring wrapper ship in `bench/aacr/ext/`, with the full golden audit's per-comment verdicts in `ext/martian-tuning/golden-audit-full.json`; the figures of this section are drawn by `bench/aacr/martian_figs.py` from `charts/martian-data.json`. The analysis notes behind this section (method and deviations, the posting-policy leaderboard, the golden mining, the fair comparison, the run log and the consolidated critique) are working notes of the study, not in the public repository. Review and judge run directories are in the run archive, as for the main benchmark (Section 14).

12 External check: Qodo's PR-Review-Bench

Qodo's benchmark [27] (Hugging Face `Qodo/PR-Review-Bench` at `a73957c`, MIT) is 100 PRs from 8 repositories into which an LLM injected 580 issues: 309 functional bugs and 271 violations of repository rules, which Qodo extracted with an LLM and committed to every fork as an `AGENTS.md` file. Every vendor's raw review is public on GitHub forks; Qodo publishes no judge, prompt or scorer, so its published F1 cannot be reproduced. We therefore judged every tool, QuickReview included, with one judge of ours under the matching rule Qodo's description states.

How we ran it. Each case is the fork's PR rebuilt as two synthetic commits, the merge-base tree and the head tree with neutral messages, because the fork's branch names and history reveal the injection; the `AGENTS.md` stays in the tree, as every vendor reviewed with it, and the PR text is the upstream PR's own title and description. QuickReview ran in its frozen build (Section 8.7), `default`, on gpt-6.1-sol, offline; nothing was posted. The nine vendors' inline review comments were read with anonymous public API calls; they are the vendors' January 2026 runs at default settings. The judge (gpt-6.1-sol at low reasoning) receives, per PR and tool, the PR's issues and the tool's inline comments and returns which comment states which issue, one to one; code then checks location. The primary rule counts a match when the comment is on the issue's file and its lines overlap the issue's within five lines; "same file" and "semantic only" are sensitivities. Only inline comments count, as in Qodo's rule; for QuickReview, its inline comments and the review-body findings that carry a path and line. Under "semantic only" our

Table 27: Qodo PR-Review-Bench, 100 PRs, 580 injected issues (309 functional, 271 rule violations), one gpt-6.1-sol judge for every tool, primary location rule (same file, lines within ±5); one run per tool, one judgement. The nine vendors are their published January 2026 reviews (`ext-r6-qodo-pr100`); QuickReview rows are runs `final1-qodo-pr100` (frozen build) and `ext-r6-qodo-pr100` (2026-10-02 build); the three marked † are the third-party harnesses of the eight-reviewer comparison in that configuration on gpt-6.1-sol (Table 4; runs `final1-{opencode,ocr,pragent}-qodo-pr100`); all scored by `bench/aacr/ext/qodo_judge.py`. Qodo's F1: Qodo's own published F1, judge unpublished. Cost at list prices and mean wall-clock minutes per PR, for the runs we made.

Tool	Comments/PR	P	R	F1	R functional	R rule	Qodo's F1	$/PR	min/PR
qodo-exhaustive	4.6	58.0	**46.4**	**51.5**	48.2	**44.3**	60.1	—	—
qodo-precise	3.1	68.0	35.9	47.0	36.2	35.4	55.4	—	—
augment	2.5	67.5	29.7	41.2	49.2	7.4	44.1	—	—
QuickReview frozen build	7.1	37.3	45.3	40.9	**52.1**	37.6	—	2.48	19.6
OpenCode 1.18.33†	2.6	64.9	29.0	40.0	43.7	12.2	—	0.20	2.3
copilot	4.2	46.8	33.6	39.1	46.3	19.2	42.8	—	—
greptile	2.4	64.7	26.6	37.7	37.2	14.4	39.0	—	—
cursor	1.9	73.8	24.3	36.6	41.1	5.2	39.3	—	—
open-code-review 1.3.1†	3.5	47.1	28.1	35.2	45.6	8.1	—	0.46	1.9
QuickReview, 2026-10-02 build	5.5	35.8	34.0	34.9	50.8	14.8	—	—	—
codex	1.6	**78.2**	22.2	34.6	32.4	10.7	37.6	—	—
PR-Agent 0.46.0†	3.4	47.2	27.2	34.5	43.7	8.5	—	0.10	1.2
sentry	0.9	76.8	12.6	21.6	22.7	1.1	23.7	—	—
coderabbit	1.9	43.7	14.3	21.6	23.0	4.4	28.0	—	—

judge comes within 0.1–1.3 F1 points of Qodo's published figure for six of the nine vendors and keeps their order, the only calibration available.

Results. The frozen build finds the most functional bugs of the thirteen tools (52.1%, against augment's 49.2% and qodo-exhaustive's 48.2%) and the second-most issues overall (45.3% against qodo-exhaustive's 46.4%); it posts the most comments (7.1 per PR) at the lowest precision (37.3%), and its F1, 40.9, is fourth, behind Qodo's two configurations and augment (Table 27). The change from the 2026-10-02 build is almost all in rule violations, 14.8% to 37.6%, second to qodo-exhaustive's 44.3%: the frozen build checks every rule the repository writes down and plans a lint-conformance lens when CI runs a linter, where the earlier build treated most such items as nits it did not post. The sensitivities do not change the order at the top: under "semantic only" the frozen build scores F1 46.7 (P 42.5, R 51.7, functional 61.2), against qodo-exhaustive's 56.5 and qodo-precise's 51.2.

The same third-party harnesses. The vendors' reviews ran on models of their choosing. The three third-party harnesses of the eight-reviewer comparison ran here in exactly that configuration, on gpt-6.1-sol, which makes their rows a comparison of harnesses on one model (Table 27, †; the same three ran on Martian, Section 11). QuickReview finds more than any of them: 45.3% of the issues against 27.2–29.0%, 52.1% of the functional bugs against 43.7–45.6%, and 37.6% of the rule violations against 8.1–12.2%. They post a third to a half as many comments at higher precision, and OpenCode's F1, 40.0 (P 64.9), is within a point of QuickReview's 40.9; open-code-review's is 35.2 and PR-Agent's 34.5. PR-Agent is Qodo's own open-source reviewer; `qodo-exhaustive` and `qodo-precise` are its commercial product, so this is PR-Agent's first row on the benchmark. Each costs $0.10–0.46 and takes 1.2–2.3 minutes per PR, against QuickReview's $2.48 and 19.6; these PRs are larger than AACR's, so QuickReview's cost per PR is above its $2.06 there.

Caveats. Qodo built, injected and validated this benchmark, and 47% of its issues are rule compliance, which its own product's rules feature targets; the bugs are LLM-injected into real

PRs, not organic. The vendor reviews are January 2026 versions; QuickReview is a build of October 2026. The judge is ours, not Qodo's. A real defect that is not in the issue list counts as a false positive, as on Martian: in a hand check of the 40 unmatched comments of the 2026-10-02 build on ten PRs (one model, the tool hidden but recognisable by style), 12 were real defects the list lacks, 19 true but minor, 9 restated a listed issue or duplicated a credited comment, and none was wrong. On the 31 PRs that none of the frozen build's development used (Section 13), its recall rose from 52.5% to 60.3% over the 2026-10-02 build (+7.8 points, 95% CI +2.1 to +14.1, paired bootstrap over PRs) and its F1 from 49.7 to 53.0, against qodo-exhaustive's 51.3 on the same PRs.

13 Threats to validity

This section summarises `docs/THREATS-TO-VALIDITY.md`, ordered by severity, and extends it to the final comparison. In short: the order at the top of the final comparison (QuickReview's default first on overall recall) held under every check we could run; the compared default's lead on high-severity defects is not established over the panel or Codex CLI (the frozen build's is); several threats push the measured lead in QuickReview's favour, while one (the pool, Threat 2) pushes against every final run. The authors build QuickReview; vendor-run evaluations are known to favour their sponsor even when standard risk-of-bias checks pass [107, 28], and interested parties bias even third-party leaderboards [108], so none of the mitigations below substitutes for an independent replication.

Development sets (disclosure). QuickReview's September levers were designed from its misses on T40, including holdout10 and fresh10, each also used for later decisions (Appendix D); R145 was selected mechanically after the last of those changes and never opened for them, and the two halves are reported apart (Section 8.6). The frozen build's changes were designed from the compared runs' misses on R145, on Martian's `tune20` and on 59 Qodo issues a vendor found and the 2026-10-02 build missed, and validated on smoke10 and dev10; holdout10, fresh10, Martian's `test30` and 31 of Qodo's PRs were not used. On those 31 Qodo PRs its recall rose 7.8 points over the 2026-10-02 build [+2.1, +14.1] (Section 12).

Threat 1: the key was built partly from our reviewer's output (severe; measured). Recall is against the union of defects somebody in the candidate pool raised and a verifier confirmed. QuickReview default (on gpt-6-sol) raised 88% of the v1 R145 defects and 29 that nobody else did. *Mitigations:* anonymised, shuffled candidates with the key withheld; every rejection recorded (2,118 and 691); few additions from the extension. *Measured:* on the 369 defects the hunt or a reference raised, the final comparison's order at the top is unchanged (default 81%, `low` 74%, Claude Code 70%, the panel 69%; Section 8.6); in the earlier study the leave-one-reviewer-out key moved no reviewer's recall by more than 0.8 points (Section 9.2); on a key without the 28 defects only QuickReview's earlier runs raised, the final order is unchanged and the default leads the panel by 10.7 points (Section 8.6). *Open:* the other final reviewers were not re-scored on leave-own-out keys; a blind-first key built before any reviewer runs is the only way to remove pool bias by construction, the same bias pooled relevance judgements carry in IR [46, 48].

Threat 2: the final runs are outside the pool (moderate; indicated, not measured). No final gpt-6.1-sol run's comments were candidates for the key, so a real defect that only those runs found is credited to nobody (Section 8.8). The judge flags 219 such possible defects, 0.02 to 0.33 per PR per reviewer, against at most 2 per reviewer on R145 for the in-pool runs. Every final reviewer's recall is understated by its share; the default has the most flags in absolute

terms, three third-party tools the most per comment. The flags are unverified. *Open:* extending the key with the final runs' comments.

Threat 3: the competitors' configurations (moderate; measured in part). Each third-party configuration was picked as the best of two to four single runs on S40, then carried from gpt-6-sol to gpt-6.1-sol without re-selection. *Measured:* on O145 the picks move by −11 to +4 points and the order is unchanged (Table 7); the transfer held within run-to-run noise for four of five picks and failed for OpenCode, whose pick lost 10 points (Table 6). *Not measured, and plausibly against the tool:* Claude Code was not given the protocol at all. It stayed at its medium-effort `/code-review`, which its documentation describes as reporting only its most confident findings [10]; its read-only `git` commands were denied on 46 of 185 PRs; and it runs on a different model. Its 62% is therefore not a best-shot figure. *Open:* the candidate lists are what each tool's documentation ships or recommends, not an exhaustive search; OpenCode's other variants were not re-run on gpt-6.1-sol. Each configuration is our reading of that tool's documentation, not its maintainers' recommendation; corrections from maintainers will be published as errata.

Threat 4: no human-verified key and no human-validated judge (severe; unmeasured). Claude agents did every step of the key; critics changed 6 things in the first pass and 4 in the extension. No human has checked any part of the defect key, any judge verdict, or the AACR reference labels on the 30 original PRs, which agents produced (Section 3.2). If the verifier accepted plausible-but-wrong defects, recall is wrong for every reviewer, and the reviewers whose comments seeded those defects benefit; severity grades, which drive weighted recall and the high-severity results, are also unchecked. Two LLM judges agreeing is not agreement with humans [85, 86]. *Mitigation:* every defect carries quoted-code evidence and every exclusion a reason, both committed, so an audit is possible; a blinded audit of 20 defects and 5 exclusions is prepared (Section 5), and is smaller than the stratified sample that would bound the error rate where the lead is concentrated.

Threat 5: one designer for the reviewer, the key's definition and the judge (moderate; partly measured). The same assistant designed QuickReview's levers, wrote the admission rule for defects (including when maintainability counts), wrote the "counts if" notes and the judge prompt, and orchestrated the key. Independent-source subsets test whether QuickReview's comments *seeded* the key, not whether the key's *definition* of a defect matches QuickReview's design. The lead is concentrated where such an alignment would show: low severity and maintainability (Section 8). *Measured in part:* without the 72 maintainability defects the default still leads the panel (78% against 71%); on high and medium defects only, its lead over the panel is 2.3 points, 95% interval [−2.7, +7.0] (Section 8); the frozen build's lead extends to both (Section 8.7). *Open:* an admission rule and judge prompt written independently of QuickReview's design.

Threat 6: model-family overlap between judge, key and contestants (moderate). The primary judge (gpt-6-sol) shares a family with the seven OpenAI-based reviewers, including ours; the key builder and the second judge share a family with Claude Code. LLM judges can prefer their own family's outputs [31, 29]. In the earlier study a second judge from another family agreed on 98.7% of verdicts and moved no reviewer's recall (Section 6.2); the one visible same-family effect was on comments correct. That check was not repeated on the final comparison's reviews. The key half is not addressed by judge agreement at all.

Threat 7: unequal inputs, effort and budget (moderate; direction known). In the final comparison QuickReview, OpenCode and PR-Agent read the PR title and body; pi-review, open-code-review, Codex CLI and Claude Code do not (their harness invocations have no channel

for it, or their selection runs did better without it). QuickReview default spends 2.6 to 21 times the cost of each third-party tool and writes 5.1 to 24 times its output tokens; Claude Code runs a different and more expensive model. Most of these favour QuickReview on defects found; the cost column states the budget difference, and the companion report [1] varies QuickReview's own budget. A strict equal-budget comparison is not done.

Threat 8: per-tool harness deviations (moderate, per tool). Documented in Section 7.2. In the earlier study the pi-review and OpenCode deviations disfavoured those tools by 7 (OpenCode's bridge) and 10 points (pi-review's reasoning level) of recall on S40; the best-shot protocol removed them. The Claude Code parser mainly affects AACR numbers.

Threat 9: one run per reviewer (moderate; measured for QuickReview). Every reviewer in the final comparison ran once, and the bootstrap intervals cover PR sampling. For QuickReview we measured the run-to-run spread: a full reproduction on 185 PRs came within 1.5 defects, while six runs on the same 8 PRs found between 11 and 14 of 17 defects and two runs of an older build differed by 4–8 points on single 10-PR sets (Section 9.1).

Threat 10: metric construction (moderate for AACR, low for ours). AACR: reference quality, the line gate, a non-paper judge, and "human" references that are at least 22% rewritten review-bot comments (removing them moves F1 by at most 1.4 points and does not change the leaders; Table 12). Ours: partial credit as $\frac{1}{2}$ (counting it as zero moves no reviewer by a point); weighted recall's 3/2/1 weights are a convention; the same judge grades recall and comment accuracy; the judge sees exclusions written from the tools' own comments.

Threat 11: order of decisions (low). The model of the final comparison, gpt-6.1-sol, was chosen after QuickReview's 185-PR result on it was known. QuickReview finds 5 points *fewer* defects on it than on gpt-6-sol, so the choice does not favour it, but it was not fixed in advance. The selection rule and set were fixed before any selection run.

Threat 12: data handling (low). Eleven unrunnable PRs excluded for every arm; OCR under-credited on fresh10 in the key's construction; different benchmark-harness builds across arms (a later QuickReview build reproduced the original default run within 1.5 defects); runs stopped and resumed in chunks, and one stalled by an auto-suspend; one accidental local run of the real `claude` binary during parser work, in a container with no network and no credential, so no model call was made.

Threat 13: training-time contamination (unknown; cannot be measured here). AACR-Bench's PRs, their follow-up fixes and their review threads are public, so any reviewer's model may have seen them in training [42, 79]. The sandbox prevents only the review-time leak. Every reviewer is exposed to the same risk, but not necessarily equally. A key built on PRs merged after every model's cutoff, as time-segmented code benchmarks do [80, 78], would remove the threat.

Things we could not verify. The real billing price of gpt-6-sol, gpt-6.1-sol and gpt-6-luna behind the subscription login (every OpenAI-model cost is a list-price computation); wall-clock time per PR without contention; whether Claude Code's `/code-review` behaves the same on a subscription token as on an API key (we used an API key); whether a newer harness commit would run Codex or Claude Code differently.

13.1 What we claim and what we do not

We claim that on AACR-Bench's 185 runnable PRs, under our defect judge and our verified key, QuickReview's frozen build finds 97% of the high-severity defects, 94.5% of the high- and medium-severity ones and 89% of all 520, more than every other reviewer we ran at every severity, at the highest cost and time of the eight ($2.06 and a median 17.5 minutes per PR); and that on Martian's audited real goldens and on Qodo's functional bugs it has the highest recall of the tools scored by one judge, including the same three third-party harnesses on the same model, with the lowest Martian F1 and the fourth-highest Qodo F1, at $2.84 and $2.48 per PR against their $0.09–0.86. We claim that in the eight-reviewer comparison, with every third-party tool except Claude Code in the best configuration we found among those its documentation ships or recommends and every OpenAI-based reviewer on gpt-6.1-sol, QuickReview's default finds more of the key's defects than every other reviewer we ran (79% against 37–70%), with every paired difference on overall recall excluding zero over PR resampling; that it ranks first on the PRs not used to choose the configurations (O145), on the PRs not used to tune it (R145), and on the 369 defects the key would hold without any reviewer's comment; that its lead is concentrated in low-severity defects; and that it has the highest share of correct comments, a third of them minor. In the earlier study, with every tool as shipped on gpt-6-sol, it also ranked first, under a second judge from another model family as well, and a re-run on a later build reproduced its result. We claim that on AACR's own metric the reviewers are close, because that metric mostly rewards reproducing a list that is largely not real defects, and that every other reviewer is cheaper per defect found.

We do not claim a lead of the compared default on high-severity defects over pi-review's panel or Codex CLI; absolute recall; an exact size of the lead; that the final runs' recall is free of the out-of-pool undercount; that the best configurations we found are the best that exist for each tool, or that Claude Code ran at its best; that the lead comes from the multi-pass design rather than from budget or the PR description; a human-verified key or a human-validated judge; a like-for-like comparison at equal compute; or comparability with the published AACR leaderboard.

14 Reproducibility and data availability

Pinned inputs. Dataset commit and file hash (checked by `select_cases.py`, which refuses any other file); harness commit `68a5697`; every tool binary by hash (Codex CLI 0.159.2 by its sha256); pi 0.99.1 for every gpt-6.1-sol arm; reviewer builds by commit (`2d2036f`, `1ad710b`, `3581ebe`, `641f373`, `3bd89b3`, `42ec500` for the earlier study; `b2ce03a` for QuickReview in the final comparison); seeds and prompts by sha256 (`docs/PROMPTS.md` §4.1). The selection sets are derived by a committed rule (`bench/aacr/final/select40.py --check`).

Released. In the repository: the harness wrapper, every arm, the case sets, the defect keys (JSON) with all 3,093 exclusions and their reasons, the reference labels and the 80 bot-derived references, the scoring and chart scripts, the final comparison's launch and scoring scripts (`bench/aacr/final/`, which record the exact command, image and build behind every row), every prompt verbatim (key construction, both judges, our reviewer, each other reviewer, the parsers), and this paper. `bench/aacr/final/score.sh` recomputes every final table from the archived judge files with no model call, and `bench/aacr/README.md` ("Reproduce") gives the three ways in: re-score the slim bundle with no key, re-judge the archived reviews, or re-run every arm, on a ChatGPT login or on one's own API keys. The generated charts and the readable renderings of the keys are in the dataset. The file names some footnotes cite — `FULL-BENCHMARK.md`, `RESULTS.md`, `EXTERNAL-BENCHMARKS.md`, `MARTIAN-CRITIQUE.md`,

`RUNS.md`, `REVIEW-benchmark.md`, and the scratch analyses under `wp-data/` and `paper-data/` — are the study's working notes. They are not part of the public repository; every number this paper takes from them is stated here, and the run data they were computed from is in the run archive below.

Run archive. The full run directories (reviews, transcripts, judge files, eval metrics) are published as one archive on Hugging Face only, under a Hugging Face DOI (`huggingface.co/datasets/XXXX/XXXX`, DOI `10.57967/hf/XXXX`), with names rewritten to placeholders and model reasoning blobs blanked.

Local only. Extracted PR code and the key-construction analysis scripts. AACR-Bench is Apache-2.0, so the derived index, labels and key may ship with attribution; code quotes ship with per-repository attribution, and full checkouts are not redistributed.

Key datasheet (summary). Following the datasheet template [97]: *motivation*, an answer key of real defects for AACR-Bench v1.0's 185 runnable PRs; *composition*, 520 defects (90/145/285 by severity) with sites, evidence, "counts if" notes and provenance, plus 3,093 rejected candidates with reasons; *collection*, LLM agents (Claude Opus 5.5) pooling an expert hunt, the dataset's references and the comments of seven reviewers run in 2026-09, anonymised, verified against the code and critic-checked (Section 5); *known gaps*, no human audit yet, defects that only the final runs found are absent (Section 8.8), three CI-gate defects need CI knowledge; *intended use*, comparing reviewers' recall of real defects on these PRs alongside cost; *not intended for* absolute recall claims, or for comparison with numbers on the original reference metric; *maintenance*, versioned files, every verdict stored under the key version that produced it.

Judging cost and operations. Judging the 1,295 earlier-study reviews once cost $41.85 at list price with the primary judge (3,713 model calls, 13.4M uncached input, 16.2M cached input and 1.2M output tokens, or $0.035 per review) and $54.16 with the second judge (Opus 5.5 at $4/$20 per million input/output tokens, $0.2 cache read, $5 cache write; 2,448 calls, or $0.045 per review). The 96 reviews with neither defects nor comments needed no call.[33] The Opus judge ran into the organisation's rate limit (50 requests per minute; about 1.7 API calls per judgement); its HTTP 429 error files were re-run at lower parallelism until 0 errors remained in 1,295 judgements; the GPT judge's 2 error files were retried to 0. The AACR judge shim records no token usage; from the prompt length (about 425 tokens) and the number of line-gated pairs, the three-round AACR table for the earlier study's seven reviewers costs about $13–30 at gpt-6-sol rates ($13 for the calls in the surviving shim logs, $30 if every line-gated pair was judged in every round).[34]

What a re-run reproduces. The method, not the exact numbers: models are nondeterministic. Our own re-run of QuickReview default came within 1.5 defects of the original on 185 PRs, while repeats on 8 PRs spread by 3 of 17 (Section 9.1); the other reviewers ran once.

[33] Computed from the assistant `message_end` usage in the judges' event logs for the runs in `FULL-BENCHMARK.md` (`wp-data/cost.py`), 2026-09-29; recomputing from the raw token counts matches pi's own cost total to the cent.

[34] Estimated from the harness's judge prompt (`evaluation/judge.py`), the result files and the shim logs (`wp-data/cost.py`), 2026-09-30: 7,986 line-gated pairs per round is an upper bound on calls (23,958 over three rounds, about $0.00125 each); the surviving shim logs record 10,752 calls.

15 Conclusion

A code-review benchmark is only as good as its answer key. The answer key AACR-Bench ships, the comments its PRs received, mostly written or rewritten by models, is a poor list of those PRs' defects: most of its entries are not defects, most real defects are not on it, and at least a fifth of its "human" entries are rewritten review-bot comments. Replacing it with a verified, severity-graded defect key and a judge that reads whole reviews changes what the benchmark can see: eight reviewers that AACR's metric squeezes into a few points spread from 37% to 79% of the key's defects, the spread is stable across two judge families (on an earlier set of reviews) and across the PRs used and not used for any decision, and the best-shot protocol showed how much an evaluator can move a competitor by configuration alone (12 to 17 points for three tools on S40).

The authors' own reviewer, QuickReview, is designed for recall, and recall is where it leads. Its frozen build finds 97% of the key's high-severity defects and 89% of all of them, ahead of every reviewer compared at every severity; on Martian's audited real goldens and on Qodo's functional bugs it has the highest recall of the tools scored by one judge, ahead of the same three third-party harnesses on the same model on both. The price is stated beside it: $2.06 and a median 17.5 minutes per PR, 51% more and 1.8 times as long as the build of the eight-reviewer comparison and several times every third-party tool, and more comments, many of them true but minor, which gives it the lowest Martian F1 and the fourth-highest Qodo F1. The weakest link is the key: built by agents of one model family, partly seeded by that reviewer's earlier output, defined by the same assistant that designed it, not reaching the final runs' own finds, and not yet checked by a human. Until a human audit is published, the key's recall figures are best read as agreement with a careful, documented, machine-built answer key, not as ground truth.

15.1 Limitations and future work

The key. A blinded human audit, stratified by severity and provenance; extending the key with the final runs' comments, which would turn the out-of-pool flags into credited or rejected defects; and, to remove pool bias by construction, a key built from a blind hunt alone on PRs from outside AACR-Bench, merged after every model's training cutoff, before any tool runs. The AACR reference labels on the 30 original PRs need a human check too.

The judges. A human-graded sample of judge verdicts, and a re-judge with an independently written prompt and independently written "counts if" notes, which would test the bias the two current judges could share. On AACR's side, re-scoring with the paper's own judge, with a published leaderboard baseline re-run under both judges, would place our AACR numbers against the public leaderboard.

Where the gain comes from. No per-component ablation of QuickReview on R145; no QuickReview run without the PR text; no strict equal-budget arm; no Claude Code at a higher review level or with read-only `git` commands allowed; OpenCode's other variants not re-run on gpt-6.1-sol.

Cost and time. Every OpenAI-model cost is a list-price computation for runs made through a ChatGPT subscription; the Anthropic arms are API list prices. Wall-clock minutes were measured under a shared scheduler, never per PR in isolation.

References

[1] Mateusz Jakubczak. QuickReview: a multi-pass automated code reviewer, its design, its cost, and how it performs on a verified defect benchmark. Technical report, companion to this paper, 2026. arXiv:XXXX.XXXXX.

[2] L. Zhang, Y. Yu, M. Yu, X. Guo, Z. Zhuang, G. Rong, D. Shao, H. Shen, H. Kuang, Z. Li, B. Wang, G. Zhang, B. Xiang and X. Xu. AACR-Bench: Evaluating automatic code review with holistic repository-level context. arXiv:2601.19494 [cs.SE], 2026 (v3, 30 January 2026). `https://arxiv.org/abs/2601.19494`

[3] Alibaba. `alibaba/aacr-bench`: dataset and evaluation harness (Apache-2.0), commit `68a5697`, 24 August 2026. `https://github.com/alibaba/aacr-bench`

[4] Alibaba. Open Code Review (`@alibaba-group/open-code-review`), version 1.3.1 (npm, 10 June 2026); documentation at `main` (v1.12.11), Apache-2.0. `https://github.com/alibaba/open-code-review`

[5] OpenAI. Codex CLI (`@openai/codex`), versions 0.158.0 and 0.159.2, Apache-2.0. `https://github.com/openai/codex`

[6] OpenCode contributors. OpenCode (`opencode-ai`), version 1.18.33, MIT. Repository `anomalyco/opencode` (formerly `sst/opencode`). `https://github.com/anomalyco/opencode`

[7] Earendil Works. pi coding agent (`@earendil-works/pi-coding-agent`), versions 0.87.1 and 0.99.1, MIT. `https://github.com/earendil-works/pi`

[8] Z. Deng. pi-review (`@zephyrdeng/pi-review`), version 0.18.0, MIT. `https://github.com/ZephyrDeng/pi-review`

[9] Qodo and contributors. PR-Agent (`pr-agent` on PyPI), version 0.46.0, MIT. Repository `the-pr-agent/pr-agent` (formerly `qodo-ai/pr-agent`); documentation `https://docs.pr-agent.ai/`. `https://github.com/the-pr-agent/pr-agent`

[10] Anthropic. Claude Code (Anthropic's npm package), version 2.1.283. Documentation: "Code Review" (section "Review a diff locally") and "Model configuration" (effort levels), accessed 1 October 2026. `https://code.claude.com/docs/en/code-review`, `https://code.claude.com/docs/en/model-config`

[11] GitHub. Using GitHub Copilot code review. GitHub Docs, accessed 1 October 2026. `https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review`

[12] A. Bacchelli and C. Bird. Expectations, outcomes, and challenges of modern code review. In *Proc. 35th International Conference on Software Engineering (ICSE)*, pages 712–721, 2013. doi:10.1109/ICSE.2013.6606617

[13] C. Sadowski, E. Söderberg, L. Church, M. Sipko and A. Bacchelli. Modern code review: a case study at Google. In *Proc. 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 181–190, 2018. doi:10.1145/3183519.3183525

[14] M. V. Mäntylä and C. Lassenius. What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering*, 35(3):430–448, 2009. doi:10.1109/TSE.2008.71

[15] M. Beller, A. Bacchelli, A. Zaidman and E. Juergens. Modern code reviews in open-source projects: which problems do they fix? In *Proc. 11th Working Conference on Mining Software Repositories (MSR)*, pages 202–211, 2014. doi:10.1145/2597073.2597082

[16] S. McIntosh, Y. Kamei, B. Adams and A. E. Hassan. The impact of code review coverage and code review participation on software quality: a case study of the Qt, VTK, and ITK projects. In *Proc. 11th Working Conference on Mining Software Repositories (MSR)*, pages 192–201, 2014. doi:10.1145/2597073.2597076

[17] J. Czerwonka, M. Greiler and J. Tilford. Code reviews do not find bugs: how the current code review best practice slows us down. In *Proc. 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 27–28, 2015. doi:10.1109/ICSE.2015.131

[18] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu and N. Sundaresan. Automating code review activities by large-scale pre-training. In *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1035–1047, 2022. doi:10.1145/3540250.3549081. arXiv:2203.09095

[19] A. Naik, M. Alenius, D. Fried and C. Rose. CRScore: Grounding automated evaluation of code review comments in code claims and smells. In *Proc. 2025 Conference of the Nations of the Americas Chapter of the ACL (NAACL), Volume 1: Long Papers*, pages 9049–9076, 2025. `https://aclanthology.org/2025.naacl-long.457/`. arXiv:2409.19801

[20] Z. Zeng, R. Shi, K. Han, Y. Li, K. Sun, Y. Wang, Z. Yu, R. Xie, W. Ye and S. Zhang. SWR-Bench: Assessing LLM performance in real-world code review comment generation. arXiv:2509.01494 [cs.SE], 2025 (v2, 2026).

[21] R. Hu, X. Wang, X.-C. Wen, Z. Zhang, B. Jiang, P. Gao, C. Peng and C. Gao. Benchmarking LLMs for fine-grained code review with enriched context in practice. arXiv:2511.07017 [cs.SE], 2025.

[22] Y. Zhang, Z. Pan, I. N. B. Yusuf, H. Ruan, R. Shariffdeen and A. Roychoudhury. Code review agent benchmark. arXiv:2603.23448 [cs.SE], 2026.

[23] T. I. Khan, S. Wang, H. Zhang and T.-H. Chen. A survey of code review benchmarks and evaluation practices in pre-LLM and LLM era. arXiv:2602.13377 [cs.SE], 2026.

[24] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar and E. Tüzün. Automated code review in practice. In *Proc. 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 425–436, 2025. doi:10.1109/ICSE-SEIP66354.2025.00043. arXiv:2412.18531

[25] V. Karakaya, U. B. Torun, B. M. Uçar and E. Tüzün. Understanding the limits of automated evaluation for code review bots in practice. In *Proc. EASE 2026* (accepted); arXiv:2604.24525, 2026.

[26] Greptile. AI code review benchmarks 2025 (evaluation conducted July 2025). `https://www.greptile.com/benchmarks`, accessed 1 October 2026.

[27] T. Yanay and B. Fingerman (Qodo). How we built a real-world benchmark for AI code review. Qodo blog, 4 February 2026. `https://www.qodo.ai/blog/how-we-built-a-real-world-benchmark-for-ai-code-review/`

[28] DeepSource. Every AI code review vendor benchmarks itself, and wins. DeepSource blog, 26 February 2026. `https://deepsource.com/blog/ai-code-review-benchmarks`

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685

[30] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu and Z. Sui. Large language models are not fair evaluators. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 9440–9450, 2024. doi:10.18653/v1/2024.acl-long.511

[31] A. Panickssery, S. R. Bowman and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. arXiv:2404.13076

[32] P. Verga, S. Hofstätter, S. Althammer, Y. Su, A. Piktus, A. Arkhangorodsky, M. Xu, N. White and P. Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. arXiv:2404.18796 [cs.CL], 2024.

[33] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni and J. Guo. A survey on LLM-as-a-judge. arXiv:2411.15594 [cs.CL], 2024.

[34] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi:10.1177/001316446002000104

[35] J. R. Landis and G. G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi:10.2307/2529310

[36] B. Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:10.1214/aos/1176344552

[37] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993. doi:10.1007/978-1-4899-4541-9

[38] P. Koehn. Statistical significance tests for machine translation evaluation. In *Proc. 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 388–395, 2004. `https://aclanthology.org/W04-3250/`

[39] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770

[40] OpenAI and the SWE-bench team. Introducing SWE-bench Verified. 13 August 2024. `https://openai.com/index/introducing-swe-bench-verified/` (announcement recorded in `https://github.com/SWE-bench/SWE-bench`).

[41] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin and S. Wang. SWE-Bench+: Enhanced coding benchmark for LLMs. arXiv:2410.06992 [cs.SE], 2024.

[42] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle and E. Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10776–10787, 2023. doi:10.18653/v1/2023.findings-emnlp.722

[43] B. Recht, R. Roelofs, L. Schmidt and V. Shankar. Do ImageNet classifiers generalize to ImageNet? In *Proc. 36th International Conference on Machine Learning (ICML)*, PMLR 97, pages 5389–5400, 2019.

[44] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold and A. Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015. doi:10.1126/science.aaa9375

[45] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.

[46] J. Zobel. How reliable are the results of large-scale information retrieval experiments? In *Proc. 21st Annual International ACM SIGIR Conference*, pages 307–314, 1998. doi:10.1145/290941.291014

[47] C. Buckley and E. M. Voorhees. Retrieval evaluation with incomplete information. In *Proc. 27th Annual International ACM SIGIR Conference*, pages 25–32, 2004. doi:10.1145/1008992.1009000

[48] C. Buckley, D. Dimmick, I. Soboroff and E. Voorhees. Bias and the limits of pooling for large collections. *Information Retrieval*, 10:491–508, 2007. doi:10.1007/s10791-007-9032-x

[49] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk and G. Bavota. Using pre-trained models to boost code review automation. In *Proc. 44th International Conference on Software Engineering (ICSE)*, pages 2291–2302, 2022. doi:10.1145/3510003.3510621. arXiv:2201.06850

[50] R. Tufano, O. Dabić, A. Mastropaolo, M. Ciniselli and G. Bavota. Code review automation: strengths and weaknesses of the state of the art. *IEEE Transactions on Software Engineering*, 50(2):338–353, 2024. doi:10.1109/TSE.2023.3348172. arXiv:2401.05136

[51] C. Liu, H. Y. Lin and P. Thongtanunam. Too noisy to learn: enhancing data quality for code review comment generation. In *Proc. 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pages 236–248, 2025. doi:10.1109/MSR66628.2025.00043. arXiv:2502.02757

[52] J. Lu, X. Li, Z. Hua, L. Yu, S. Cheng, L. Yang, F. Zhang and C. Zuo. DeepCRCEval: revisiting the evaluation of code review comment generation. In *Fundamental Approaches to Software Engineering (FASE 2025)*, LNCS, pages 43–64. Springer, 2025. doi:10.1007/978-3-031-90900-9_3. arXiv:2412.18291

[53] H. Guo, X. Zheng, Z. Liao, H. Yu, P. Di, Z. Zhang and H.-N. Dai. CodeFuse-CR-Bench: a comprehensiveness-aware benchmark for end-to-end code review evaluation in Python projects. arXiv:2509.14856 [cs.SE], 2025.

[54] D. Kumar. SWE-PRBench: benchmarking AI code review quality against pull request feedback. arXiv:2603.26130 [cs.SE], 2026.

[55] Martian. Code review benchmark (offline benchmark: 50 pull requests, golden comments, extraction and judging pipeline). GitHub repository `withmartian/code-review-benchmark`, commit `e616e84`, MIT licence. `https://github.com/withmartian/code-review-benchmark`

[56] Martian. Code review benchmark leaderboard. `https://codereview.withmartian.com`, accessed 2026-10-02.

[57] K. Pereira, N. Sinha, R. Ghosh and D. Dutta. CR-Bench: evaluating the real-world utility of AI code review agents. arXiv:2603.11078 [cs.SE], 2026.

[58] D. Zheng, Y. Wang, X. Wang, K. Duan, H. Zhang, X. Liu, Y. Ma and Z. Zheng. From static to dynamic: benchmarking real-world code review with MCR-Bench. arXiv:2608.27442 [cs.SE], 2026 (accepted at ISSTA 2026 according to the arXiv record).

[59] M. Vijayvergiya, M. Salawa, I. Budiselić, D. Zheng, P. Lamblin, M. Ivanković, J. Carin, M. Lewko, J. Andonov, G. Petrović, D. Tarlow, P. Maniatis and R. Just. AI-assisted assessment of coding practices in modern code review. In *Proc. 1st ACM International Conference on AI-Powered Software (AIware)*, pages 85–93, 2024. doi:10.1145/3664646.3665664. arXiv:2405.13565

[60] T. Sun, J. Xu, Y. Li, Z. Yan, G. Zhang, L. Xie, L. Geng, Z. Wang, Y. Chen, Q. Lin, W. Duan, K. Sui and Y. Zhu. BitsAI-CR: automated code review via LLM in practice. In *Proc. 33rd ACM International Conference on the Foundations of Software Engineering (FSE), Companion*, pages 274–285, 2025. doi:10.1145/3696630.3728552. arXiv:2501.15134

[61] D. Olewicki, L. Da Silva, S. Mujahid, A. Amini, B. Mah, M. Castelluccio, S. Habchi, F. Khomh and B. Adams. Impact of LLM-based review comment generation in practice: a mixed open-/closed-source user study. arXiv:2411.07091 [cs.SE], 2024. (A revised version appeared in *IEEE Transactions on Software Engineering*, 2026, doi:10.1109/TSE.2026.3663093, under a different title.)

[62] K. Sun, H. Kuang, S. Baltes, X. Zhou, H. Zhang, X. Ma, G. Rong, D. Shao and C. Treude. Does AI code review lead to code changes? A case study of GitHub Actions. *IEEE Transactions on Software Engineering*, 52(7):2111–2126, 2026. doi:10.1109/TSE.2026.3688237. arXiv:2508.18771

[63] K. Chowdhury, D. Banik, K. M. Ferdous and S. I. Shamim. From industry claims to empirical reality: an empirical study of code review agents in pull requests. In *Proc. 23rd International Conference on Mining Software Repositories (MSR)*, pages 984–988, 2026. doi:10.1145/3793302.3793614. arXiv:2604.03196

[64] R. Just, D. Jalali and M. D. Ernst. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proc. 2014 International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi:10.1145/2610384.2628055

[65] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo and E. L. Ouh. BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proc. 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1556–1560, 2020. doi:10.1145/3368089.3417943

[66] C. Le Goues, N. Holtschulte, E. K. Smith, Y. Brun, P. Devanbu, S. Forrest and W. Weimer. The ManyBugs and IntroClass benchmarks for automated repair of C programs. *IEEE Transactions on Software Engineering*, 41(12):1236–1256, 2015. doi:10.1109/TSE.2015.2454513

[67] D. A. Tomassi, N. Dmeiri, Y. Wang, A. Bhowmick, Y.-C. Liu, P. T. Devanbu, B. Vasilescu and C. Rubio-González. BugSwarm: mining and continuously growing a dataset of reproducible failures and fixes. In *Proc. 41st IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 339–349, 2019. doi:10.1109/ICSE.2019.00048

[68] T. Durieux and R. Abreu. Critical review of BugSwarm for fault localization and program repair. arXiv:1905.09375 [cs.SE], 2019.

[69] A. Habib and M. Pradel. How many of all bugs do we find? A study of static bug detectors. In *Proc. 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, pages 317–328, 2018. doi:10.1145/3238147.3238213

[70] K. Herzig, S. Just and A. Zeller. It's not a bug, it's a feature: how misclassification impacts bug prediction. In *Proc. 35th International Conference on Software Engineering (ICSE)*, pages 392–401, 2013. doi:10.1109/ICSE.2013.6606585

[71] S. Herbold, A. Trautsch, F. Trautsch and B. Ledel. Problems with SZZ and features: an empirical study of the state of practice of defect prediction data collection. *Empirical Software Engineering*, 27(2):42, 2022. doi:10.1007/s10664-021-10092-4

[72] R. Croft, M. A. Babar and M. M. Kholoosi. Data quality for software vulnerability datasets. In *Proc. 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 121–133, 2023. doi:10.1109/ICSE48619.2023.00022. arXiv:2301.05456

[73] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray and Y. Chen. Vulnerability detection with code language models: how far are we? In *Proc. 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1729–1741, 2025. doi:10.1109/ICSE55347.2025.00038. arXiv:2403.18624

[74] C. G. Northcutt, A. Athalye and J. Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. arXiv:2103.14749

[75] Y. Wang, M. Pradel and Z. Liu. Are "solved issues" in SWE-bench really solved correctly? An empirical study. In *Proc. 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 169–181, 2026. doi:10.1145/3744916.3764576. arXiv:2503.15223

[76] B. Yu, Y. Zhu, P. He and D. Kang. UTBoost: rigorous evaluation of coding agents on SWE-Bench. In *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 3762–3774, 2025. doi:10.18653/v1/2025.acl-long.189

[77] S. Liang, S. Garg and R. Zilouchian Moghaddam. The SWE-Bench illusion: when state-of-the-art LLMs remember instead of reason. In *Proc. 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 395–405, 2026. doi:10.1145/3786583.3786882. arXiv:2506.12286

[78] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan and D. Zhang. SWE-bench goes live! In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. arXiv:2505.23419

[79] M. Riddell, A. Ni and A. Cohan. Quantifying contamination in evaluating code generation capabilities of language models. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 14116–14137, 2024. doi:10.18653/v1/2024.acl-long.761

[80] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen and I. Stoica. LiveCodeBench: holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2403.07974

[81] T. Ahmed, P. Devanbu, C. Treude and M. Pradel. Can LLMs replace manual annotation of software engineering artifacts? In *Proc. 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pages 526–538, 2025. doi:10.1109/MSR66628.2025.00086. arXiv:2408.05534

[82] R. Wang, J. Guo, C. Gao, G. Fan, C. Y. Chong and X. Xia. Can LLMs replace human evaluators? An empirical study of LLM-as-a-judge in software engineering. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1955–1977, 2025. doi:10.1145/3728963. arXiv:2502.06193

[83] W. Tong and T. Zhang. CodeJudge: evaluating code generation with large language models. In *Proc. 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 20032–20051, 2024. doi:10.18653/v1/2024.emnlp-main.1118

[84] T. Y. Zhuo. ICE-Score: instructing large language models to evaluate code. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 2232–2242, 2024. doi:10.18653/v1/2024.findings-eacl.148

[85] A. Bavaresco, R. Bernardi, L. Bertolazzi, D. Elliott, R. Fernández, A. Gatt, E. Ghaleb, M. Giulianelli, M. Hanna, A. Koller, A. F. T. Martins, P. Mondorf, V. Neplenbroek, S. Pezzelle, B. Plank, D. Schlangen, A. Suglia, A. K. Surikuchi, E. Takmaz and A. Testoni. LLMs instead of human judges? A large scale empirical study across 20 NLP evaluation tasks. In *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 2: Short Papers*, pages 238–255, 2025. doi:10.18653/v1/2025.acl-short.20

[86] S. Shankar, J. D. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran and I. Arawjo. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. In *Proc. 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2024. doi:10.1145/3654777.3676450

[87] A. S. Thakur, K. Choudhary, V. S. Ramayapally, S. Vaidyanathan and D. Hupkes. Judging the judges: evaluating alignment and vulnerabilities in LLMs-as-judges. In *Proc. Fourth Workshop on Generation, Evaluation and Metrics (GEM^2)*, pages 404–430, 2025. arXiv:2406.12624

[88] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer and H. Hajishirzi. FActScore: fine-grained atomic evaluation of factual precision in long form text generation. In *Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100, 2023. doi:10.18653/v1/2023.emnlp-main.741

[89] A. R. Feinstein and D. V. Cicchetti. High agreement but low kappa: I. The problems of two paradoxes. *Journal of Clinical Epidemiology*, 43(6):543–549, 1990. doi:10.1016/0895-4356(90)90158-L

[90] R. Artstein and M. Poesio. Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi:10.1162/coli.07-034-R2

[91] A. F. Hayes and K. Krippendorff. Answering the call for a standard reliability measure for coding data. *Communication Methods and Measures*, 1(1):77–89, 2007. doi:10.1080/19312450709336664

[92] K. Spärck Jones and C. J. van Rijsbergen. Report on the need for and provision of an 'ideal' information retrieval test collection. British Library Research and Development Report 5266, Computer Laboratory, University of Cambridge, 1975.

[93] E. M. Voorhees. Variations in relevance judgments and the measurement of retrieval effectiveness. *Information Processing & Management*, 36(5):697–716, 2000. doi:10.1016/S0306-4573(00)00010-8

[94] G. Faggioli, L. Dietz, C. L. A. Clarke, G. Demartini, M. Hagen, C. Hauff, N. Kando, E. Kanoulas, M. Potthast, B. Stein and H. Wachsmuth. Perspectives on large language models for relevance judgment. In *Proc. 2023 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR)*, pages 39–50, 2023. doi:10.1145/3578337.3605136

[95] P. Thomas, S. Spielman, N. Craswell and B. Mitra. Large language models can accurately predict searcher preferences. In *Proc. 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1930–1940, 2024. doi:10.1145/3626772.3657707

[96] C. L. A. Clarke and L. Dietz. LLM-based relevance assessment still can't replace human relevance assessment. In *Proc. Tenth International Workshop on Evaluating Information Access (EVIA 2025), NTCIR-18 satellite workshop*, 2025. doi:10.20736/0002002105. arXiv:2412.17156

[97] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III and K. Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi:10.1145/3458723. arXiv:1803.09010

[98] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy and M. J. Kochenderfer. BetterBench: assessing AI benchmarks, uncovering issues, and establishing best practices. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. arXiv:2411.12990

[99] Y. Zhu, T. Jin, Y. Pruksachatkun, A. K. Zhang, S. Liu, S. Cui, S. Kapoor, S. Longpre, K. Meng, R. Weiss, F. Barez, R. Gupta, J. Dhamala, J. Merizian, M. Giulianelli, H. Coppock, C. Ududec, A. Kellermann, J. S. Sekhon, J. Steinhardt, S. Schwettmann, A. Narayanan, M. Zaharia, I. Stoica, P. Liang and D. Kang. Establishing best practices in building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. arXiv:2507.02825

[100] I. D. Raji, E. Denton, E. M. Bender, A. Hanna and A. Paullada. AI and the everything in the whole wide world benchmark. In *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. arXiv:2111.15366

[101] G. Melis, C. Dyer and P. Blunsom. On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1707.05589

[102] M. Lucic, K. Kurach, M. Michalski, S. Gelly and O. Bousquet. Are GANs created equal? A large-scale study. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. arXiv:1711.10337

[103] J. Dodge, S. Gururangan, D. Card, R. Schwartz and N. A. Smith. Show your work: improved reporting of experimental results. In *Proc. 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2185–2194, 2019. doi:10.18653/v1/D19-1224. arXiv:1909.03004

[104] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux and P. Vincent. Accounting for variance in machine learning benchmarks. In *Proc. Machine Learning and Systems (MLSys)*, volume 3, 2021. arXiv:2103.03098

[105] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald and D. Jurafsky. With little power comes great responsibility. In *Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9263–9274, 2020. doi:10.18653/v1/2020.emnlp-main.745. arXiv:2010.06595

[106] A. Arcuri and L. Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proc. 33rd International Conference on Software Engineering (ICSE)*, pages 1–10, 2011. doi:10.1145/1985793.1985795

[107] A. Lundh, J. Lexchin, B. Mintzes, J. B. Schroll and L. Bero. Industry sponsorship and research outcome. *Cochrane Database of Systematic Reviews*, 2017(2):MR000033, 2017. doi:10.1002/14651858.MR000033.pub3

[108] S. Singh, Y. Nan, A. Wang, D. D'Souza, S. Kapoor, A. Üstün, S. Koyejo, Y. Deng, S. Longpre, N. A. Smith, B. Ermis, M. Fadaee and S. Hooker. The leaderboard illusion. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025. arXiv:2504.20879

[109] G. Klees, A. Ruef, B. Cooper, S. Wei and M. Hicks. Evaluating fuzz testing. In *Proc. 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2123–2138, 2018. doi:10.1145/3243734.3243804. arXiv:1808.09700

[110] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench and T. Holz. SoK: prudent evaluation practices for fuzzing. In *Proc. 2024 IEEE Symposium on Security and Privacy (SP)*, pages 1974–1993, 2024. doi:10.1109/SP54263.2024.00137. arXiv:2405.10220

[111] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir and A. Narayanan. AI agents that matter. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2407.01502

A Prompts (summary)

The full text of every prompt that produced a number is in `bench/aacr/docs/PROMPTS.md`, rendered from the saved workflow scripts, not retyped. Summary:

- **Ground truth, first 30 PRs**: one blind hunter per PR, one consolidator merging the hunt, labelled references and anonymised reviewer comments, one critic per set; v2 adjudicator for the judge's "new defect" candidates plus a critic.
- **fresh10**: four hunt agents, one finalise agent checking hunt drafts (H), anonymised reviewer comments (C) and references (A), instructed to be even-handed between hunt and reviewer candidates.

- **rest145**: the stopped blind hunt; the final hunt with references as leads (never reading run outputs); the independent verifier (never opening the candidate key); the critic; a single-PR re-verification; the four-reviewer extension verifier and critic ("you only ADD").
- **Defect judge**: the prompt in Section 6, identical on both judge models.
- **AACR judge**: the harness's own yes/no "same concern" prompt, unmodified, answered through the local shim.
- **QuickReview**: the production session system prompt and round-first order, the addendum registry (42 addenda; the ones written during this work are `quality-cap-validate`, `validate-demote-not-kill`, `every-problem-on-the-lines`, `validate-restate-not-kill`, and two `nits-open` addenda that are off in every scored run), the quality lenses, and the BENCHMARK MODE override: no GitHub, CI gate not applicable, decide on the merits, write `comments.json`, `review.md`, `findings-record.json` and `outcome.json` instead of posting; the PR title and body are included as untrusted data.
- **Other reviewers**: OCR's command line (its prompts are in a compiled binary); the harness's Codex and Claude runners; OpenCode's built-in review with the Codex findings fragment; pi-review's own review contract.
- **Parsers**: deterministic, no model: per-arm conversion to {path, line range, note}; the Claude Code prose rules.

The defect-judge prompt

The prompt below is the one every scored judgement used (judge label `v2-gpt-6-sol-high`; `judgePrompt` in `src/bench/defects.ts`, unchanged since ground truth v2). It is reproduced verbatim except that each interpolated value is shown as an `<UPPERCASE>` placeholder. Each defect renders as `<ID> [<severity>, <kind>] <title>`, then its statement, its sites (`path:from-to (role)`) and, when present, its "counts if" note; each comment as `<ID> [<tag>] path:line`, then its text. The rejected-claims block appears only when the PR has rejected claims; the body line only when the review has a body.

```
You are the judge of a code-review benchmark that measures one thing: does a review FIND the real defects
 in a pull request and point them out, and is what it says true? Wording, tone, tags and exact line
numbers do not matter; substance does.

THE PULL REQUEST: <TITLE> - <REPO>, head <HEAD_SHA>, merge base <MERGE_BASE_SHA>. The repository is
checked out at the PR head in your working directory. You may read any code (read tool; bash for git diff
/show/log/grep, rg, ls, find, sed -n, head, tail, wc). Never run project code.

THE REAL DEFECTS (ground truth - each verified against the code; trust these):
<DEFECTS, or: (none - this PR has no real defect in the ground truth)>

CLAIMS ALREADY CHECKED AGAINST THE CODE AND REJECTED (not defects - each with the reason it failed). A
comment repeating one of these is never a new defect: judge it incorrect, partly-correct or correct-minor
 as its reason says, reading the code if the comment adds something the reason does not cover:
<REJECTED CLAIMS, one per line: - <SOURCE>: <REASON>>

THE REVIEW (what the author received):
Verdict: <VERDICT, or (none)>
Body:
<BODY>
Comments:
<COMMENTS, or (no comments)>

YOUR TASK
1. For EACH defect decide whether the review found it:
   - found: some comment (or the body) identifies THIS defect - the same underlying problem, with its
essential point right (see "counts if"), so the author would fix it. It may sit at the cause or the
symptom site, or name the site in its text.
```

```
   - partial: the review points at this problem or its site, but with a wrong or missing mechanism or
consequence, so the author could fix the wrong thing.
   - missed: nothing in the review identifies it.
2. For EACH comment: which defect it finds (its id, or null), and whether it is correct. Check every
comment that finds no listed defect against the code:
   - correct-valuable: true at the head and worth raising before merge;
   - correct-minor: true but low value (style, a test request, a nice-to-have, a nit);
   - partly-correct: a real problem, but the mechanism, scope or severity is materially wrong;
   - incorrect: false at the head (the code does not do what it says, or the scenario cannot happen);
   - unverifiable: cannot be decided from the repository.
   A comment that finds a listed defect is correct-valuable, or partly-correct if it found it only
partially. Set new_defect true only for a correct-valuable comment that describes a real defect that is
neither in the list nor among the rejected claims.
Be strict both ways: do not credit a comment for sounding plausible, and do not dismiss one without
reading the code it cites.

Answer with ONE JSON object and nothing after it:
{"defects":[{"id":"D1","verdict":"found|partial|missed","comments":["C2"],"why":"one sentence"}],"
comments":[{"id":"C1","defect":"D1","verdict":"correct-valuable|correct-minor|partly-correct|incorrect|
unverifiable","new_defect":false,"why":"one sentence with file:line"}]}
Every defect id and every comment id above must appear exactly once.
```

B Per-set tables

Table 28: Case sets. Selection rules abridged; the full rule string is stored in each set file.

Set	PRs	Refs	Rule (abridged)	Role
smoke10	10	62	20–160 changed lines, ≥4 refs, language quota, smallest first, one PR per repo	development
holdout10	10	85	same filters, no smoke10 repo, different quota	development (was a holdout)
dev10	10	106	20–400 lines, no smoke10/holdout10 repo, 9-language quota	development
fresh10	10	58	60–400 lines, none of the above, no style/docs/CI-only categories	out-of-sample, then used for design
rest145	145	1,107	every other runnable scored PR	never used for design

Defects per original set: smoke10 34, dev10 37, holdout10 26, fresh10 29 (126 total, 34 high). Table 29 gives problems found per set in the final comparison, and Tables 30 and 31 in the earlier gpt-6-sol study, under each judge.

Table 29: Problems found per set, final comparison, primary judge (recall in brackets).

Reviewer	smoke10 (34)	dev10 (37)	holdout10 (26)	fresh10 (29)	rest145 (394)
QuickReview default	26 (76%)	23.5 (64%)	17 (65%)	21.5 (74%)	321 (81%)
QuickReview low	23 (68%)	21.5 (58%)	16 (62%)	20.5 (71%)	284.5 (72%)
pi-review panel (high)	19.5 (57%)	18.5 (50%)	16.5 (63%)	23 (79%)	270 (69%)
Claude Code (Opus 5.5)	22 (65%)	17 (46%)	16 (62%)	20 (69%)	249.5 (63%)
open-code-review high	16 (47%)	14 (38%)	6.5 (25%)	20 (69%)	242 (61%)
Codex CLI high	17 (50%)	15 (41%)	14 (54%)	15 (52%)	226 (57%)
OpenCode native + PR text	14 (41%)	14 (38%)	14 (54%)	18 (62%)	203.5 (52%)
PR-Agent	13 (38%)	12 (32%)	7.5 (29%)	11.5 (40%)	150 (38%)

The earlier study's per-set figures:[35] Per-set numbers on 10 PRs move by about 2 defects between identical runs.

Table 30: Problems found per set, earlier gpt-6-sol study, primary judge (recall in brackets).

Reviewer	smoke10 (34)	dev10 (37)	holdout10 (26)	fresh10 (29)	rest145 (394)
QuickReview default	25 (74%)	26.5 (72%)	23 (88%)	27.5 (95%)	335.5 (85%)
QuickReview low	18 (53%)	16.5 (45%)	14 (54%)	24.5 (84%)	272 (69%)
open-code-review	12.5 (37%)	17 (46%)	8 (31%)	19.5 (67%)	234.5 (60%)
Codex CLI	16 (47%)	10 (27%)	15 (58%)	21 (72%)	186.5 (47%)
OpenCode	13.5 (40%)	14 (38%)	16 (62%)	14.5 (50%)	192 (49%)
pi-review	12.5 (37%)	17 (46%)	16.5 (63%)	21.5 (74%)	225 (57%)
Claude Code (Opus 5.5)	22 (65%)	17 (46%)	16 (62%)	20 (69%)	249.5 (63%)

Table 31: Problems found per set, earlier gpt-6-sol study, second judge (Claude Opus 5.5; recall in brackets).[37]

Reviewer	smoke10 (34)	dev10 (37)	holdout10 (26)	fresh10 (29)	rest145 (394)
QuickReview default	25 (74%)	27 (73%)	23.5 (90%)	27.5 (95%)	336 (85%)
QuickReview low	18 (53%)	17 (46%)	14 (54%)	24.5 (84%)	272 (69%)
open-code-review	13 (38%)	16.5 (45%)	8 (31%)	19.5 (67%)	234 (59%)
Codex CLI	16 (47%)	10 (27%)	15 (58%)	21 (72%)	185 (47%)
OpenCode	13.5 (40%)	14 (38%)	16 (62%)	14.5 (50%)	191.5 (49%)
pi-review	12.5 (37%)	17 (46%)	16.5 (63%)	21.5 (74%)	222 (56%)
Claude Code (Opus 5.5)	20.5 (60%)	16.5 (45%)	15.5 (60%)	20 (69%)	249 (63%)

C The earlier study: supporting tables

These tables belong to the earlier study (Section 9): every third-party tool as shipped, gpt-6-sol except Claude Code (Opus 5.5), primary judge unless stated. They are kept because the main text refers to them; the earlier study's per-language, per-kind, per-size, cost and AACR-category breakdowns are in `FULL-BENCHMARK.md`.

AACR matches by reference label. On the 30 labelled PRs, the AACR-metric matches per label, for all seven reviewers of the earlier study, are in Table 34.

[35] Computed from the primary judge's verdict files for the runs in `FULL-BENCHMARK.md`, 2026-09-28.

[37] Computed from the second judge's verdict files (`judge/v<ver>-claude-opus-5-5/`) for the runs in `FULL-BENCHMARK.md` (`wp-data/size_sets.py`), 2026-09-29.

Table 32: Recall with 95% bootstrap intervals over PRs (primary judge), high-severity recall with the same, and cost per defect found (list prices; total cost over 185 PRs divided by problems found).

Reviewer	Recall [95% CI]	High recall [95% CI]	$/defect	$/high defect
QuickReview default	84.1% [81.3, 87.0]	95.6% [90.9, 99.0]	0.72	3.68
QuickReview low	66.3% [61.8, 70.9]	83.3% [76.1, 90.2]	0.43	1.97
open-code-review	56.1% [50.6, 61.4]	71.7% [59.8, 83.1]	0.38	1.72
Codex CLI	47.8% [42.9, 52.8]	77.8% [70.3, 85.3]	0.11	0.40
OpenCode	48.1% [43.8, 52.2]	71.7% [62.0, 81.0]	0.62	2.38
pi-review	56.2% [52.2, 60.5]	79.4% [71.6, 87.5]	0.07	0.27
Claude Code (Opus 5.5)	62.4% [58.6, 66.5]	80.6% [71.0, 89.8]	0.11	0.50

Table 33: Comment grades in the gpt-6-sol study, primary judge, all 185 PRs. Shares of each reviewer's comments. Computed from the primary judge's verdict files, 2026-09-28. (Low and pi-review have two and one more judged comments than their AACR comment counts, 824 and 395.)

Reviewer	Comments	Per PR	Valuable	Minor	Partly	Incorrect	Unverif.
QuickReview default	1,191	6.4	44.8%	41.4%	4.4%	9.2%	0.2%
QuickReview low	826	4.5	45.8%	33.9%	6.4%	13.9%	0.0%
open-code-review	688	3.7	54.8%	18.5%	6.4%	20.3%	0.0%
Codex CLI	390	2.1	64.1%	5.4%	4.9%	25.4%	0.3%
OpenCode	557	3.0	51.0%	4.3%	15.1%	29.4%	0.2%
pi-review	396	2.1	67.2%	11.6%	7.3%	13.9%	0.0%
Claude Code (Opus 5.5)	407	2.2	75.9%	4.7%	6.9%	11.8%	0.7%

Table 34: AACR-metric matches by reference label (references matched, averaged over the 3 judge rounds), 30 labelled PRs, all 253 references. Recomputed 2026-09-28 from each run's `results/all/metrics_ocr_*_round_*.json` and `sets/*.labels.json`, aligning labels to references by position and checking path and start line as `src/bench/report.ts` does (all 253 align). Supersedes the 225-reference table in `FULL-BENCHMARK.md`.

Reviewer	Real defect (29)	Real maint. (20)	Dubious (99)	Trivial (77)	Duplicate (28)
QuickReview default	**15.3**	**6.0**	1.0	3.3	2.0
QuickReview low	11.7	4.0	1.0	3.0	0.0
open-code-review	14.3	2.0	3.0	3.0	2.0
Codex CLI	12.3	0.0	1.0	0.0	0.0
OpenCode	14.7	1.0	0.0	0.0	0.0
pi-review	12.0	1.0	2.0	0.0	0.0
Claude Code (Opus 5.5)	11.7	0.0	1.3	1.0	0.0

On the list's 29 genuine defects the seven reviewers are close (11.7–15.3 matched), line gate included; on the 20 genuine maintainability issues default leads (6.0 against at most 4.0). None of the seven chases the roughly 70% of references that are dubious or trivial (at most 6.0 of 176 matched). With 29 and 20 references these differences are small in absolute terms.

Without the bot-derived references. Table 35 re-scores the earlier study's AACR table with the 80 bot-derived references (Section 3.2) removed, leaving 1,338. No model was called: the harness's per-round files record, for every reference, whether it was matched and which review comment matched it (`matched_note`), so the metric is recomputed from the published evaluation (`compare.py --exclude-refs bot_references.json`). A comment whose only match was a removed reference now matches nothing; we count it as unmatched, which is what the metric would have done had the reference never been on the list. Dropping such comments from the count instead changes precision by at most 0.4 points.

Every reviewer loses matches, most of all those that match the list most: default loses 23, low 18, OCR 16, OpenCode 11, pi-review 8.3, Codex CLI 5 and Claude Code 4. The bot-derived references are easier to match than the rest (default matched 23 of the 80, 29%, against 11% of the other 1,338), which fits their being model-written. The ranking barely moves: default keeps the highest recall and OCR the highest precision and F1. The only changes are at the bottom, where Claude Code passes pi-review on F1 (8.8% against 8.5%) and pi-review and OpenCode on recall, all by less than half a point.

D Chronology of result-affecting decisions

Table 35: AACR-Bench metric with the 80 bot-derived references removed, before → after; 185 PRs, 1,418 → 1,338 references; same evaluation as Section 6.3. The last two rows are the default on the current build and on gpt-6.1-sol (the companion report [1]).

Reviewer	Refs found	Recall	Precision	F1
QuickReview default	164.3 → 141.3	11.6 → **10.6**%	13.8 → 11.9%	12.6 → 11.2%
QuickReview low	133.0 → 115.0	9.4 → 8.6%	16.1 → 14.0%	11.9 → 10.6%
open-code-review	147.3 → 131.3	10.4 → 9.8%	21.4 → **19.1**%	14.0 → **13.0**%
Codex CLI	62.7 → 57.7	4.4 → 4.3%	16.1 → 14.8%	6.9 → 6.7%
OpenCode	83.0 → 72.0	5.9 → 5.4%	14.9 → 12.9%	8.4 → 7.6%
pi-review	82.3 → 74.0	5.8 → 5.5%	20.8 → 18.7%	9.1 → 8.5%
Claude Code (Opus 5.5)	81.0 → 77.0	5.7 → 5.8%	19.9 → 18.9%	8.9 → 8.8%
QuickReview default, current build	166.0 → 148.0	11.7 → 11.1%	13.5 → 12.1%	12.6 → 11.5%
QuickReview default, gpt-6.1-sol	151.7 → 132.7	10.7 → 9.9%	15.2 → 13.3%	12.5 → 11.4%

Table 36: Defect benchmark split into T40, used for tuning QuickReview, and R145, unseen. Primary judge; Opus judge in brackets where it differs.

	T40 (126 defects, 34 high)			R145 (394 defects, 56 high)		
Reviewer	Found	High	Correct	Found	High	Correct
QuickReview default	102 (81%) [103]	32	89% [87%]	335.5 (85%) [336]	54	86%
QuickReview low	73 (58%) [73.5]	29	79% [83%]	272 (69%)	46 [46.5]	80% [81%]
open-code-review	57 (45%)	24	73% [77%]	234.5 (60%) [234]	40.5	73% [78%]
Codex CLI	62 (49%)	27.5	87% [84%]	186.5 (47%) [185]	42.5	65% [66%]
OpenCode	58 (46%)	24.5	76% [63%]	192 (49%) [191.5]	40	50% [53%]
pi-review	67.5 (54%)	27.5 [28]	82% [80%]	225 (57%) [222]	44	78% [79%]
Claude Code (Opus 5.5)	75 (60%) [72.5]	25.5	72% [85%]	249.5 (63%) [249]	47	83% [86%]

Table 37: Chronology (2026). "Tuned on" says which PRs informed each decision.

When	What	Tuned on
09-25	Harness wrapper, OCR arm, smoke10/holdout10/dev10, reference labels	—
09-25/26	Review iterations 1–2 (validator premises, sibling sites, judgment lenses)	smoke10 AACR misses (before the defect benchmark existed)
09-26	Defect judge; ground truth v1/v2 for the 30	—
09-26	Levers: effort, sub-lenses, sweep, dependency sources, restate-not-kill	missed defects on the 30 (holdout10 stops being a holdout)
09-26	fresh10 created as out-of-sample check; its truth v1	—
09-27	Lean plan; presets default/low/max	16 of the 30 + fresh10 cost/recall
09-27	Tests lens; every-problem-on-the-lines	high-severity misses on fresh10 + the 30 (fresh10 used for design from here)
09-27	Code-quality lenses	9 PRs (3 fresh10 + 6 of the 30), both metrics
09-27	Validator demote-not-kill, `[review] nits`, pass-consistency	3 fresh10 PRs
09-27 23:34	**rest145 created**; default/low/OCR runs start on `1ad710b`	— (no reviewer change after this)
09-28	Four new arms, Claude prose parser, key extension (+15), re-judging, second judge	— (no reviewer behaviour change)
09-28/29	Three reviewer changes (optional validator with a doubting veto, stack-neutral frozen passes, style guide from directive files)	— (setup-and-scan work)
09-29/30	Current-build re-run of default (`9106a64`), stability repeats, base-model arms	— (measurement only)
09-30	Fairness audit; 40-PR selection set fixed by rule; every documented variant run once on gpt-6-sol; picks by the pre-set rule	— (competitor configurations; QuickReview unchanged)
09-30/10-01	Every OpenAI-based contender on gpt-6.1-sol; picks re-run unchanged on all 185 PRs	— (measurement only)